andrea

GESTIONE DELLA MEMORIA CENTRALE

Un programma per essere eseguito deve essere eseguito deve essere, almeno parzialmente, presenti in memoria centrale.

Il gestore della memoria:

- si occupa di allocare memoria fisica ai processi.
- in un ambiente multiprogrammato deve garantire la protezione dei dati, impedendo ai processi attivi di sconfinare nello spazio di indirizzamento di altri processi.
- in un ambiente multiprogrammato deve permettere meccanismi di condivisione affinchè i processi cooperanti possano accedere ad altre aree comuni di memoria.

L'utilizzo globale delle risorse e le prestazioni di un calcolatore vengono influenzate dalle prestazioni del *modulo di gestione* sia in funzione della sua efficienza nell'allocare memoria, sia che può avere sullo *scheduling*.

INTRODUZIONE

Un programma, prima di essere eseguito, attraversa diversi stadi, alcuni dei quali opzionali. Questi stadi includono:

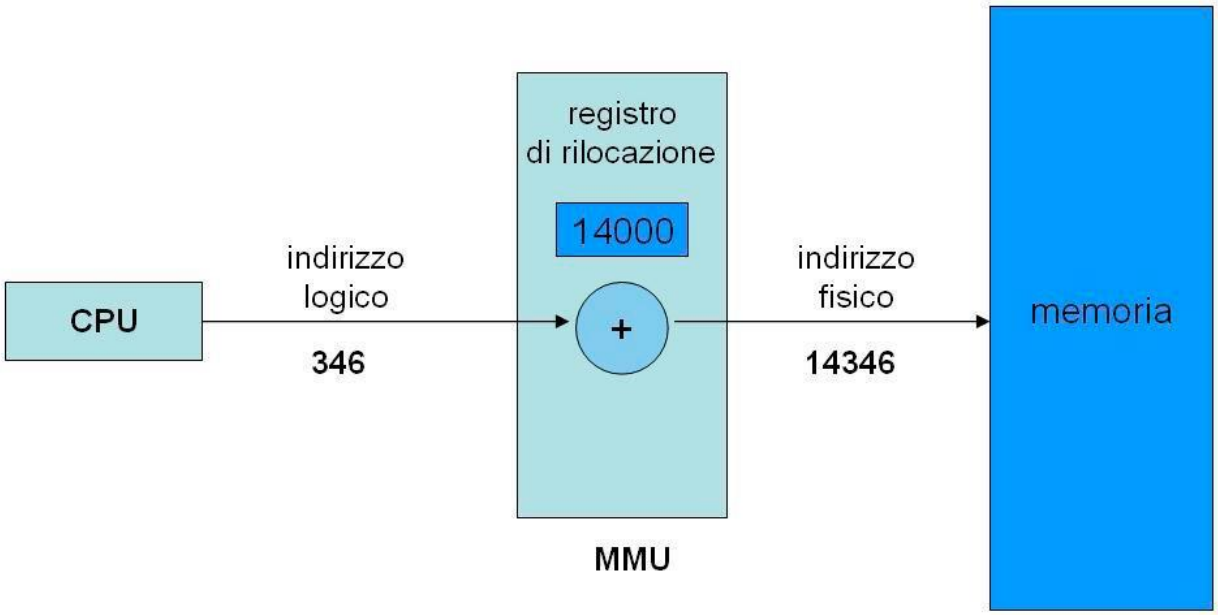
- **Programma sorgente con indirizzi simbolici**:
  - All'inizio, il programma è scritto in un linguaggio di programmazione con indirizzi simbolici, cioè variabili come `a`, `x`, `sommatore`,`contatore.
- **Compilazione**:
  - Durante la fase di compilazione, il programma sorgente viene tradotto in un modulo compilato con indirizzi rilocabili. Questi indirizzi non sono ancora definitivi e sono espressi come **offset relativi**, ad esempio `+14 bytes` dall'inizio del modulo.
- **Caricamento e Linkage Editore**:
  - Il caricatore (loader) ha il compito di assegnare indirizzi assoluti agli indirizzi rilocabili generali durante la compilazione. Questo avviene dopo l'intervento dell'editor dei collegamenti (linker editor), che può combinare più moduli compilati in un unico eseguibile.

GENERAZIONE DI INDIRIZZI DI MEMORIA (ASSOLUTI) NEI VARI STADI

- **Compilazione**:
  - Si genera codice con indirizzi assoluti che devono risiedere in memoria. Esempio: nei file `.COM` di MS-DOS, il codice è assoluto.
- **Caricamento**:
  - Se non è noto dove un programma risiederà in memoria, viene generato codice rilocabile. Durante il caricamento, questi indirizzi rilocabili vengono convertiti in indirizzi assoluti.
- **Esecuzione**:
  - Se durante l'esecuzione il processo può essere spostato da un segmento di memoria ad un altro, l'associazione degli indirizzi viene ritardata fino alla fase di esecuzione, permettendo una maggiore flessibilità nell'uso della memoria.

Indirizzo Logico (virtuali): indirizzo generato dalla cpu
Indirizzo Fisico: indirizzo reale (MAR) visto dalla memoria

MMU (Memory Management Unit): modulo che si occupa della traduzione.



Rilocazione dinamica tramite un registro di rilocazione.

ALLOCAZIONE DELLA MEMORIA

I moduli di gestione della memoria si differenziano in base al tempo di allocazione della stessa, che può essere:

- CONTIGUA

- NON CONTIGUA

Allocazione Contigua

Nell'allocazione contigua, la memoria viene assegnata in blocchi continui. Ogni oggetto occupa un insieme di posizioni la cui sequenza di indirizzi è strettamente consecutiva. Ecco i vari tipi:

- **Monoallocazione (Single Allocation)**.
- **Partizionamento Statico (Static Partitioning)**.
- **Partizionamento Dinamico (Dynamic Partitioning)**.
- **Segmentazione (Segmentation).

MONOALLOCAZIONE (MONITOR MONOPROCESSO)

Allocazione della memoria in un sistema operativo monoprocesso, dove l'intero sistema è dedicato all'esecuzione di un singolo processo alla volta.

Memoria Divisa in Due Aree Principali

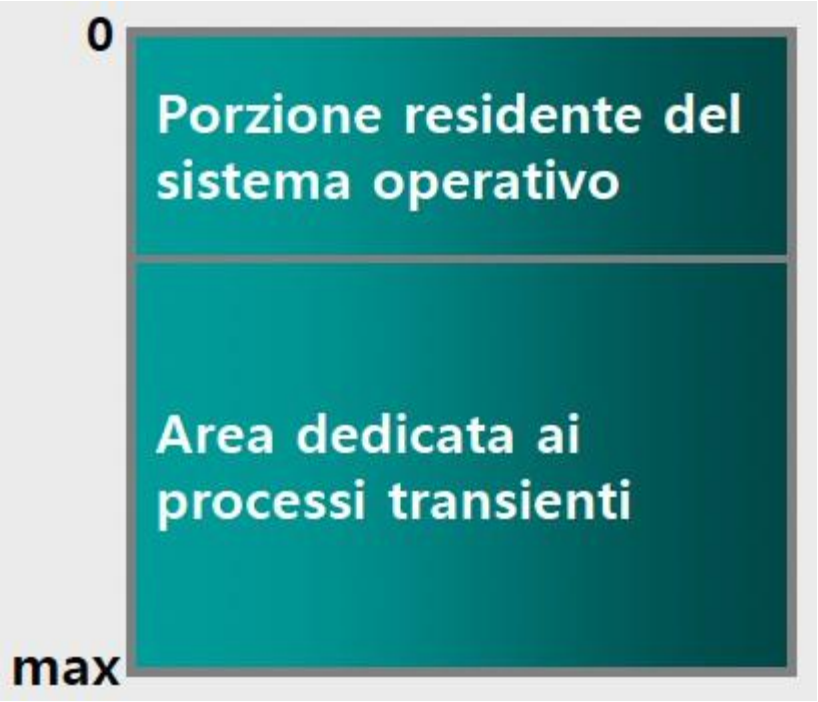
1. **Porzione Residente del Sistema Operativo**:
   - Questa parte della memoria è riservata in modo permanente al sistema operativo (Monitor). Contiene tutti i componenti essenziali che devono essere sempre presenti per garantire il funzionamento del sistema.
2. **Area Dedicata ai Processi Transienti**:
   - Questa parte della memoria è riservata ai processi in esecuzione, ai processi utente o alle porzioni non residenti del sistema operativo. Questa area è dinamica e cambia in base ai processi che vengono eseguiti.

Funzioni del Sistema Operativo in un Ambiente Monoprocesso

- **Tracciamento della Memoria Disponibile**:
  - Il sistema operativo tiene traccia delle prime e ultime posizioni di memoria disponibili per i processi transienti. Questo permette di sapere quanta memoria è libera e può essere allocata ai nuovi processi.
- **Posizionamento del Sistema Operativo**:
  - Di solito, il sistema operativo è posizionato a uno dei due estremi della memoria (all'inizio o alla fine) per ottimizzare l'uso dello spazio disponibile e facilitare la gestione della memoria.
- **Gestione degli Interruttori**:

- Include la gestione dei vettori di interruzione (interrupt), che sono essenziali per rispondere a eventi hardware e software in modo efficiente.

GESTIONE DELLA MEMORIA IN MONOALLOCAZIONE



Alla richiesta di esecuzione di un programma, il sistema operativo deve:

1. **Compatibilità della Dimensione del Processo con la Memoria Disponibile**:
   - Il sistema operativo verifica che la dimensione del processo sia compatibile con la memoria disponibile. Se il processo è troppo grande per la memoria, non può essere eseguito.
2. **Controllo al Processo Utente**:
   - Una volta che il processo è caricato in memoria e avviato, il controllo viene passato al processo utente. Questo significa che il sistema operativo permette al programma dell'utente di essere eseguito fino alla sua conclusione o fino a quando non si verifica un errore.
3. **Rilascio della Memoria**:
   - Alla fine dell'esecuzione del processo (sia per completamento che per errore), la memoria utilizzata viene liberata e resa disponibile per altri processi che potrebbero essere in attesa di esecuzione.

Protezione nel Monitor Monoprocesso

I monitor monoprocesso raramente supportano meccanismi di protezione tra processi utente, poiché solo un processo alla volta può risiedere in memoria. Tuttavia, è importante proteggere il

codice del sistema operativo dal processo transiente in esecuzione. La protezione è garantita da alcuni meccanismi hardware:

- **Registri di Barriera (Barrier Registers)**:
  - **Cosa sono**: Sono registri che tracciano un confine tra le aree di memoria riservate al sistema operativo e quelle riservate ai processi degli utenti.
  - **Come funzionano**: Il registro memorizza il primo indirizzo di memoria disponibile per i processi degli utenti. Il sistema operativo usa questo confine per assicurarsi che i processi utente non accedano all'area riservata al sistema operativo
- **Diritti di Accesso tramite Bit di Protezione**:
  - **Cosa sono**: Ogni parola di memoria ha un bit di protezione associato (1 per le aree del sistema operativo, 0 per il resto).
  - **Come funzionano**: I processi utente possono accedere solo alle parole di memoria con il bit di protezione impostato a 0, mentre il sistema operativo può accedere a tutte le aree di memoria senza restrizioni.
- **Memoria di Sola Lettura per il Sistema Operativo (Read-Only Memory)**:
  - **Cosa è**: Un approccio dove il codice del sistema operativo è collocato in memoria di sola lettura.
  - **Come funziona**: Questo metodo impedisce che il codice del sistema operativo venga modificato dai processi utente. Tuttavia, è poco flessibile perché non permette di aggiornare o correggere facilmente il codice del sistema operativo.
- **Condivisione del Codice e dei Dati**:
  - **Cosa è**: La capacità di condividere codice e dati tra diversi processi.
  - **Come funziona**: In un ambiente monoprocesso, questa condivisione non è molto utile e **raramente supportata**. I programmi utente possono comunque condividere dati attraverso accordi interni, collocando i dati in aree di memoria non sovrascritte durante l'esecuzione dei processi, o tramite file.

PRO E CONTRO LA MONOALLOCAZIONE

Difetti

4. **Assenza di Multiprogrammazione**:
   - Nei sistemi a singola allocazione, solo un programma alla volta può essere caricato ed eseguito in memoria.
   - Questo riduce l'efficienza della CPU e della memoria, poiché queste risorse non vengono sfruttate appieno. Se un programma è in attesa (ad esempio, per un'operazione I/O), la CPU rimane inattiva.
5. **Dimensione della Memoria Sovradimensionata**:

- La memoria disponibile può essere sovradimensionata rispetto alle esigenze della maggior parte dei processi.
- Questo porta a uno spreco di memoria, poiché gran parte della memoria potrebbe rimanere inutilizzata.

6. **Incapacità di Eseguire Processi Grandi**:
   - Se un processo è più grande della memoria disponibile, non può essere eseguito.
   - È necessario suddividere il codice in parti più piccole, chiamate overlay, che vengono caricate in memoria in modo dinamico durante l'esecuzione.
7. **Ottimizzazione Focalizzata sulla Dimensione della Memoria**:
   - I programmi tendono a essere ottimizzati per adattarsi alla memoria disponibile.
   - Questa ottimizzazione può comportare una riduzione delle funzionalità e della velocità del programma per risparmiare memoria.

Vantaggi

8. **Basso Costo di Progettazione del Modulo di Gestione della Memoria**:
   - La progettazione del sistema di gestione della memoria è relativamente semplice e a basso costo.
   - Questo rende il sistema facile da implementare e manutenere.
9. **Supporto Hardware Specifico**:
   - Il sistema può sfruttare specifiche funzionalità hardware per la gestione della memoria.
   - L'uso di hardware specializzato può migliorare l'efficienza del sistema.

PARTIZIONAMENTO STATICO

Il partizionamento statico è una tecnica utilizzata per la gestione della memoria nei sistemi operativi per supportare la multiprogrammazione. Ecco come funziona e quali sono i suoi elementi chiave:

- **Dividere la Memoria**: La memoria fisica del computer viene suddivisa in diverse partizioni. Ogni partizione può essere allocata a un processo diverso, permettendo a più processi di risiedere in memoria contemporaneamente.
- **Dimensioni Fisse**: Le partizioni hanno dimensioni fisse, stabilite durante la fase di configurazione del sistema. Questo significa che una volta create, le dimensioni delle partizioni non cambiano durante l'esecuzione del sistema.

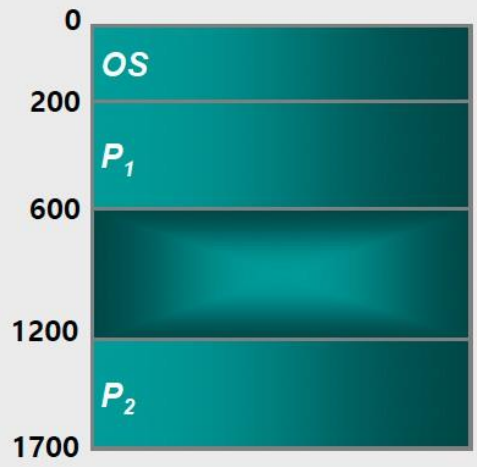
Determinazione del Numero e delle Dimensioni delle Partizioni

- **Capacità della Memoria Fisica**: La quantità totale di memoria disponibile determina quante partizioni possono essere create e quali possono essere le loro dimensioni.

- **Grado di Multiprogrammazione**: Questo è il numero di processi che il sistema intende eseguire simultaneamente. Maggiore è il grado di multiprogrammazione, maggiore sarà il numero di partizioni.
- **Dimensioni dei Processi Frequenti**: Si considerano le dimensioni tipiche dei processi che vengono eseguiti frequentemente, per garantire che la maggior parte dei processi possa adattarsi alle partizioni senza sprecare troppa memoria.

Ogni partizione può contenere un processo diverso. Il sistema operativo deve tener traccia delle partizioni definite.
Lo stato corrente delle partizioni ed i loro attributi vengono raccolti in una struttura dati chiamata **tabella di descrizione delle partizioni (TDP)**.

Ogni riga della TDP rappresenta una partizione e contiene informazioni importanti come la base della partizione, la sua dimensione e il suo stato.



Numero partizione	Base partizione	Dimensione partizione	Stato partizione
0	0 kB	200 kB	Allocata
1	200 kB	400 kB	Allocata
2	600 kB	600 kB	Libera
3	1200 kB	500 kB	Allocata

Esempio di TDP

Diagramma di Memoria

Il diagramma mostra come la memoria è suddivisa:

- **Sistema Operativo**: All'inizio della memoria, la parte riservata al sistema operativo.
- **Partizioni P1 e P2**: Partizioni create per i processi utente, con le rispettive basi e dimensioni.

Processo di Caricamento di un Processo

10. **Consultazione della TDP (Tabella di Descrizione delle Partizioni)**:
    - Il sistema operativo verifica la TDP per trovare una partizione libera adatta.
    - Se trova una partizione libera abbastanza grande, imposta il campo "stato della partizione" su "Allocata".

11. **Caricamento del Processo**:
    - La memoria occupata dal processo viene allocata nella partizione designata.
    - Viene aggiornato il campo nel PCB (Process Control Block), che tiene traccia dello stato del processo.

Metodi per la Ricerca di una Partizione Libera

- **First fit**:
  - Allocare la prima partizione libera sufficientemente grande per contenere il processo.
  - Vantaggio: è molto veloce.
- **Best fit**:
  - Allocare la più piccola delle partizioni libere che possa contenere il processo.
  - Vantaggio: ottimizza lo spazio di memoria, producendo la più piccola frammentazione interna.
  - Svantaggio: richiede la verifica di tutte le partizioni libere prima dell'allocazione.
- **Worst fit**:
  - Allocare la partizione più grande disponibile.
  - Un metodo che può aiutare a ridurre la frammentazione esterna.

Processo di Terminazione di un Processo

12. **Deallocazione della Partizione**:
    - Quando un processo termina, la partizione allocata viene liberata.
13. **Aggiornamento della TDP**:
    - La TDP viene aggiornata per riflettere che la partizione è ora libera e disponibile per altri processi.

Interazione tra Gestore della Memoria e Scheduler

- **Scheduler**:
  - Determina quali processi sono pronti e residenti in memoria in attesa della CPU.
- **Gestore della Memoria**:
  - Può decidere di "sfrattare" un processo in memoria per liberare una partizione e fare spazio per un altro processo.

Cause di Non Disponibilità delle Partizioni di Memoria

1. Nessuna Partizione di Memoria Abbastanza Grande

Quando un nuovo processo arriva, potrebbe non esserci una partizione di memoria abbastanza grande da contenerlo. Ciò può essere causato da:

- **Errore nella Scelta delle Dimensioni delle Partizioni**: Se le partizioni sono state definite con dimensioni che non corrispondono alle esigenze tipiche dei processi, è probabile che alcuni processi non riescano a trovare una partizione adatta.
- **Processo Eccezionalmente Grande**: Alcuni processi possono essere molto più grandi della media e non riescono a trovare una partizione abbastanza grande.

Soluzione:

- **Ridefinire le Dimensioni delle Partizioni**: Modificare la configurazione delle partizioni per adattarle meglio alle dimensioni dei processi.
- **Schema di Overlay**: Progettare il programma in modo da suddividere il codice in sezioni più piccole (overlay) che vengono caricate in memoria solo quando necessario.

2. Tutte le Partizioni Sono Allocate

Quando tutte le partizioni di memoria sono già in uso da altri processi, un nuovo processo non può essere caricato.

Soluzioni:

- **Attendere una Partizione Libera**: Il nuovo processo deve aspettare che uno dei processi attualmente in esecuzione termini e liberi la partizione.
- **Swapping di un Processo**: Spostare temporaneamente un processo dalla memoria principale al disco (swapping), per liberare una partizione per il nuovo processo.

SWAPPING

Processo di Swapping:

14. **Decisione del Scheduler**:
    - Il gestore dei processi (scheduler) decide quando introdurre un nuovo processo in memoria.
15. **Scelta del Processo da Rimuovere**:

- Il gestore dello swapping sceglie quale processo rimuovere dalla memoria basandosi su diversi criteri:
  - **Dimensione della Partizione Richiesta**: Quanto grande deve essere la partizione per il nuovo processo.
  - **Priorità del Processo**: I processi con priorità inferiore possono essere rimossi prima.
  - **Tipo di Evento Atteso dal Processo**: Se il processo sta aspettando un evento a lungo termine (come l'input dell'utente), potrebbe essere scelto per lo swapping.
  - **Tempo Trascorso in Memoria**: Processi che sono stati in memoria per un lungo periodo possono essere scelti per lo swapping.

16. **Componenti del Processo in Memoria**:
    - Un processo che risiede in memoria include:
      - **Codice Eseguibile**: Le istruzioni del programma.
      - **Dati Processati**: I dati già elaborati dal processo.
      - **Stack**: L'area di memoria che contiene le variabili locali e le chiamate di funzione.
      - **Registri di Stato**: Lo stato del processore quando il processo è in esecuzione.
      - **File Aperti**: I file che il processo sta utilizzando.
      - **Descrittore del Processo**: Le informazioni di controllo del processo (PCB).
17. **Salvataggio nel File di Swapping**:
    - Quando un processo viene swappato fuori dalla memoria, i suoi componenti vengono salvati in un file sul disco rigido chiamato "file di swapping".
    - Questo file contiene tutte le informazioni necessarie per ripristinare il processo quando viene swappato di nuovo in memoria.

Il concetto di swapping riguarda la gestione della memoria in cui i processi possono essere temporaneamente spostati dalla memoria principale (RAM) a un disco rigido per liberare spazio in memoria. Questo processo permette di eseguire più processi rispetto alla capacità fisica della memoria disponibile.

Due Metodi per Gestire i Processi Swappati

18. **In un File Unico per Tutto il Sistema**:
    - **Creazione e Posizione**: Questo file viene creato all'inizializzazione del sistema e viene collocato su un dispositivo di memoria secondaria veloce.
    - **Dimensioni Statiche**: L'indirizzo e la dimensione del file sono solitamente statici, il che facilita l'accesso diretto al disco.
    - **Parametro Critico: Dimensione**:
      - Se il file è troppo grande, si spreca spazio su dischi veloci.

    - Se il file è troppo piccolo, le operazioni di swapping possono diventare non disponibili.

19. **In File Separati, Ciascuno Associato a un Singolo Processo**:
    - **Creazione**: Questi file possono essere creati dinamicamente alla creazione del processo o staticamente durante la preparazione del programma.
    - **Vantaggi**:
        - Elimina il problema di dimensionamento di un unico file.
        - Non ci sono restrizioni sul numero di processi attivi.
    - **Svantaggi**:
        - Maggiore spreco di spazio.
        - Tempi di accesso più lunghi ai file distribuiti su diversi punti del disco.

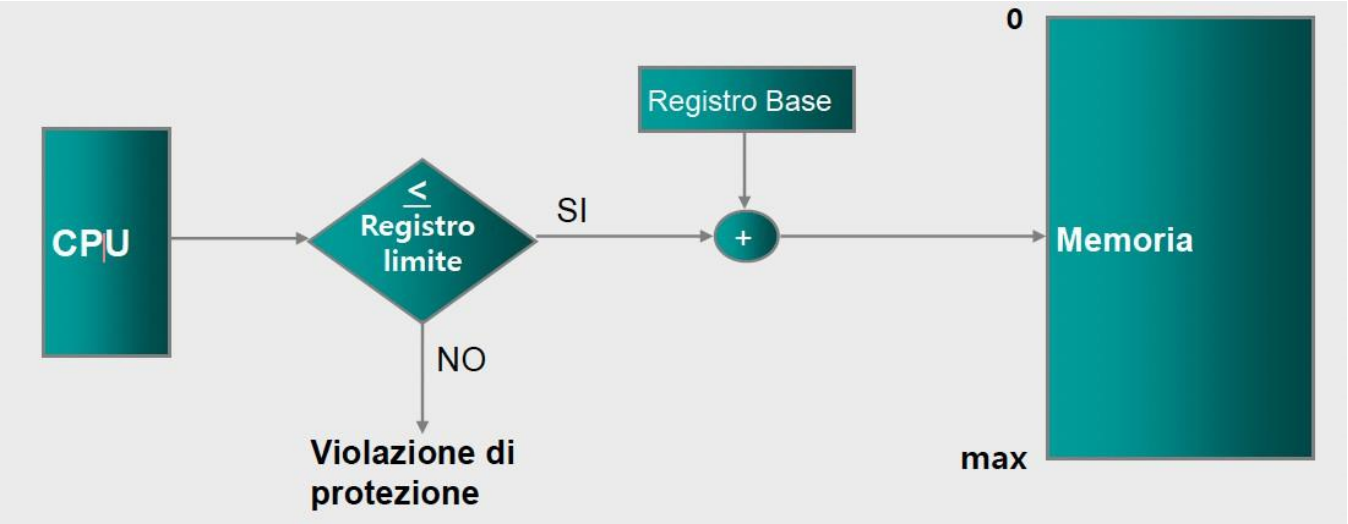
Requisiti per lo Swapping Efficiente

20. **Rilocabilità Dinamica del Processo**:
    - **Cos'è**: Un processo è considerato dinamicamente rilocabile se può essere spostato da un'area di memoria a un'altra senza problemi.
    - **Importanza**: La rilocabilità dinamica permette al processo di iniziare la sua esecuzione in qualsiasi area di memoria disponibile e di essere spostato altrove se necessario.
21. **Calcolo Dinamico degli Indirizzi**:
    - **Descrizione**: Gli indirizzi di memoria usati dal processo devono essere calcolati dinamicamente, in modo che il processo possa funzionare correttamente indipendentemente dalla posizione in cui è stato caricato.
    - **Esempio**: Utilizzare un registro base, che contiene l'indirizzo iniziale della partizione corrente, per calcolare gli indirizzi relativi durante l'esecuzione.
22. **Protezione degli Indirizzi**:
    - **Descrizione**: Per proteggere l'integrità del sistema multiprogrammato, ogni processo deve essere confinato all'interno della propria area di memoria.
    - **Metodi**: Utilizzo di registri limite per impedire al processo di accedere alla memoria al di fuori della propria partizione.

Componenti Chiave per la Rilocabilità Dinamica

- **Registro Base (Base Register)**:
    - Contiene l'indirizzo iniziale della partizione assegnata al processo.
    - Utilizzato per calcolare gli indirizzi fisici reali aggiungendo l'offset dell'indirizzo relativo.
- **Registro Limite (Limit Register)**:
    - Definisce i confini della memoria che il processo può utilizzare.

- Protegge il sistema prevenendo accessi non autorizzati a memoria fuori dalla partizione del processo.



- **CPU**:
- La CPU elabora l'indirizzo virtuale richiesto dal processo.

- **Condizione di Verifica (Registro Limite)**:
  - L'indirizzo virtuale viene confrontato con il valore nel registro limite.
  - **Se** la condizione è soddisfatta (`<=`), l'indirizzo è valido.
  - **Se no**, si verifica una violazione di protezione.
- **Calcolo dell'Indirizzo Fisico (Registro Base)**:
  - Se l'indirizzo è valido, viene aggiunto al valore nel registro base per ottenere l'indirizzo fisico.
- **Memoria**:
  - La memoria è mostrata come un intervallo da `0` a `max`, con la partizione del processo che occupa una sezione specifica.

Condivisione

Nei sistemi con partizioni di memoria fisse, uno degli ostacoli principali è l'isolamento degli spazi di indirizzi. Ogni processo ha il proprio spazio di indirizzamento separato, il che rende difficile la condivisione diretta di dati e codice.

Metodi di Condivisione

23. **Affidare Oggetti Condivisi al Sistema Operativo**:
    - **Descrizione**: Il sistema operativo gestisce direttamente gli oggetti condivisi.
    - **Svantaggi**:
      - L'area del sistema operativo deve poter variare dinamicamente.

    - Le routine dei processi utente devono potersi collegare dinamicamente al sistema operativo.

24. **Copia Identica dell'Oggetto Condiviso per Ogni Processo**:
    - **Descrizione**: Ogni processo ha una copia identica dell'oggetto condiviso e diffonde gli aggiornamenti.
    - **Svantaggi**:
        - In un sistema che supporta lo swapping, uno o più processi potrebbero non essere in memoria e quindi non ricevere gli aggiornamenti.
25. **Collocare i Dati in una Partizione Comune Dedicata**:
    - **Descrizione**: I dati condivisi vengono collocati in una partizione comune dedicata.
    - **Considerazioni**:
        - Il sistema operativo considera una violazione se i processi tentano di accedere a aree di memoria al di fuori delle loro rispettive partizioni.
        - Se il sistema utilizza registri base e limite, sono necessari aggiustamenti per gestire le partizioni che potrebbero non essere contigue.

Vantaggi del Partizionamento Statico

26. **Metodo Semplice di Gestione della Memoria**:
    - Il partizionamento statico è facile da implementare e comprendere. Consiste nel dividere la memoria fisica in partizioni di dimensioni fisse e assegnare queste partizioni ai processi.
27. **Supporta la Multiprogrammazione**:
    - Permette l'esecuzione di più processi contemporaneamente, aumentando l'utilizzo della CPU e migliorando l'efficienza del sistema.
28. **Richiede Supporto Hardware Modesto**:
    - Non necessita di hardware complesso per gestire la memoria. I registri base e limite sono generalmente sufficienti per garantire la protezione e la gestione degli indirizzi di memoria.
29. **Adatto per Ambienti Statici con Carichi di Lavoro Prevedibili**:
    - È ideale per sistemi dove i carichi di lavoro non cambiano frequentemente, come applicazioni aziendali, sistemi di controllo di processi industriali e sistemi bancari.

Svantaggi del Partizionamento Statico

30. **Frammentazione Interna**:
    - Si verifica quando i processi non utilizzano completamente la memoria assegnata, causando spreco di spazio. Ogni partizione ha una dimensione fissa, quindi se un processo è più piccolo della partizione, la memoria rimanente non utilizzata è sprecata.

31. **Necessità di Progettare Programmi con Schemi di Overlay**:
    - Per eseguire programmi più grandi delle partizioni disponibili, potrebbe essere necessario dividere i programmi in segmenti più piccoli che vengono caricati e scaricati in memoria dinamicamente durante l'esecuzione (overlay).
32. **Non Adatto per Contenere Oggetti come Stack che Possono Crescere**:
    - Le strutture dati che possono crescere dinamicamente, come gli stack, non sono ben gestite in partizioni statiche a causa delle dimensioni fisse delle partizioni.
33. **Numero Fisso di Partizioni Limita il Grado di Multiprogrammazione**:
    - Il numero di partizioni è fisso, quindi anche il numero massimo di processi che possono essere eseguiti simultaneamente è limitato. Questo può ridurre la flessibilità e l'efficienza del sistema.

PARTIZIONE DINAMICO

Partizionamento Dinamico, una tecnica avanzata di gestione della memoria che permette di creare e assegnare partizioni di dimensioni variabili in base alle esigenze dei processi.

Descrizione Generale

- **Creazione e Assegnazione delle Partizioni**:
  - A differenza del partizionamento statico, dove le partizioni hanno dimensioni fisse, nel partizionamento dinamico il gestore della memoria può creare partizioni di dimensioni variabili al momento della richiesta del processo.
  - Questo permette una maggiore flessibilità e un utilizzo più efficiente della memoria.

Richiesta di Partizione

- **Ricerca di un'Area Libera**:
  - Quando un processo richiede memoria, il gestore della memoria cerca un'area libera contigua di dimensioni sufficienti per soddisfare la richiesta.
- **Creazione della Partizione**:
  - Una volta trovata un'area adeguata, viene creata una partizione registrando la sua base (indirizzo iniziale), dimensione e stato (Allocata) nella TDP (Tabella di Descrizione delle Partizioni) o in una tabella equivalente.
- **Gestione della Memoria Residua**:
  - L'eventuale memoria libera rimanente viene restituita al pool di memoria libera e resa disponibile per future allocazioni.

Terminazione del Processo o Swapping

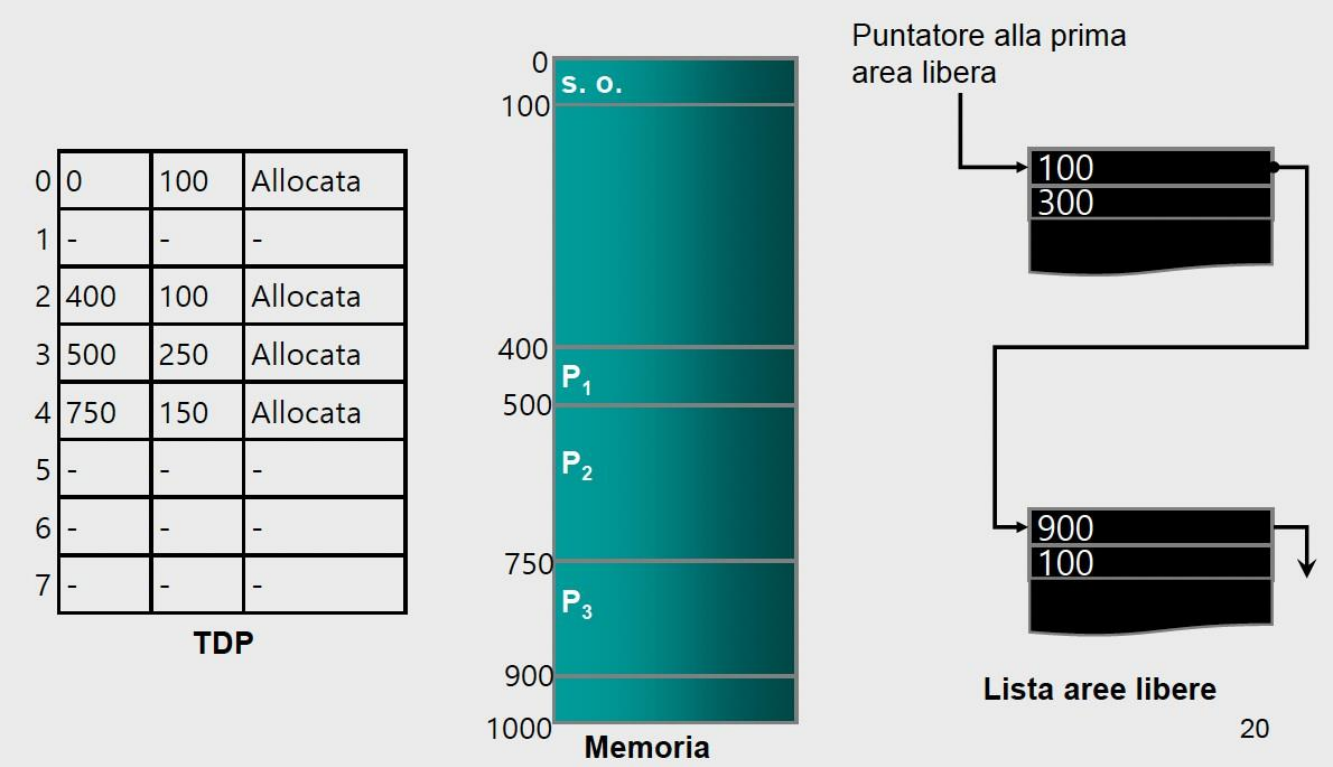
- **Rilascio della Memoria**:

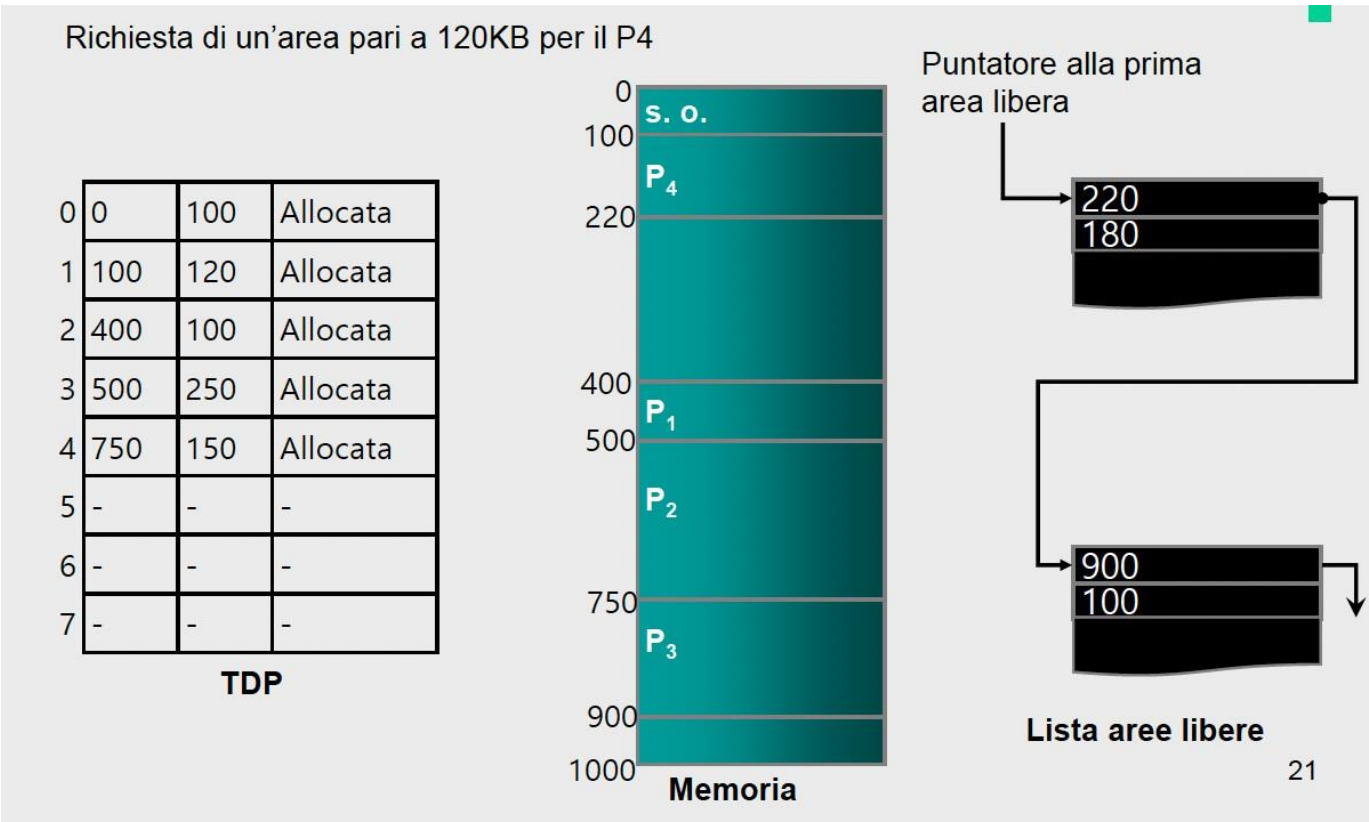
    - Quando un processo termina o viene swappato fuori dalla memoria, lo spazio liberato viene restituito al pool di aree libere.
- **Aggiornamento della TDP**:
    - La riga corrispondente nella TDP viene eliminata per riflettere che la partizione è ora disponibile per nuovi processi.

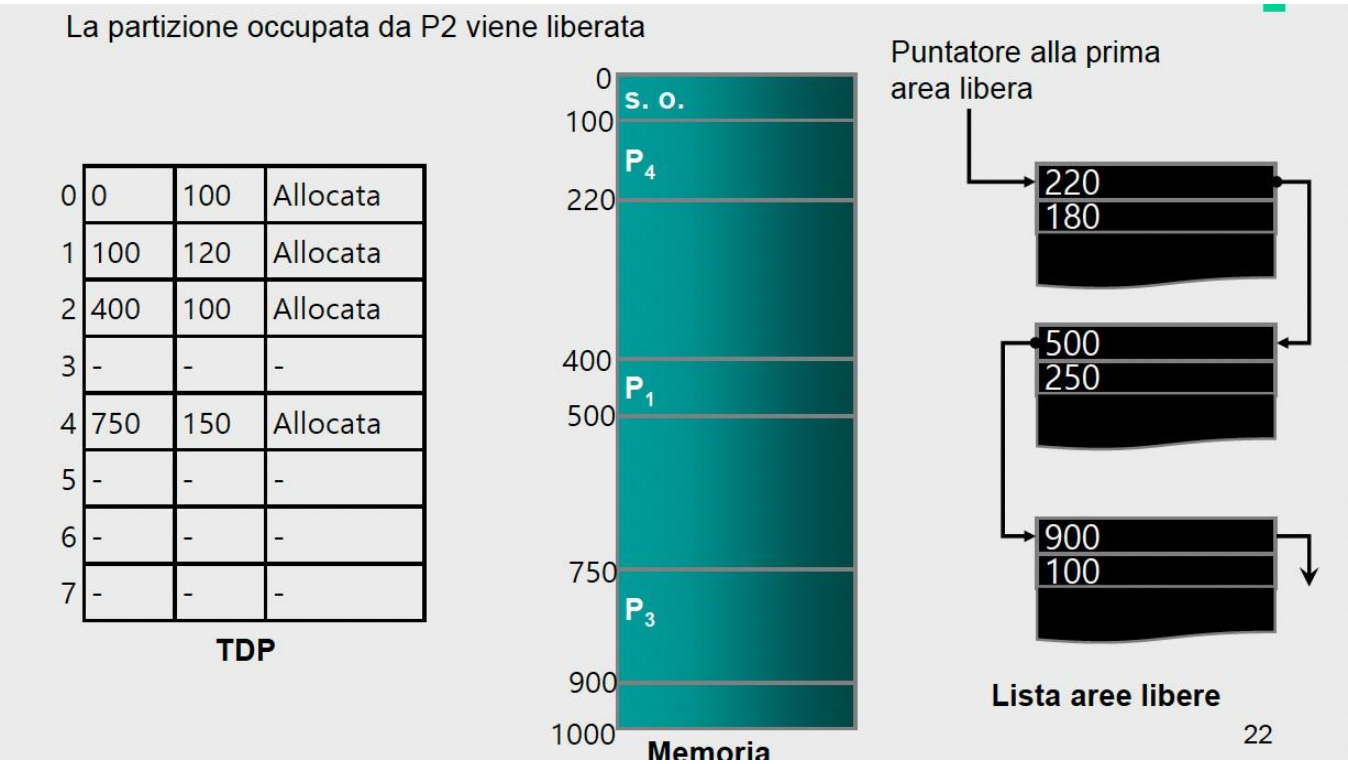
Vantaggi del Partizionamento Dinamico

- **Utilizzo Efficiente della Memoria**:
    - Permette di evitare la frammentazione interna che si verifica nel partizionamento statico, poiché le partizioni sono create su misura per ogni processo.
- **Flessibilità**:
    - Adatta meglio alle esigenze dinamiche dei processi, permettendo di gestire richieste di memoria variabili.

Esempio di partizione dinamica







Partizionamento Dinamico: Algoritmi di Selezione della Memoria

1. First-Fit

- **Descrizione**: L'algoritmo first-fit cerca la prima area di memoria libera sufficientemente grande per contenere il processo.

- **Vantaggio**: La ricerca termina non appena viene trovata la prima area adeguata, rendendo questo metodo molto veloce.
- **Svantaggio**: Può causare una frammentazione della memoria più elevata rispetto ad altri metodi.

2. Next-Fit

- **Descrizione**: Il next-fit è simile al first-fit, ma invece di ripartire sempre dall'inizio della lista delle aree libere, continua la ricerca dal punto in cui si era fermato l'ultimo first-fit.
- **Vantaggio**: Riduce il tempo di ricerca successiva perché evita di iniziare sempre dall'inizio della lista.
- **Svantaggio**: Può anche causare frammentazione della memoria.

3. Best-Fit

- **Descrizione**: L'algoritmo best-fit cerca la più piccola area di memoria libera che possa contenere il processo, riempiendo lo spazio in modo ottimale.
- **Vantaggio**: Produce le porzioni "non utilizzate" più piccole, minimizzando la frammentazione interna.
- **Svantaggio**: Richiede un controllo completo di tutte le aree libere, il che può richiedere più tempo rispetto ad altri algoritmi.

4. Worst-Fit

- **Descrizione**: L'algoritmo worst-fit cerca la più grande area di memoria libera disponibile.
- **Vantaggio**: Produce grandi porzioni "non utilizzate", che possono essere riutilizzate più facilmente per grandi processi futuri.
- **Svantaggio**: Questo metodo può portare a una frammentazione esterna maggiore.

Tutti gli algoritmi di selezione delle aree da allocare nella memoria creano dei "buchi", di memoria inutilizzabile.

Frammentazione Esterna

- **Descrizione**: La frammentazione esterna si verifica quando la memoria libera è suddivisa in piccoli blocchi non contigui. Questo significa che, anche se la somma totale della memoria libera è sufficiente per un nuovo processo, non esiste un singolo blocco contiguo abbastanza grande per contenerlo.
- **Impatto**: La frammentazione esterna porta a situazioni in cui la memoria non può essere utilizzata efficacemente, causando problemi di allocazione.

Compattazione della Memoria

La compattazione è il processo di spostamento dei processi in memoria per combinare i blocchi di memoria libera in un unico blocco contiguo più grande. Questo aiuta a ridurre la frammentazione esterna e a migliorare l'efficienza della memoria.

34. **Compattazione Continua**:
- **Quando Avviene**: La memoria viene compattata ogni volta che un processo libera un'area.
- **Vantaggi**: Riduce al minimo la frammentazione.
- **Svantaggi**: Ha un alto costo di CPU, poiché richiede spostamenti frequenti dei processi in memoria.

35. **Compattazione su Necessità**:
    - **Quando Avviene**: La compattazione viene effettuata solo quando non è possibile allocare memoria a un processo richiedente.
    - **Vantaggi**: Riduce l'uso della CPU rispetto alla compattazione continua, effettuando la compattazione solo quando è strettamente necessario.
    - **Procedura**: Prima di compattare, viene verificato se la somma totale delle aree liberabili è sufficiente per contenere il nuovo processo. Se lo è, viene eseguita la compattazione per creare uno spazio contiguo.

METODI DI COMPATTAZIONE DELLA MEMORIA

1. Spostamento Selettivo Incrementale

- **Ricerca di una Strategia Ottima di Movimento**:
  - Questo metodo cerca di trovare la strategia più efficiente per spostare i processi in memoria, in modo da compattare al meglio lo spazio disponibile.
- **Risparmio di Tempo negli Spostamenti**:
  - Poiché cerca di ottimizzare i movimenti, può risultare in un risparmio di tempo durante lo spostamento dei processi.
- **Svantaggi**:
  - Richiede un sovraccarico notevole, in quanto necessita di calcoli complessi per determinare la strategia ottimale. Per questo motivo, è poco utilizzata.

2. Spostamento Globale

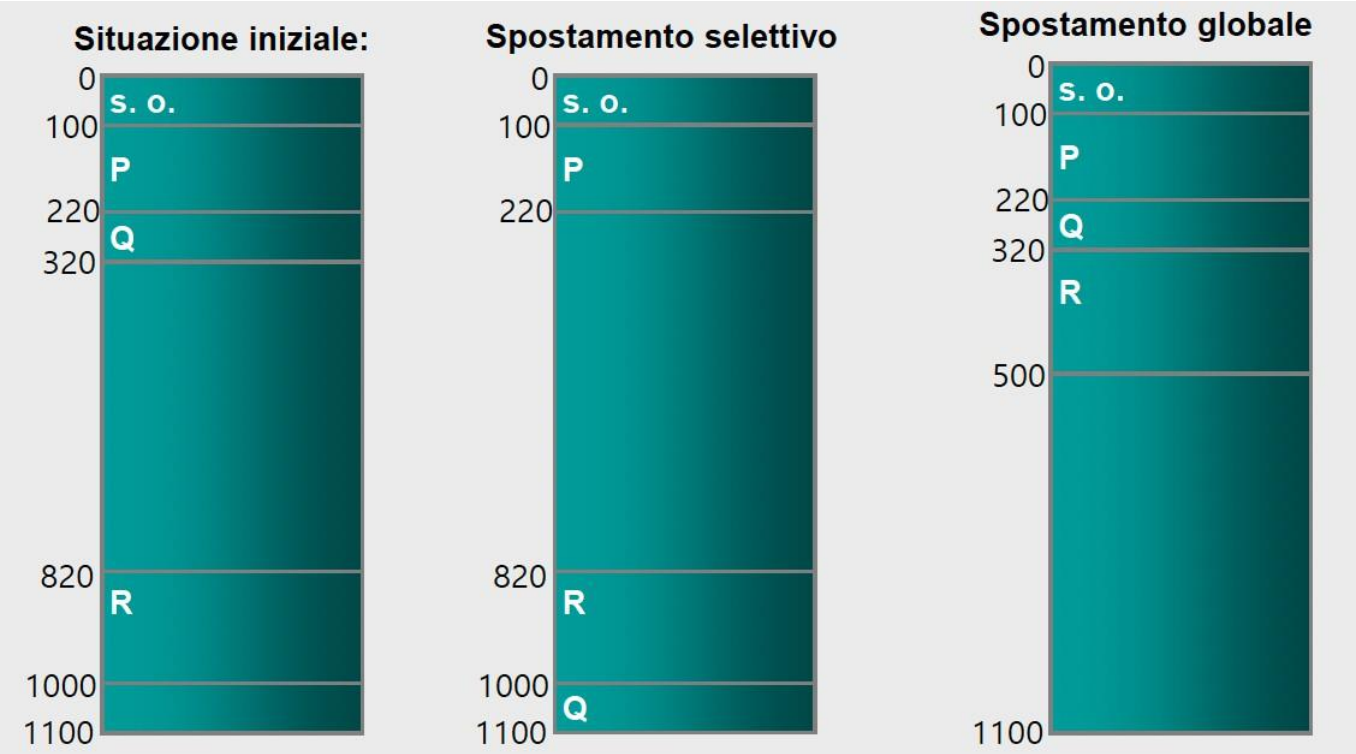
- **Rilocazione di Tutte le Partizioni**:
  - In questo metodo, tutte le partizioni vengono spostate ad uno dei due estremi della memoria, creando un ampio blocco contiguo di memoria libera.
- **Tempo di Trasferimento Non Ottimizzabile**:
  - Il tempo necessario per spostare le partizioni non può essere ridotto, quindi può risultare più lungo rispetto ad altri metodi.

- **Semplicità**:
  - Non richiede strategie particolari o calcoli complessi, rendendolo più semplice da implementare.

Condizione Necessaria per la Compattazione

- **Rilocazione Dinamica (Run Time)**:
  - La compattazione è possibile solo se il sistema supporta la rilocazione dinamica, ovvero la capacità di spostare i processi durante il tempo di esecuzione.
  - Non è possibile se la rilocazione è statica o avviene solo durante la compilazione o il caricamento.

ESEMPI DI COMPATTAZIONE



Partizionamento Dinamico: Conclusioni e Protezione

Protezione e Condivisione

- **Analoghe al Partizionamento Statico**:
  - Le tecniche di protezione e condivisione nel partizionamento dinamico sono simili a quelle utilizzate nel partizionamento statico. In entrambi i casi, i processi sono confinati nelle loro partizioni per evitare accessi non autorizzati alla memoria.
- **Condivisione Ristretta**:

- In alcune situazioni, due partizioni contigue possono condividere un'area comune. Tuttavia, questa forma di condivisione è molto limitata e può avvenire solo tra due processi.
- Questa possibilità permette di ottimizzare l'uso della memoria in casi specifici, ma richiede una gestione attenta per evitare problemi di sicurezza e conflitti di accesso.

Conclusioni sul Partizionamento Dinamico

36. **Supporto Hardware Modesto**:
    - Il partizionamento dinamico richiede un supporto hardware simile a quello del partizionamento statico, rendendolo accessibile senza la necessità di componenti hardware complessi.
37. **Differenze nel Software**:
    - La principale differenza tra il partizionamento dinamico e quello statico risiede nel software di gestione della memoria. Il partizionamento dinamico necessita di algoritmi più sofisticati per la gestione delle partizioni e la compattazione della memoria.
38. **Eliminazione della Frammentazione Interna**:
    - Il partizionamento dinamico elimina la frammentazione interna (spreco di memoria all'interno delle partizioni), ma può causare frammentazione esterna (blocchi di memoria non contigui).
    - La frammentazione esterna può essere risolta tramite la compattazione, un processo che unisce blocchi di memoria liberi per creare aree contigue più grandi.
39. **Adattabilità a Carichi di Lavoro Imprevedibili**:
    - Il partizionamento dinamico è particolarmente adatto a sistemi con carichi di lavoro non prevedibili, come ambienti di sviluppo software. La capacità di creare partizioni di dimensioni variabili permette di gestire efficacemente richieste di memoria variabili.

SEGMENTAZIONE

La segmentazione è una tecnica di gestione della memoria che organizza un programma in segmenti logici.
Ecco una spiegazione dettagliata dei concetti principali:

Concetti Fondamentali della Segmentazione

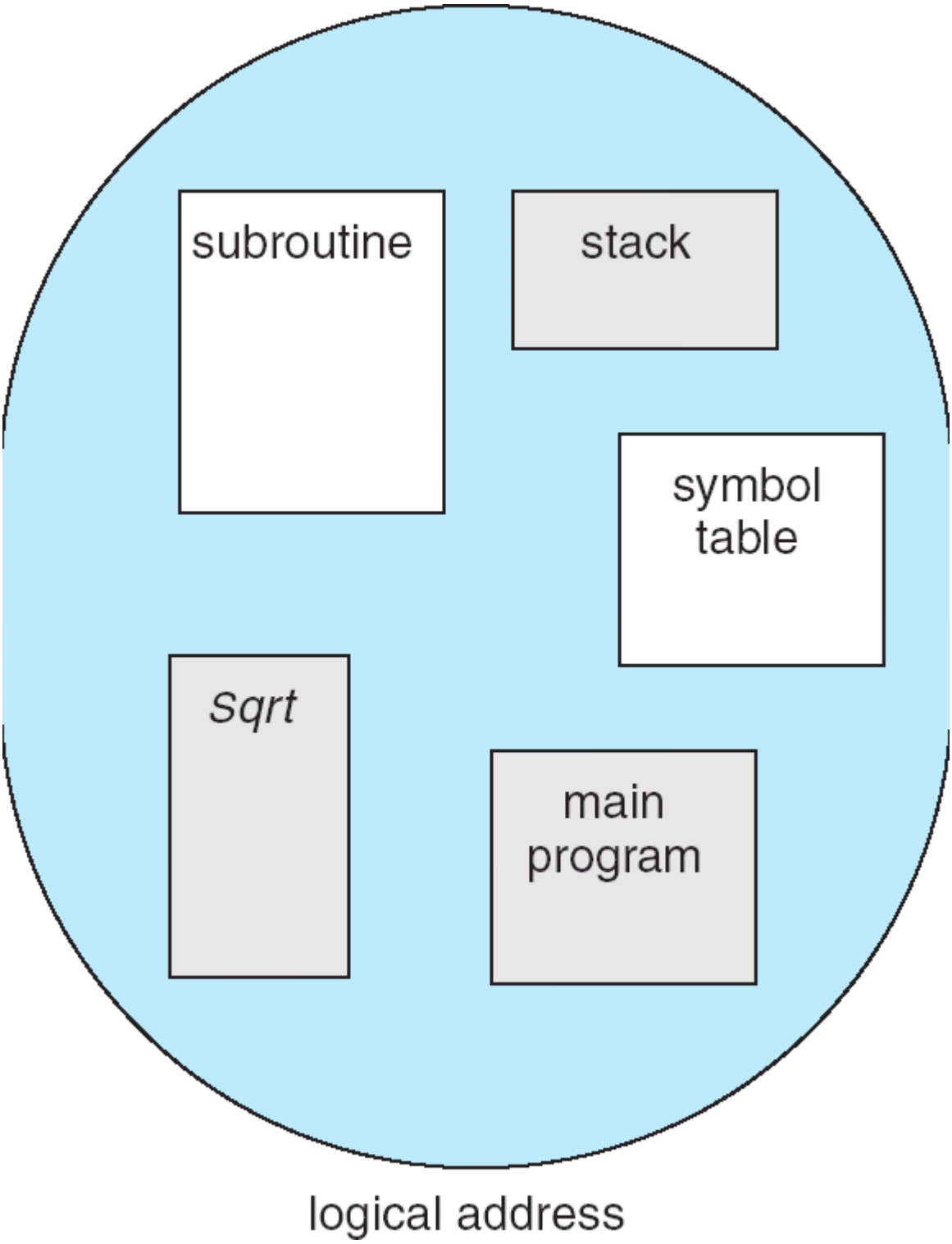
40. **Programma come Collezione di Segmenti**:
    - Un programma è considerato come una collezione di segmenti, ognuno dei quali rappresenta un'unità logica del programma. Ogni segmento può contenere:
        - Programma principale (main program)
        - Procedure

    - Funzioni
    - Metodi
    - Oggetti
    - Variabili locali
    - Variabili globali
    - Blocchi comuni
    - Stack
    - Tabelle dei simboli
    - Array
41. **Segmenti di Diverse Dimensioni**:
    - Ogni segmento può avere dimensioni diverse, adattandosi alle specifiche esigenze dei dati o delle istruzioni che contiene.
42. **Identificazione dei Segmenti**:
    - Ogni segmento è identificato da un nome unico e da un numero di identificazione. Questo permette di accedere ai segmenti in modo indipendente.
43. **Memoria Non Contigua**:
    - I segmenti possono essere caricati in memoria in posizioni non contigue. Questo permette una maggiore flessibilità nella gestione della memoria e riduce la frammentazione.
44. **Indirizzo Logico**:
    - L'indirizzo logico in un sistema segmentato è composto da due parti:
        1. **Numero del Segmento**: Identifica il segmento specifico.
        2. **Offset**: Indica la posizione all'interno del segmento.

Esempio di Segmentazione

Il diagramma nella diapositiva mostra diversi segmenti di un programma, tra cui:

- **Subroutine**
- **Stack**
- **Symbol Table**
- **Sqrt (funzione)**
- **Main Program (programma principale)**



Funzionamento

45. **Generazione dell'Indirizzo Logico**:
    - Quando un programma accede a una variabile o a una funzione, l'indirizzo logico viene generato utilizzando il numero del segmento e l'offset all'interno del segmento.
46. **Conversione in Indirizzo Fisico**:
    - L'indirizzo logico viene poi tradotto in un indirizzo fisico. Questo processo è gestito dall'hardware del computer, utilizzando tabelle di segmenti che mantengono traccia

delle basi e delle dimensioni dei segmenti.

La segmentazione condivide alcune proprietà:

- **Allocazione Contigua e Non Contigua**:
  - **Allocazione Contigua**: relativamente ad ogni singolo segmento, i dati di ogni entità logica, come una funzione o una variabile, è memorizzata in un'area di memoria contigua.
  - **Allocazione Non Contigua**: Diversi blocchi logici possono essere allocati in segmenti non contigui. Questo consente una maggiore flessibilità nella gestione della memoria, evitando la frammentazione.
- **Raccolta degli Oggetti in Segmenti**:
  - **Responsabilità del Programmatore**: È il programmatore che suddivide i dati in segmenti, raggruppandoli logicamente.
  - **Indirizzamento Virtuale**: Ogni segmento inizia dall'indirizzo virtuale zero, ma deve essere tradotto in un indirizzo fisico quando viene caricato in memoria.
  - **Conversione degli Indirizzi**: Per ogni segmento, il nome viene convertito in un indirizzo fisico.
- **Bidimensionalità degli Indirizzi**:
  - **Identificazione dei Dati**: Per identificare un dato all'interno di un segmento, sono necessari due componenti: il nome del segmento e lo spiazzamento (offset) all'interno del segmento stesso.
  - **Indirizzamento Lineare**: La memoria fisica utilizza un indirizzamento lineare, quindi è necessario un meccanismo per convertire gli indirizzi bidimensionali (segmento e spiazzamento) in indirizzi fisici lineari.
- **Vantaggi della Segmentazione**:
  - **Flessibilità**: Permette una gestione più flessibile della memoria, riducendo la frammentazione e migliorando l'utilizzo complessivo della memoria.
  - **Organizzazione**: Facilita l'organizzazione dei dati in base alla loro funzione logica, migliorando la gestione delle risorse e l'efficienza dei processi.

Caricamento di un Processo Segmentato

- **Allocazione delle Partizioni**: Il sistema operativo (S.O.) cerca di allocare diverse partizioni di memoria adatte per i segmenti in cui è suddiviso il processo. Ad esempio, un programma può essere suddiviso in più segmenti logici come subroutine, funzioni matematiche, programma principale, stack e tabella dei simboli.
- **Creazione dei Descrittori di Segmento**: Il S.O. crea un descrittore per ogni segmento, registrando l'indirizzo fisico in cui è stato collocato il segmento e la sua dimensione. Questo

descrittore è essenziale per tenere traccia di dove ogni segmento si trova nella memoria fisica.

- **Tabella dei Descrittori di Segmento**: L'insieme dei descrittori di segmento di un processo viene raccolto in una "segment table". Questa tabella permette al S.O. di tradurre gli indirizzi logici dei segmenti in indirizzi fisici.

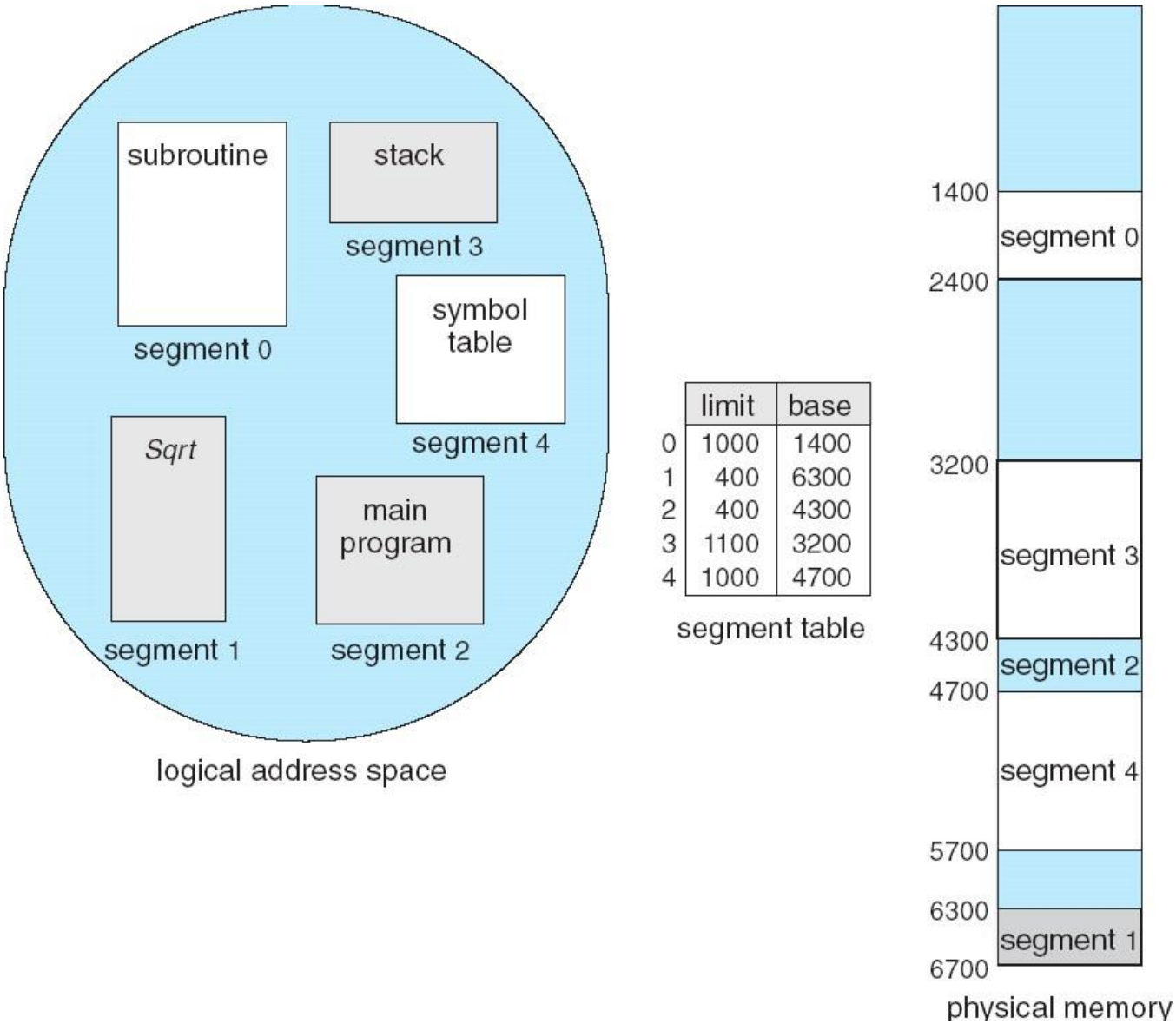


Diagramma del Processo:
Diagramma che mostra lo spazio degli indirizzi logici e la memoria fisica, con una tabella dei segmenti che mappa i segmenti ai loro indirizzi fisici. Ecco un riepilogo dei segmenti e delle loro proprietà:

- **Indirizzi Logici**:
  - Segmento 0: Subroutine
  - Segmento 1: Sqrt
  - Segmento 2: Programma Principale

    - Segmento 3: Stack
    - Segmento 4: Tabella dei Simboli
- **Tabella dei Segmenti**:
    - Segmento 0: Limite 1000, Base 1400
    - Segmento 1: Limite 400, Base 6300
    - Segmento 2: Limite 400, Base 4300
    - Segmento 3: Limite 1100, Base 3200
    - Segmento 4: Limite 1000, Base 4700
- **Memoria Fisica**:
    - Segmento 0: 1400
    - Segmento 3: 3200
    - Segmento 2: 4300
    - Segmento 4: 4700
    - Segmento 1: 6300

Definizioni:

- **Base**: L'indirizzo fisico di inizio del segmento nella memoria.
- **Limite**: Specifica la lunghezza del segmento.

Processo di Traduzione degli Indirizzi

Indirizzi Logici: Quando il processore vuole accedere a un dato, fornisce un numero di segmento (`s`) e uno spiazzamento (`d`).

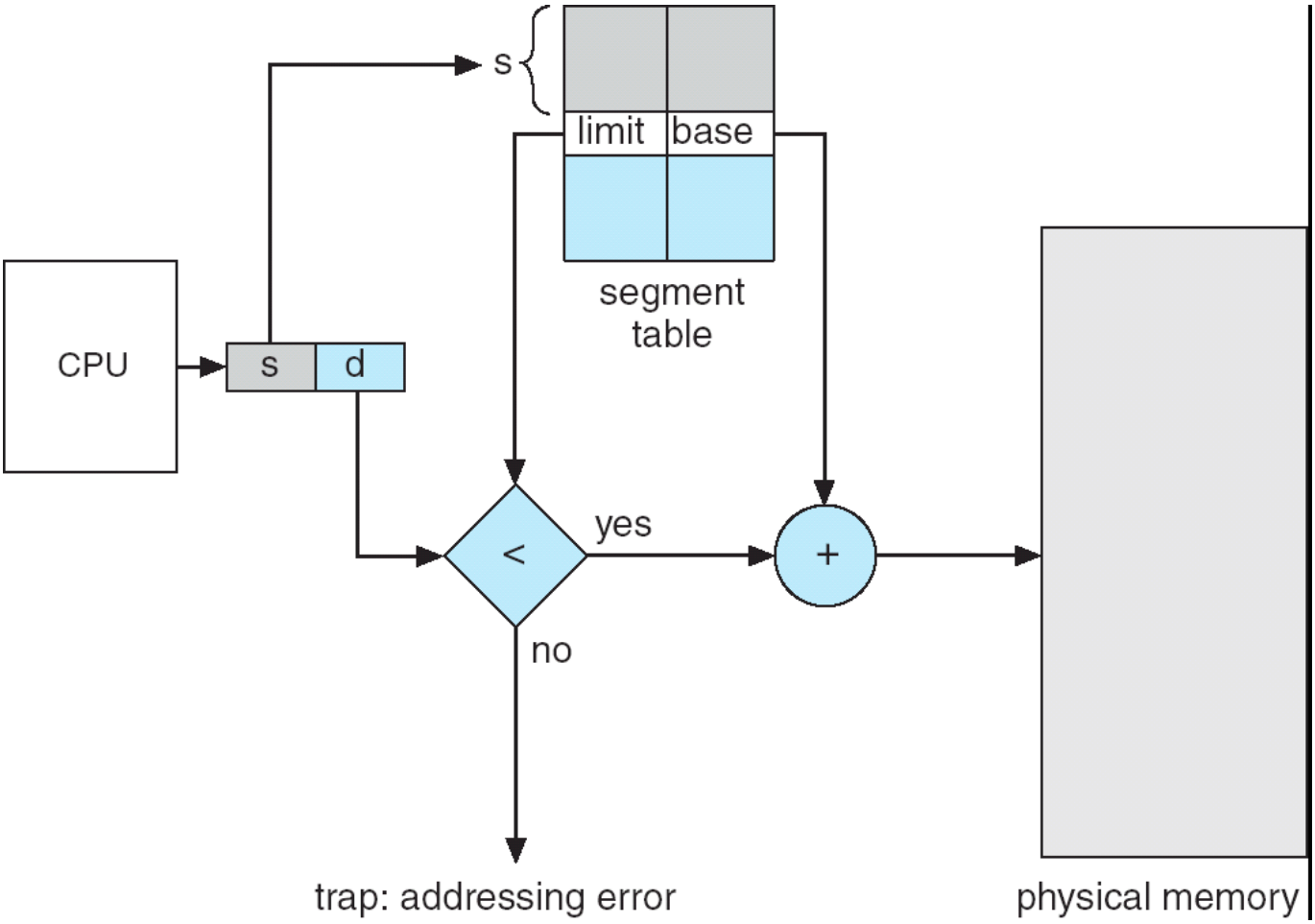


Tabella dei Segmenti:
- Il numero di segmento (`s`) viene utilizzato per accedere alla tabella dei segmenti, dove si trova l'indirizzo base e il limite del segmento.
- **Registro base della tabella dei segmenti (STBR)**: Punta alla posizione della tabella dei segmenti nella memoria.
- **Registro lunghezza della tabella dei segmenti (STLR)**: Indica il numero di segmenti utilizzati da un programma, permettendo di controllare l'accesso ai segmenti non assegnati.

Verifica dello Spiazzamento:
- Lo spiazzamento (`d`) viene confrontato con il limite per assicurarsi che sia entro i limiti del segmento.
- Se `d` è entro i limiti, viene aggiunto all'indirizzo base per formare l'indirizzo fisico.
- Se `d` è fuori dai limiti, si genera un errore di indirizzamento.

Indirizzo Fisico: Se lo spiazzamento è valido, l'indirizzo fisico viene calcolato aggiungendo lo spiazzamento all'indirizzo base del segmento. Questo indirizzo fisico viene poi utilizzato per accedere alla memoria.

Nota Importante:

Quando un processo subisce uno swapping (viene spostato fuori dalla memoria e poi ricaricato), è necessario aggiornare la sua tabella dei segmenti per riflettere i nuovi indirizzi

fisici.

Quando un programma richiede l'accesso a una determinata locazione di memoria, vengono utilizzati **indirizzi bidimensionali** (numero di segmento `s` e spiazzamento `d`). Per ccedere fisicamente alla memoria, si seguono questi passaggi:

1. **Accesso alla Segment Table**:
   - Il numero di segmento `s` è utilizzato per accedere alla tabella dei segmenti.
   - La tabella dei segmenti contiene gli indirizzi di base e i limiti dei segmenti, che sono utilizzati per tradurre gli indirizzi virtuali in indirizzi fisici.
2. **Verifica dello Spiazzamento**:
   - Lo spiazzamento `d` è controllato per verificare che rientri nei limiti del segmento.
   - Se `d` è valido, viene aggiunto all'indirizzo base del segmento per ottenere l'indirizzo fisico.
3. **Calcolo dell'Indirizzo Fisico**:
   - Se lo spiazzamento è valido, l'indirizzo fisico è ottenuto sommando lo spiazzamento `d` all'indirizzo base del segmento.

Questa procedura richiede **due accessi alla memoria**: uno per accedere alla segment table e un altro per accedere alla locazione richiesta, **raddoppiando il tempo di accesso alla memoria**.

Soluzione

Per ridurre il tempo di accesso, se i descrittori di segmento sono pochi o utilizzati frequentemente, è possibile caricarli in **registri hardware specializzati**. Ad esempio, la famiglia di processori Intel utilizza registri come CS (Code Segment), DS (Data Segment), SS (Stack Segment) ed ES (Extra Segment) per gestire i diversi segmenti.

Diritti e Modalità di Accesso

Ogni segmento può avere diritti di accesso specifici:

- **Segmento Codice**: Accesso in modalità "execute" o "read-only".
- **Segmento Stack**: Accesso in modalità "read/write".
- **Segmento Dati**: Accesso in modalità "read-only" o "read/write".

Questi diritti di accesso sono registrati in un campo della Tabella dei Descrittori di Segmento (TDS).

Condivisione

- **Segmenti Condivisi**: Gli oggetti che devono essere condivisi tra più processi sono collocati in segmenti dedicati e separati.
- **Mappatura Segmenti**: Il segmento condiviso può essere mappato, attraverso la tabella dei segmenti, nello spazio di indirizzamento virtuale di tutti i processi autorizzati ad accedervi. Questo significa che ogni processo può accedere allo stesso segmento di memoria, rendendo la condivisione dei dati più efficiente.
- **Spiazzamento Interno**: La posizione relativa (spiazzamento) di un dato all'interno del segmento è la stessa per tutti i processi che condividono il segmento. Questo consente un accesso uniforme ai dati condivisi.

Collegamento Dinamico

- **Caricamento su Richiesta**: Una procedura, come una libreria, viene caricata in memoria solo quando un processo ne fa richiesta, durante l'esecuzione. Ciò significa che la memoria viene occupata solo quando necessario.
- **Ottimizzazione della Memoria**: La segmentazione permette di aggiungere nuovi segmenti al processo richiedente, aggiornando la tabella dei segmenti e i registri base (`STBR` - Segment-table base register) e lunghezza (`STLR` - Segment-table length register) della tabella dei segmenti.
- **Efficienza**: Questo metodo ottimizza l'uso della memoria, riducendo il carico complessivo e migliorando le prestazioni dei programmi.

Segmentazione: Conclusioni

- Eliminando la necessità di allocare processi in aree contigue si rende più efficiente la gestione della memoria fisica;
- no frammentazione interna;
- Supporto efficiente di protezione e condivisione;
- Supporto del collegamento dinamico.

SVANTAGGI:

- Frammentazione esterna (buchi troppo piccoli per ospitare un intero segmento), necessità di effettuare comunque la compattazione (resa anche più complessa);
- il duplice accesso alla memoria deve essere supportato da hardware apposito;
- la gestione della memoria da parte del sistema operativo è più complessa rispetto a quella per il partizionamento statico e dinamico;
- non rende ancora possibile l'esecuzione di processi più grandi delle dimensioni fisiche della memoria.

ALLOCAZIONE NON CONTIGUA

L'allocazione non contigua della memoria è una tecnica in cui la memoria è allocata in modo che un singolo oggetto logico (ad esempio, un programma o un processo) sia suddiviso in più aree di memoria fisica che non sono necessariamente adiacenti tra loro.

Funzionamento:

- La memoria fisica non è contigua, quindi diverse parti di un programma possono essere memorizzate in luoghi fisicamente separati.
- Durante l'esecuzione di un processo, viene effettuata la **traduzione degli indirizzi** per ristabilire la corrispondenza tra lo spazio di indirizzamento virtuale (che appare contiguo) e gli indirizzi fisici effettivi.

PAGINAZIONE

La paginazione è una tecnica di gestione della memoria in cui la memoria fisica è suddivisa in blocchi di dimensione fissa chiamati *pagine fisiche* o *frame*, mentre la memoria logica (o virtuale) è suddivisa in *pagine logiche* o *virtuali* della stessa dimensione delle pagine fisiche.

Caricamento in Memoria di un Processo

Quando un processo deve essere caricato in memoria, avviene quanto segue:

- Le pagine logiche del processo vengono prelevate dalla memoria ausiliaria (come il disco).
- Queste pagine logiche vengono caricate nelle pagine fisiche (frame) della memoria centrale.

Traduzione degli Indirizzi

Ogni indirizzo generato dalla CPU durante l'esecuzione di un programma è diviso in due parti:
4. **Numero di Pagina (P)**: Costituito dai bit più significativi, rappresenta il numero di pagina virtuale.
5. **Spiazzamento (D)**: Costituito dai bit meno significativi, rappresenta la posizione relativa all'interno della pagina.

Tabella delle Pagine (TDP)

Il sistema operativo tiene traccia della corrispondenza tra le pagine virtuali e le pagine fisiche usando una struttura chiamata *tabella delle pagine* (TDP). La TDP viene costruita quando il processo viene caricato e contiene le seguenti informazioni:

- **Indirizzo di Partenza**: Ogni riga della TDP contiene l'indirizzo fisico iniziale della pagina fisica in cui è allocata la corrispondente pagina virtuale.
- **Numero di Righe**: La tabella è costituita da un numero di righe pari al numero di pagine occupate.

Paginazione esempio

ES:

Memoria:	1 Mb
Dimensione Pagina:	256 byte (se la memoria è indirizzabile al byte occorrono 8bit per il displacement)
Numero di pagine:	4096 (12 bit per indirizzare la pagina)
Lunghezza dell'indirizzo:	20 bit (12+8)

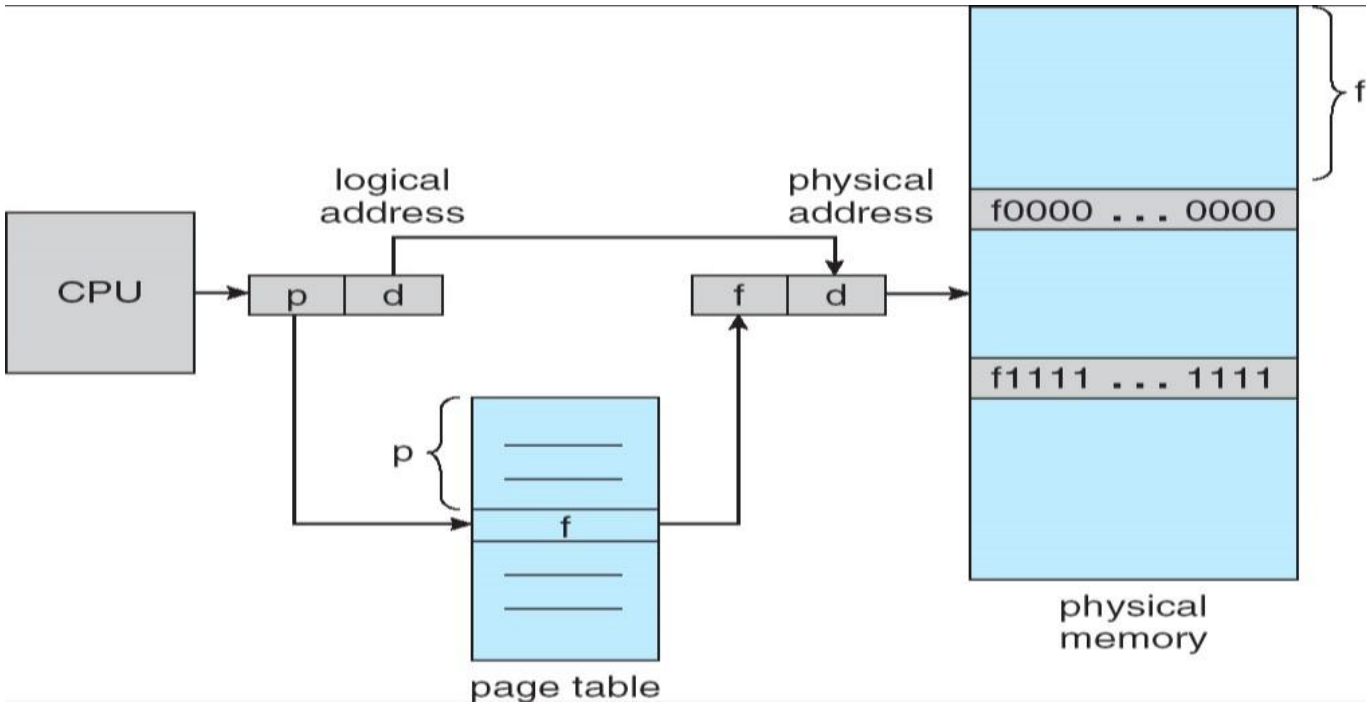
Lo spazio di indirizzamento di un ipotetico processo utente che occupi 1008 byte viene diviso in quattro pagine virtuali numerate da 0 a 3.

Funzionamento:
6. **Numero di Pagina:** I 12 bit più significativi dell'indirizzo identificano la pagina.
7. **Displacement:** Gli 8 bit meno significativi dell'indirizzo identificano la posizione relativa all'interno della pagina.

Vantaggi della Paginazione

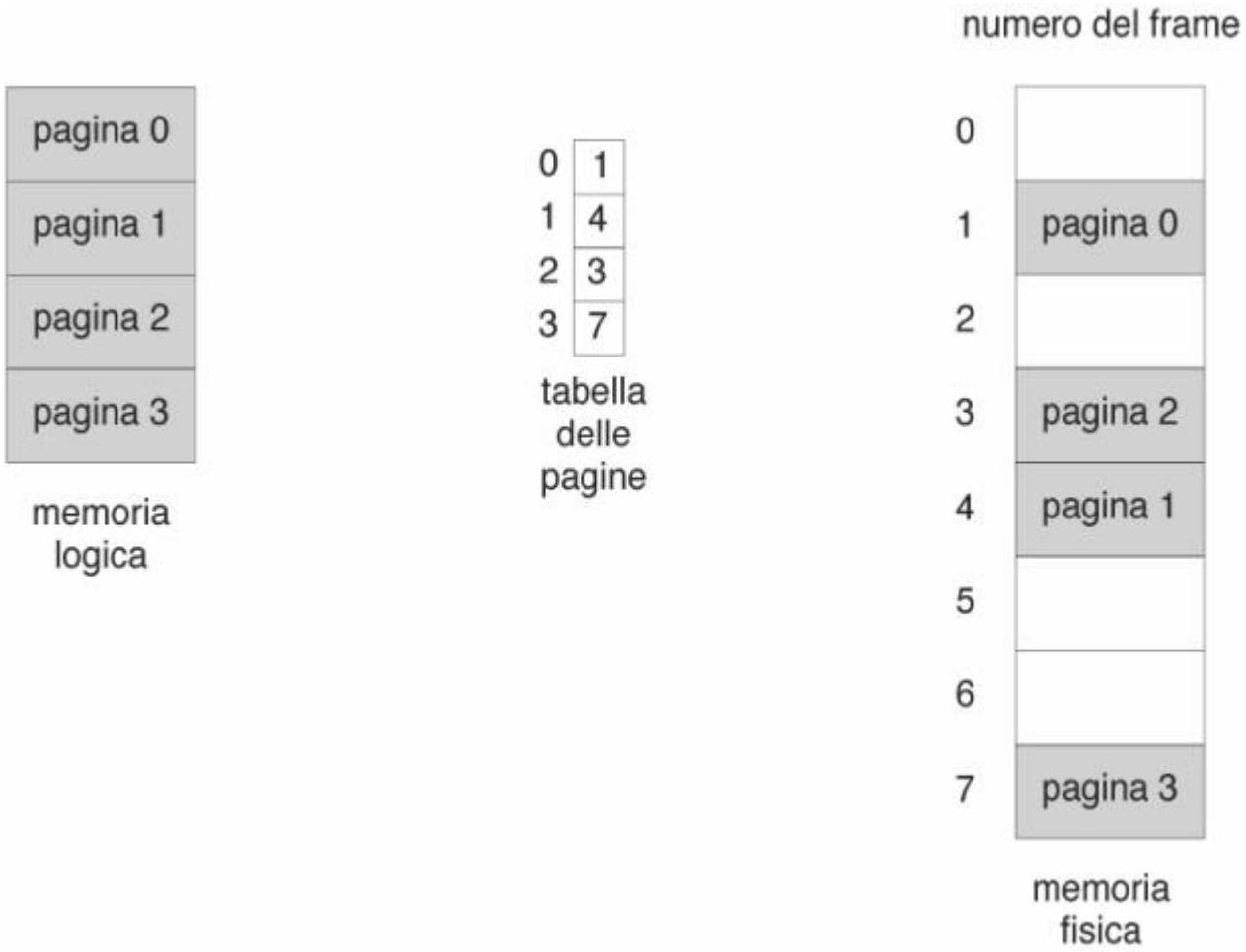
- **Eliminazione della Frammentazione Esterna:** Poiché la memoria è suddivisa in blocchi di dimensioni fisse, la frammentazione esterna è minimizzata.
- **Utilizzo Efficiente della Memoria:** Le pagine non devono essere contigue, permettendo un uso più flessibile della memoria disponibile.

MECCANISMO DI TRADUZIONE



Vista Logica:

- **Memoria vista dall'utente**: Mostra come il processo vede la memoria logica, suddivisa in pagine numerate (pagina 0, pagina 1, pagina 2, pagina 3).



Vista Fisica:

- **Memoria Fisica**: Mostra i frame della memoria fisica numerati da 0 a 7.

- **Tabella delle Pagine**: Mappa ogni pagina logica a un frame fisico. Indica dove ogni pagina logica è memorizzata nella memoria fisica.

Processo di Traduzione

8. **Generazione dell'Indirizzo Logico**: La CPU genera un indirizzo logico che è diviso in numero di pagina (p) e displacement (d).
9. **Accesso alla Tabella delle Pagine**: Utilizzando il numero di pagina, la CPU accede alla tabella delle pagine per trovare il numero di frame corrispondente.
10. **Calcolo dell'Indirizzo Fisico**: Combina il numero di frame ottenuto dalla tabella delle pagine con il displacement per formare l'indirizzo fisico.
11. **Accesso alla Memoria Fisica**: L'indirizzo fisico viene utilizzato per accedere alla locazione di memoria desiderata.

Tabella della Memoria (TDM):

- **Stato delle Pagine**: Il sistema operativo tiene traccia dello stato di tutte le pagine fisiche usando una tabella della memoria (TDM). Ogni riga della TDM indica se una pagina è "Libera" o "Allocata".
- **Numero di Righe**: Il numero di righe della TDM è uguale al numero di pagine fisiche disponibili nella memoria.
- **Tabella delle Pagine Libere**: Esiste anche una tabella che elenca tutte le pagine fisiche disponibili (libere).

Allocazione dei Frame a un Nuovo Processo

12. **Dimensione del Processo**: Quando si deve eseguire un processo, il sistema operativo esamina la dimensione del processo in pagine.
13. **Assegnazione dei Frame**: Se ci sono sufficienti frame disponibili, ad ogni pagina logica del processo viene assegnato un frame fisico.
    - La prima pagina viene caricata in un frame e il numero di quel frame è registrato nella tabella delle pagine del processo.
    - Questo processo continua per tutte le pagine del processo fino a che tutte le pagine non hanno un frame assegnato.

Osservazioni Importanti

14. **Isolamento dei Processi**: Un processo non può accedere alle pagine di un altro processo perché queste non sono incluse nella sua tabella delle pagine. Questo garantisce l'isolamento e la sicurezza dei dati tra processi diversi.

15. **Gestione del Context Switching**: Ad ogni processo è associata una tabella delle pagine. Durante il context switching, oltre a cambiare il contesto del processo, il sistema operativo deve gestire anche le tabelle delle pagine, aggiungendo ulteriori dati da gestire.

Frammentazione Interna

Frammentazione interna si verifica quando le pagine assegnate a un programma non sono completamente utilizzate. Questo succede perché la dimensione delle pagine è fissa, ma la quantità di dati effettivamente necessari può essere inferiore. Di conseguenza, l'ultima pagina assegnata a un programma può avere spazio inutilizzato, causando uno spreco di memoria.

Pagine Molto Piccole

Conseguenze di avere pagine molto piccole:

16. **Minore Frammentazione**: Utilizzando pagine più piccole, si riduce lo spazio inutilizzato all'interno delle pagine, diminuendo la frammentazione interna.
17. **Carico Eccessivo sul Meccanismo di Traduzione degli Indirizzi**: Con pagine più piccole, il numero di pagine aumenta, il che può sovraccaricare il sistema durante la traduzione degli indirizzi da logici a fisici.
18. **Numerosi Accessi alla Memoria di Massa**: Più piccole sono le pagine, più spesso il sistema deve accedere alla memoria di massa per caricare le pagine necessarie, il che può rallentare le prestazioni complessive.

Esempio di Paginazione e Frammentazione Interna

Immaginiamo di avere un sistema con le seguenti caratteristiche:

- **Dimensione della memoria fisica**: 1 MB (megabyte)
- **Dimensione delle pagine**: 4 KB (kilobyte)
  - Ogni pagina fisica ha una dimensione di 4 KB.
  - Il sistema può quindi avere 1 MB4 KB=256 pagine fisiche.

Processo da Caricare

Supponiamo di avere un processo che richiede 10 KB di memoria. Questo processo sarà diviso in pagine logiche e mappato sui frame fisici della memoria:
19. **Divisione del Processo**:
- Il processo da 10 KB sarà suddiviso in 10 KB4 KB≈3 pagine logiche.
- Poiché ogni pagina ha una dimensione di 4 KB, avremo:
- Pagina 1: 4 KB

- Pagina 2: 4 KB
- Pagina 3: 2 KB (la parte rimanente del processo)

Allocazione delle Pagine

20. **Mappatura nelle Pagine Fisiche**:
    - Il sistema operativo cerca tre frame liberi nella memoria fisica per mappare le pagine logiche del processo.
    - Supponiamo che i frame fisici 5, 18 e 72 siano liberi.

Frammentazione Interna

21. **Assegnazione delle Pagine**:
    - Pagina 1: Mappata al frame fisico 5
    - Pagina 2: Mappata al frame fisico 18
    - Pagina 3: Mappata al frame fisico 72
22. **Spazio Inutilizzato** (Frammentazione Interna):
    - La Pagina 3 utilizza solo 2 KB su 4 KB, lasciando 2 KB inutilizzati nel frame fisico 72. Questa è la **frammentazione interna**.
    - Anche se il processo ha bisogno di 10 KB, il sistema ha riservato 12 KB di memoria fisica (tre pagine da 4 KB ciascuna), di cui 2 KB rimangono inutilizzati.

Tabella delle Pagine

La **Tabella delle Pagine** del processo può apparire così:

Pagina Logica	Frame Fisico
1	5
2	18
3	72

Paginazione: Memorizzazione di Aree Libere

Scopo: La paginazione divide la memoria fisica in blocchi di dimensioni fisse chiamati *pagine fisiche* e la memoria logica in *pagine logiche*. Per gestire efficientemente la memoria, è essenziale tenere traccia delle pagine fisiche libere.

Metodo Statistico

La consultazione della *Tabella di Memoria* statica per trovare n pagine fisiche libere richiede una ricerca su un numero di righe che in media è $x = n / q$.

q è la probabilità che una pagina sia libera e si lega alla percentuale di memoria libera u:
$q = u / 100 \qquad 0 \leq q \leq 1$

x aumenta all'aumentare dell'occupazione di memoria

- **Tabella di Memoria Statica**: Il metodo utilizza una tabella per tracciare quali pagine fisiche sono libere o occupate.
- **Probabilità di Una Pagina Libera (q)**:
  - È la probabilità che una pagina sia libera e si calcola come la percentuale di memoria libera (u) divisa per 100.
- **Numero di Pagine Libere Attese (x)**:
  - Il numero atteso di pagine libere è determinato dalla formula:

$$x = \frac{n}{q}$$

  - Dove `n` è il numero totale di pagine e `q` la probabilità che una pagina sia libera.
- **Osservazione**:
  - `x` aumenta con l'aumento dell'occupazione della memoria: più memoria è occupata, maggiore è il numero di pagine da controllare per trovare pagine libere.

Liste di Puntatori

Alternativa alla Tabella Statica:

- Utilizzare una **lista di puntatori** per mantenere l'indirizzo delle pagine fisiche libere.
  - **Allocazione**: Le prime `n` pagine libere vengono identificate e assegnate.
  - **Deallocazione**: Le pagine liberate vengono aggiunte in testa alla lista delle pagine libere.

Vantaggi:

- Il tempo per allocare e deallocare non dipende dalla percentuale di memoria occupata, il che rende il sistema più scalabile.

Svantaggi:

- L'uso di una lista di puntatori può comportare un leggero spreco di memoria e tempo rispetto alla gestione di una tabella statica.

ARCHITETTURA DI PAGINAZIONE

23. **Tabella delle Pagine (Page Table)**: Ogni processo ha una propria tabella delle pagine e il Process Control Block (PCB) contiene un puntatore a questa tabella.
24. **Meccanismi Hardware**: Questi meccanismi sono utilizzati principalmente per due scopi:
    - Risparmiare memoria necessaria per le Tabelle delle Pagine (TDP)
    - Velocizzare la traduzione da indirizzi virtuali a indirizzi fisici.
25. **Registri del Processore**: Per una traduzione veloce, è necessario che l'intera tabella sia posizionata nei registri del processore, anche se questo può limitare la dimensione della tabella, rendendolo inadeguato per i moderni sistemi operativi.
26. **Costruzione delle Tabelle**: È ragionevole costruire tabelle con un numero di righe pari alle pagine effettivamente utilizzate.
27. **Registro Base della Tabella delle Pagine (PTBR)**: Questo registro contiene l'indirizzo di partenza della tabella delle pagine. Cambiare tabella implica cambiare questo registro, ma ciò comporta un accesso doppio alla memoria, rallentando il processo.

Velocizzazione del Tempo di Traduzione

Quando un sistema operativo traduce un indirizzo virtuale in un indirizzo fisico, normalmente si richiedono due accessi in memoria:
28. Un accesso alla tabella delle pagine (TDP) per ottenere il numero di pagina fisica corrispondente.
29. Un accesso alla memoria fisica per accedere alla locazione richiesta.

Problema e Soluzione

Problema: Questo doppio accesso alla memoria può rallentare le prestazioni del sistema.
Soluzione: La soluzione proposta è l'uso di una **Translation Look-aside Buffer (TLB)**, una sorta di cache di traduzione. Il TLB conserva gli indirizzi delle pagine utilizzate più frequentemente, che è solo una porzione della tabella delle pagine.

Funzionamento del TLB

Ogni elemento del TLB ha due parti:

- **Chiave:** Derivata dall'indirizzo logico della pagina.
- **Valore:** L'indirizzo della pagina fisica corrispondente.

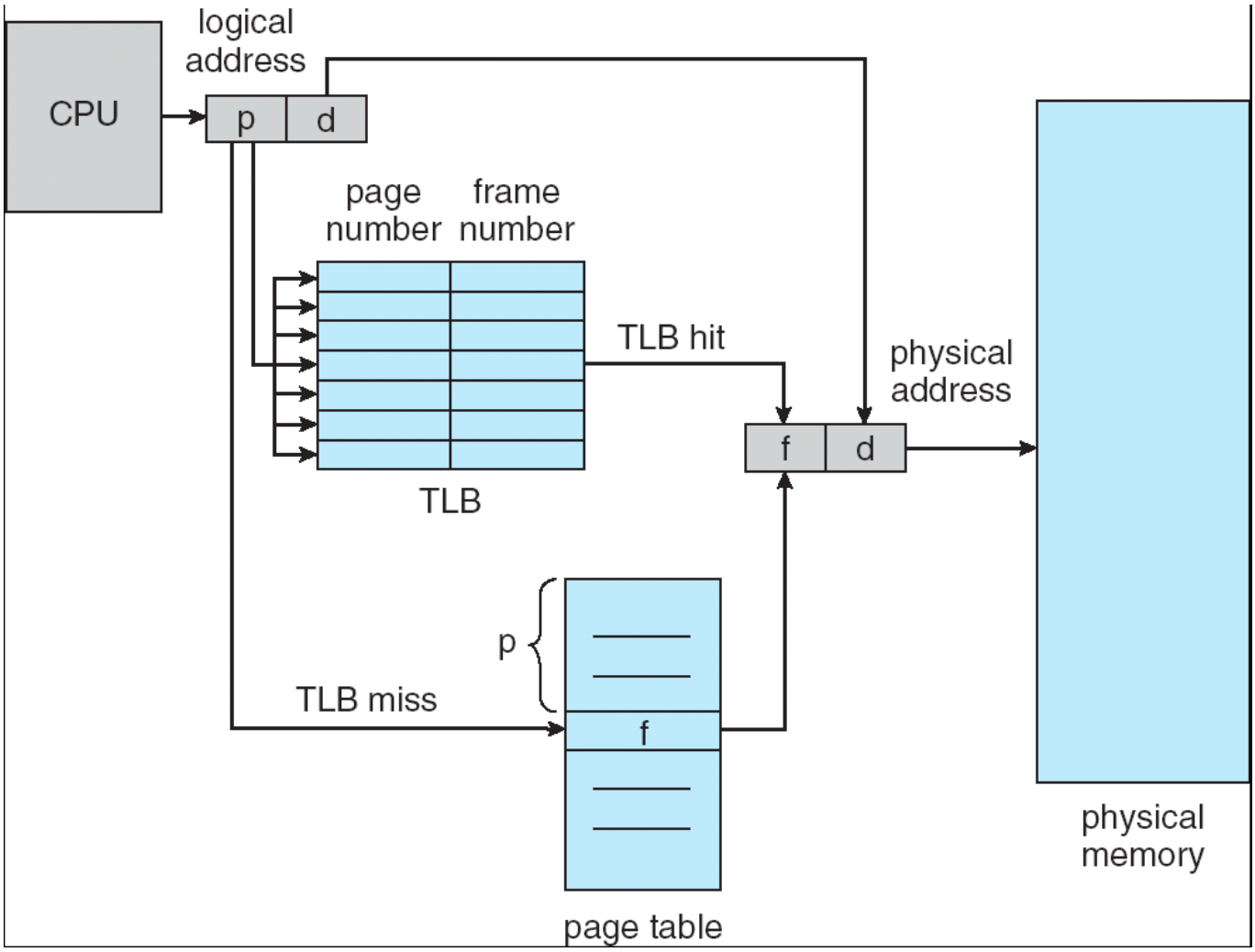
Quando il sistema deve tradurre un indirizzo virtuale:

- Confronta la chiave generata con le chiavi presenti nel TLB.
    - **Se la chiave è presente (hit):** Viene utilizzato l'indirizzo fisico memorizzato nel TLB per accedere rapidamente alla memoria.

- **Se la chiave non è presente (miss):** Deve accedere alla tabella delle pagine in memoria, aggiornando poi il TLB per future traduzioni rapide.

In sintesi, il TLB riduce il bisogno di accedere frequentemente alla tabella delle pagine completa, migliorando le prestazioni complessive del sistema.

(**Translation Lookaside Buffer**. È una memoria cache che memorizza le traduzioni di indirizzi virtuali in indirizzi fisici, riducendo il tempo necessario per accedere alla memoria fisica quando un indirizzo virtuale viene utilizzato frequentemente).



1. Sostituzione nella TLB (Translation Lookaside Buffer) con Algoritmi di Rimpiazzo

Quando si verifica un **TLB miss** e il TLB è pieno, bisogna sostituire una delle voci presenti con la nuova traduzione di indirizzo. Uno degli algoritmi utilizzati per decidere quale voce rimpiazzare è l'**LRU (Least Recently Used)**. Questo algoritmo sostituisce la voce che non è stata utilizzata per il periodo di tempo più lungo, sotto l'assunzione che le voci utilizzate più di recente saranno probabilmente riutilizzate prima di quelle usate meno recentemente.

2. Pagine Vincolate (Wired-Down) nella TLB

Alcune pagine possono essere vincolate nel TLB, ovvero mantenute permanentemente nella cache del TLB. Questo è particolarmente utile per le pagine del **kernel** del sistema operativo, che sono frequentemente utilizzate e per le quali una traduzione rapida è critica per le prestazioni e la stabilità del sistema. Vincolare queste pagine riduce la probabilità di TLB miss per operazioni critiche.

3. Utilizzo di ASID (Address Space Identifier) nella TLB

Un **ASID (Address Space Identifier)** è un identificatore associato ai processi e memorizzato insieme alle voci del TLB. Quando un processo tenta di accedere a una pagina di memoria, il sistema operativo controlla l'ASID per verificare se il processo è autorizzato ad accedere a quella pagina. Se l'ASID non corrisponde all'identificatore del processo attivo, viene generato un **TLB miss**, anche se la voce è presente nel TLB. Questo meccanismo garantisce la sicurezza e l'isolamento tra i processi, prevenendo accessi non autorizzati alla memoria.

Protezione della Memoria nella Paginazione

La paginazione assicura che i processi possano accedere solo alle aree di memoria assegnate. Questo è ottenuto tramite registri privilegiati, bit di protezione nelle tabelle delle pagine, e bit di validità, garantendo un ambiente di esecuzione sicuro e l'isolamento dei processi.

30. **Valori dei Registri**
    - I valori dei registri possono essere modificati solo da istruzioni privilegiate del sistema operativo. Questo impedisce ai processi utente di alterare i valori critici, garantendo così la sicurezza del sistema.
31. **Bit Aggiuntivi nella TDP**
    - L'aggiunta di alcuni bit alle righe della Tabella delle Pagine (TDP) permette di definire e controllare il tipo di accesso consentito alla pagina, come lettura (r), scrittura (rw) ed esecuzione (exe). Questi bit aiutano a prevenire accessi non autorizzati o indesiderati.
32. **Protezione Interna allo Spazio di Indirizzamento del Processo**
    - Non ha senso implementare una protezione interna allo spazio di indirizzamento di un processo, poiché non prevede la tipizzazione dei dati. La protezione viene gestita principalmente dal sistema operativo tramite i bit di accesso nella TDP.
33. **Bit di Validità**
    - Il bit di validità indica se la pagina corrispondente è effettivamente utilizzata dal processo. Un processo può usare solo un sottoinsieme delle pagine a sua disposizione. Pagine non valide non vengono mappate alla memoria fisica, prevenendo accessi non autorizzati.
34. **Registro della Lunghezza della Tabella delle Pagine (PTLR)**

- Alcuni sistemi utilizzano il PTLR per indicare il numero di pagine appartenenti a un processo. Questo registro aiuta il sistema operativo a gestire e proteggere le pagine di memoria assegnate a ogni processo.

CONDIVISIONE

La condivisione delle pagine nella paginazione permette a più processi di accedere a una singola copia di una pagina fisica, ottimizzando l'uso della memoria. Questo è gestito in modo trasparente dai programmi di sistema, garantendo l'integrità dei dati attraverso la mutua esclusione e migliorando la comunicazione tra processi tramite pagine condivise.

- **Copia Unica di una Pagina Fisica:**
  - Una singola copia di una pagina fisica può essere mappata in molti spazi di indirizzamento. Questo significa che diversi processi possono condividere la stessa pagina fisica, permettendo un uso efficiente della memoria.
- **Accessi Differenti:**
  - I processi che condividono una pagina fisica possono avere diversi tipi di accesso a quella pagina. Ad esempio, un processo potrebbe avere accesso in sola lettura mentre un altro potrebbe avere accesso in lettura-scrittura. Questo viene gestito dai meccanismi di protezione del sistema operativo.
- **Trasparenza della Paginazione:**
  - La condivisione delle pagine è gestita dai programmi di sistema, rendendo il processo trasparente per l'utente finale. L'utente non deve preoccuparsi della gestione della memoria condivisa, poiché il sistema operativo si occupa di tutto.
- **Mutua Esclusione:**
  - Il codice condiviso deve essere eseguito in mutua esclusione per evitare conflitti e garantire l'integrità dei dati. Questo significa che solo un processo alla volta può eseguire il codice condiviso.
- **Messaggistica tra Processi:**
  - Alcuni sistemi utilizzano le pagine condivise per inviare messaggi tra processi. Questo metodo di comunicazione è efficiente e permette ai processi di scambiarsi dati rapidamente.

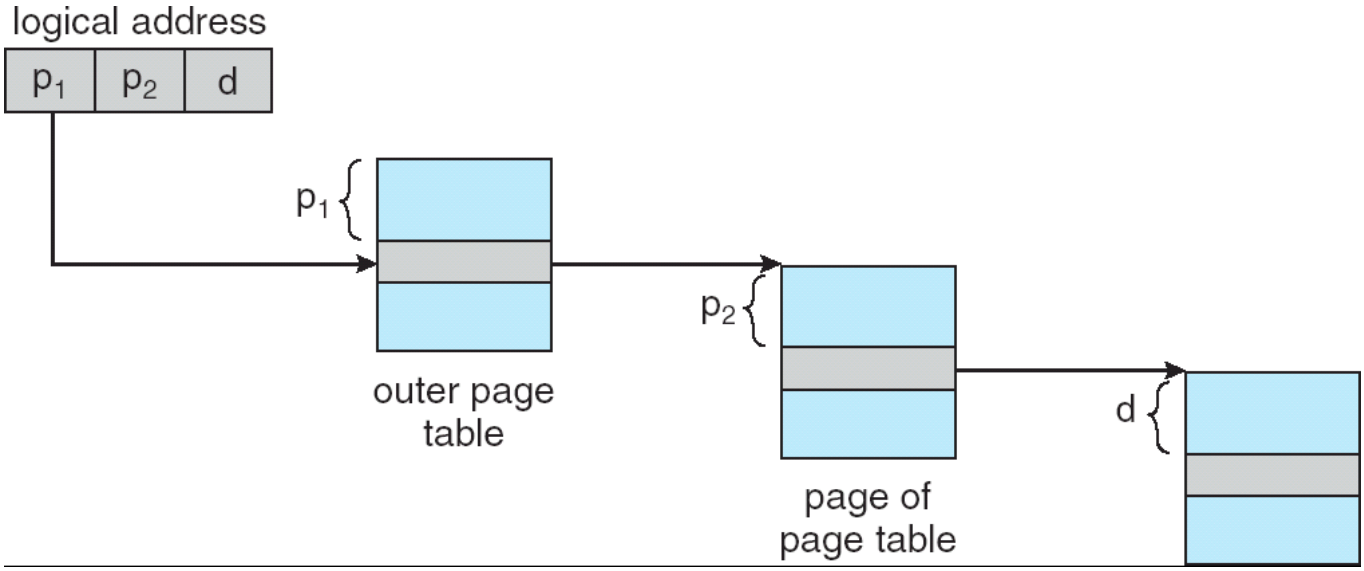
Paginazione Gerarchica

Problema

Nelle moderne architetture a 32 e 64 bit, la dimensione della tabella delle pagine può diventare estremamente grande, arrivando ad occupare diversi megabyte (MB) per ogni processo. Gestire queste tabelle di grandi dimensioni in modo efficiente è una sfida significativa.

Soluzione: Paginazione a Due Livelli

Per risolvere questo problema, viene utilizzata una tecnica chiamata **paginazione gerarchica** o **paginazione a due livelli**. Questa tecnica divide la tabella delle pagine in più livelli, riducendo la necessità di memorizzare intere tabelle di grandi dimensioni in memoria.



35. **Suddivisione dell'Indirizzo Logico**
    - L'indirizzo logico viene suddiviso in tre parti:
        - **P1**: Indirizzo della tabella di primo livello
        - **P2**: Scostamento all'interno della pagina puntata da P1
        - **d**: Offset all'interno della pagina finale
    - Diagramma di suddivisione:

```
page number | page offset
--- | ---
p1 | p2 | d
```

36. **Funzionamento del Sistema**
    - **P1** indica l'indirizzo della tabella di primo livello, che contiene puntatori a diverse tabelle di secondo livello.
    - **P2** rappresenta lo scostamento all'interno della pagina puntata da P1 e serve per identificare la pagina di secondo livello corretta.
    - **d** è l'offset finale all'interno della pagina fisica effettiva.

Cache $ Memory: Interposizione tra CPU e RAM

37. **Accesso alla Cache tramite Indirizzo Fisico:**

- Quando la cache è inserita tra la CPU e la RAM, l'accesso alla cache può avvenire utilizzando l'indirizzo fisico di memoria.
- Il processo inizia con la CPU che genera un indirizzo virtuale.
- Prima di accedere alla cache, l'indirizzo virtuale viene tradotto in un indirizzo fisico attraverso l'**MMU (Memory Management Unit)**.
- Questo indirizzo fisico viene poi usato per cercare i dati nella cache.

38. **Accesso alla Cache tramite Indirizzo Virtuale:**
    - Alcune architetture consentono l'accesso alla cache direttamente tramite indirizzi virtuali, anche se questa pratica è rara.
    - In questi casi, la traduzione dall'indirizzo virtuale a quello fisico può essere bypassata per accelerare l'accesso ai dati, ma comporta complessità aggiuntive nella gestione della coerenza della cache.

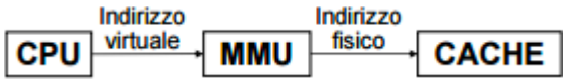


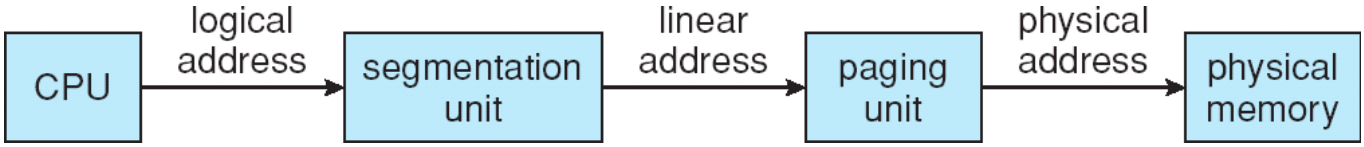
Diagramma di Flusso

Il diagramma mostra il flusso di dati dalla **CPU** all'**MMU**, e poi alla **CACHE** con le etichette "Indirizzo virtuale" e "Indirizzo fisico". Questo rappresenta visivamente come un indirizzo generato dalla CPU viene trasformato e utilizzato per accedere ai dati nella cache.

In sintesi, la reintroduzione della cache tra la CPU e la RAM permette di velocizzare l'accesso ai dati frequentemente utilizzati, riducendo i tempi di latenza complessivi e migliorando le prestazioni del sistema.

INTEL PENTIUM

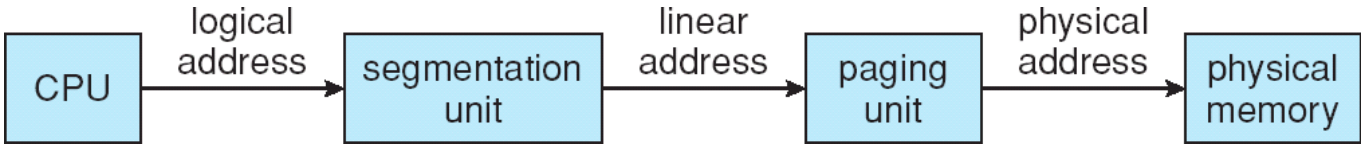
Processo di indirizzamento della memoria nel processore Intel Pentium. Questo processo coinvolge diverse fasi cruciali, dalla generazione degli indirizzi logici fino all'accesso alla memoria fisica. Attraverso la segmentazione e la paginazione, il processore ottimizza la gestione della memoria, garantendo l'isolamento e la protezione dei dati.



Processo di indirizzamento della memoria:
39. **Indirizzi logici**:
- Il processore genera indirizzi logici durante l'esecuzione delle istruzioni. Questi indirizzi rappresentano la posizione degli oggetti nella memoria virtuale del programma.

40. **Unità di segmentazione**:
- Gli indirizzi logici vengono prima processati dall'unità di segmentazione. Questa unità divide la memoria in segmenti, ciascuno con un punto di partenza e definito da una lunghezza.
- La segmentazione permette la protezione della memoria e la gestione flessibile dei dati, poichè i segmenti possono crescere indipendentemente.
- La Intel utilizza una tecnica di segmentazione con paginazione, cioè continua a contempleare i segmenti, ma ogni segmento è costituito da pagine. Questo significa che l'indirizzo generato dalla CPU contiene l'indicazione del segmento. Dopo viene risolto il segmento. Ad esempio, il segmento 6 inizia dalla locazione 100. Dopo, l'indirizzo va all'unità di paginazione, quindi pagina 5, cioè pagina 5 del segmento 1. La parte finale va concatenata all'indirizzo fisico per poter accedere alla memoria.
- Il risultato della segmentazione è un **indirizzo lineare**.
41. **Unità di paginazione**:
- Gli indirizzi lineari ottenuti dalla segmentazione vengono poi processati dall'unità di paginazione.
- La paginazione divide la memoria in piccoli blocchi di dimensione fissa chiamati **pagine**.
- L'unità di paginazione trasforma l'indirizzo lineare in un **indirizzo fisico**, che corrisponde a una posizione specifica nella memoria fisica (RAM).

Schema del Processo di Indirizzamento:



L'immagine illustra questo processo con un diagramma che segue questi passaggi:
42. Il processore (CPU) genera un indirizzo logico.
43. L'indirizzo logico è inviato all'unità di segmentazione, che produce un indirizzo lineare.
44. L'indirizzo lineare è poi elaborato dall'unità di paginazione, che genera l'indirizzo fisico.
45. L'indirizzo fisico è utilizzato per accedere alla memoria fisica.

Nota Importante:

- I segmenti sono costituiti da pagine. Questo significa che ogni segmento può essere ulteriormente diviso in pagine, permettendo un'allocazione della memoria ancora più dettagliata e flessibile.

MEMORIA VIRTUALE

In Generale:

- Il concetto di memoria virtuale è quello di poter fare idealmente riferimento a una memoria virtuale più grande della memoria fisica reale RAM. La memoria virtuale è la somma della memoria fisica più una parte di hard disk SWAP.

La **memoria virtuale** è una tecnica utilizzata dai sistemi operativi per ottimizzare l'uso della memoria. Ecco i punti chiave:

Concetti Principali:

46. **Spazio di Indirizzamento Virtuale**:
    - La memoria virtuale consente a ogni processo di avere uno spazio di indirizzamento più grande della memoria fisica disponibile. Solo una parte dello spazio di indirizzamento virtuale di un processo è effettivamente caricata in memoria.
47. **Capacità della Memoria Fisica**:
    - La somma degli spazi di indirizzamento virtuale di tutti i processi attivi può superare la capacità della memoria fisica.
    - Un singolo processo può avere uno spazio di indirizzamento virtuale maggiore della memoria fisica disponibile.
48. **Uso Raro delle Parti del Processo**:
    - Molte parti di un processo vengono usate raramente. La memoria virtuale permette di caricare in memoria solo le parti necessarie.
49. **Apparente Uso della Memoria**:
    - Con la memoria virtuale, sembra che l'uso complessivo della memoria superi il 100%, poiché il sistema può gestire più memoria di quanto ne sia fisicamente disponibile.

Vantaggi della Memoria Virtuale:

- **Isolamento dei Processi**:
    - Ogni processo funziona nel proprio spazio di indirizzamento virtuale, riducendo il rischio di interferenze tra i processi.
- **Efficienza della Memoria**:
    - Consente di eseguire applicazioni più grandi della memoria fisica disponibile, migliorando l'efficienza complessiva del sistema.

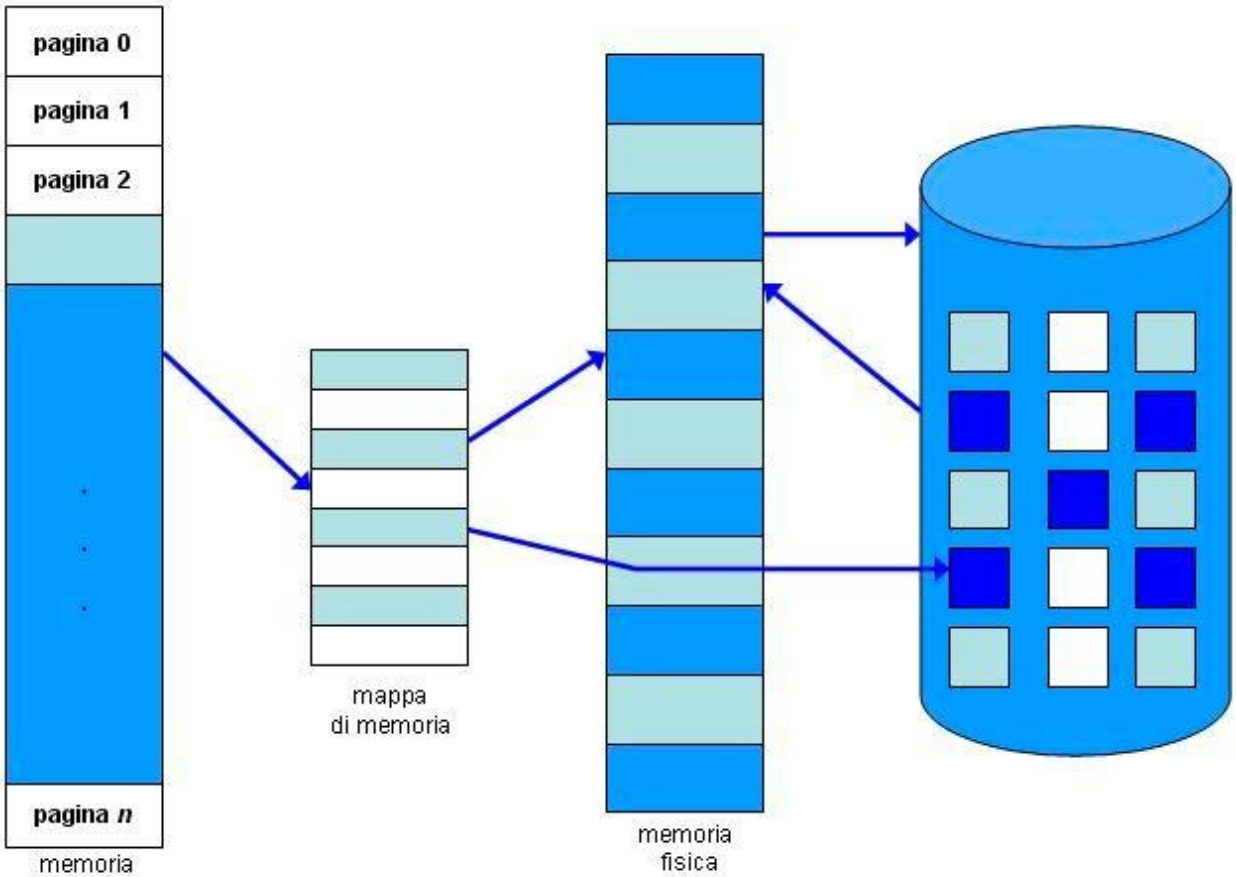
Realizzazione della Memoria Virtuale:

- La memoria virtuale può essere realizzata in modo semplice come **paginazione su richiesta**.
- In questa modalità, prendo un processo e lo divido in pagine. Dopo aver diviso il processo in pagine, siccome non so da dove iniziare (ad esempio, dalla parentesi graffa aperta), ma non conosco il flusso dei comandi `do while`, `while`, `if`, sfrutto la paginazione.

- Dico che questo processo, che vuole essere eseguito, è costituito da 200 pagine, ciascuna di 8 KB. Ho tagliato il processo in pagine.
- Quale pagina porto in memoria centrale? Inizio con la prima parentesi graffa aperta, mentre tutte le altre pagine vanno in memoria di massa.
- Idealmente, al primo giro di esecuzione, la prima pagina è caricata in memoria RAM e le restanti 199 pagine sono tutte su hard disk in un'area di SWAP.
- Cosa accade? Virtualmente vediamo che le pagine 0, 1, 2 e 3 sono occupate realmente. Quelle pagine possono essere nella RAM o nello SWAP.
- Il problema è che la CPU non sa dove si trovano. La CPU dice "pagina 5, displacement 28". Nasce una nuova esigenza: capire se la pagina 5 è in RAM o su hard disk.
- Se la pagina 5 è in RAM, abbiamo un *hit*; se non lo è, abbiamo un *miss*, che in realtà si chiama **page fault**(fallimento di pagina). Indichiamo un accesso alla pagina che non era in memoria RAM, ma che occorre recuperare dallo SWAP.

Gestione dei Page Fault:

50. Serve una mappa di memoria che ci indichi dove si trovano le pagine.
51. Dobbiamo ricordare che il page fault genera un'attesa connessa all'hard disk, poiché la pagina deve essere trasferita sulla memoria RAM. Inoltre, se l'hard disk è pieno, ci troveremo a dover scaricare una pagina per far spazio.



Schema che mostra una memoria virtuale più grande di quella fisica.

Componenti Principali:

52. **Memoria Virtuale**:
    - La memoria virtuale è rappresentata sul lato sinistro ed è divisa in pagine numerate (pagina 0, pagina 1, pagina 2, ..., pagina n).
    - Ogni pagina della memoria virtuale è una porzione dell'indirizzo virtuale di un processo.
53. **Mappa di Memoria**:
    - Al centro, troviamo la mappa di memoria che serve da ponte tra la memoria virtuale e la memoria fisica.
    - La mappa di memoria tiene traccia di dove si trovano le pagine: se sono caricate in RAM (memoria fisica) o se sono memorizzate su disco (swap).
54. **Memoria Fisica**:
    - La memoria fisica è mostrata sul lato destro ed è composta da vari blocchi.
    - Alcuni di questi blocchi sono collegati a un dispositivo di archiviazione (come un hard disk), rappresentando le pagine memorizzate in swap.

Processo di Accesso:

55. **Generazione dell'Indirizzo**:
    - La CPU genera un indirizzo virtuale che include il numero di pagina e il displacement (offset) all'interno della pagina.
56. **Controllo della Mappa di Memoria**:
    - La mappa di memoria viene consultata per determinare se la pagina richiesta è in RAM o su disco.
    - Se la pagina è in RAM, abbiamo un **hit** e l'accesso può procedere rapidamente.
    - Se la pagina non è in RAM, si verifica un **page fault** e il sistema deve recuperare la pagina dal disco (swap) per caricarla in RAM. Questo causa un ritardo.
57. **Page Fault**:
    - Quando si verifica un page fault, il sistema deve aggiornare la mappa di memoria e caricare la pagina richiesta in RAM. Se la RAM è piena, una pagina esistente deve essere scaricata per fare spazio.

PROSPETTIVE DELLA MEMORIA VIRTUALE

PUNTO DI VISTA DEL PROGRAMMATORE

- **Trasparenza**:
    - I dettagli della gestione della memoria virtuale sono trasparenti per il programmatore. Questo significa che il programmatore non deve preoccuparsi di come la memoria è gestita dal sistema operativo.

- **Adattabilità**:
  - Non è necessario adattare un programma a una memoria fisica limitata. Un programma può essere eseguito su sistemi con diverse disponibilità di memoria fisica senza bisogno di modifiche.
- **Esecuzione su Sistemi Diversi**:
  - Un programma può essere eseguito su sistemi con diverse disponibilità di memoria fisica senza alcun adattamento, rendendo il software più flessibile e portabile (gira su un pc da 32 gb gira su uno da 16 gb di ram ecc.).

PUNTO DI VISTA DEL SISTEMA OPERATIVO

- **Caricamento dei Processi**:
  - Un processo può essere caricato in uno spazio di dimensione arbitraria, indipendentemente dalla dimensione della memoria fisica disponibile.
- **Ordine di Esecuzione**:
  - Non è necessario modificare l'ordine di esecuzione previsto di un processo, anche se la memoria fisica disponibile è limitata.
- **Quantitativo di Memoria**:
  - Il quantitativo di memoria assegnato a un processo può variare durante la sua esecuzione. Il sistema operativo può decidere dinamicamente di allocare più memoria a un processo per velocizzarne l'esecuzione o ridurre la memoria per aumentare il grado di multiprogrammazione.
- **Multiprogrammazione**:
  - Il sistema operativo può aumentare il grado di multiprogrammazione, ovvero eseguire più processi contemporaneamente, grazie alla flessibilità offerta dalla memoria virtuale.
- **Frammentazione Esterna**:
  - La memoria virtuale riduce la frammentazione esterna, che è lo spreco di memoria causato dalla distribuzione inefficiente delle risorse in blocchi non contigui.

OSSERVAZIONI

58. **Completamento delle Istruzioni**:
    - Un'istruzione può essere completata soltanto se tutto ciò che è necessario (codice, stack, dati) è presente in memoria. Questo significa che per eseguire correttamente un'istruzione, il processore deve avere accesso immediato a tutte le risorse necessarie.
59. **Processi Bloccati**:
    - Un processo che richiede un "oggetto" fuori dalla memoria viene sospeso (blocked) per un tempo "relativamente" lungo. Quando un processo ha bisogno di dati che non sono

attualmente in memoria, deve attendere che questi dati vengano caricati, causando un'interruzione dell'esecuzione del processo.

- **Esempio**: Immagina che il programma "somma" si trovi in un'area di swap. Non lo possiamo eseguire perché **UN PROGRAMMA PER ESSERE ESEGUITO DEVE ESSERE SE E SOLO SE SI TROVA NELLA MEMORIA RAM**. Possiamo andare all'istruzione successiva? No, qualcuno deve accorgersi che quella pagina non era nella memoria RAM. Quando una pagina non è in memoria, si verifica un **FAULT**. Il processo andrà in stato di blocked, in attesa del caricamento della pagina mancante.

Importanza dell'uso dello schema di memoria virtuale

- **Non tutte le Parti del Programma Vengono Eseguite**:
  - Durante una specifica esecuzione, alcune parti del programma non vengono mai indirizzate o eseguite. Questo accade perché, in base al flusso di controllo (if-else-if, switch-case), solo alcune parti del codice vengono effettivamente eseguite.
- **Diversi Tracciati di Esecuzione**:
  - Le diverse condizioni interne o esterne causano diversi tracciati di esecuzione ogni volta che un processo viene eseguito. Ad esempio, un programma potrebbe seguire percorsi diversi in base agli input ricevuti o alle condizioni del sistema.
- **Esecuzione Rara di Alcune Parti del Programma**:
  - Alcune parti di un programma vengono eseguite molto raramente, come ad esempio le routine di gestione degli errori. Queste parti del codice non necessitano di essere costantemente presenti in memoria fisica, permettendo un uso più efficiente della memoria.

BIT DI VALIDITA'

la tabella va aggiornata attraverso il bit di validità, che mi dirà se quella pagina è valida oppure no.
Ma perchè si chiama valido o invalido?
La tabella si trova in RAM, quindi nell'esempio dove ci sono le celle vuote cioè il niente, ma cè un problema ma che all'interno di un calcolatore il niente non esiste dentro cè una configurazione qualunque di bit solo che ha un problema solo che referienza a niente

- **Caricamento dei Processi in Memoria**: Quando un processo viene caricato in memoria, il gestore della memoria prevede quali pagine del processo verranno utilizzate.
- **Trasferimento Selettivo**: Solo le pagine previste vengono trasferite in memoria, invece di caricare l'intero processo.
- **Indicazione Necessaria**: È essenziale avere un meccanismo per indicare se una pagina è in memoria o su disco.

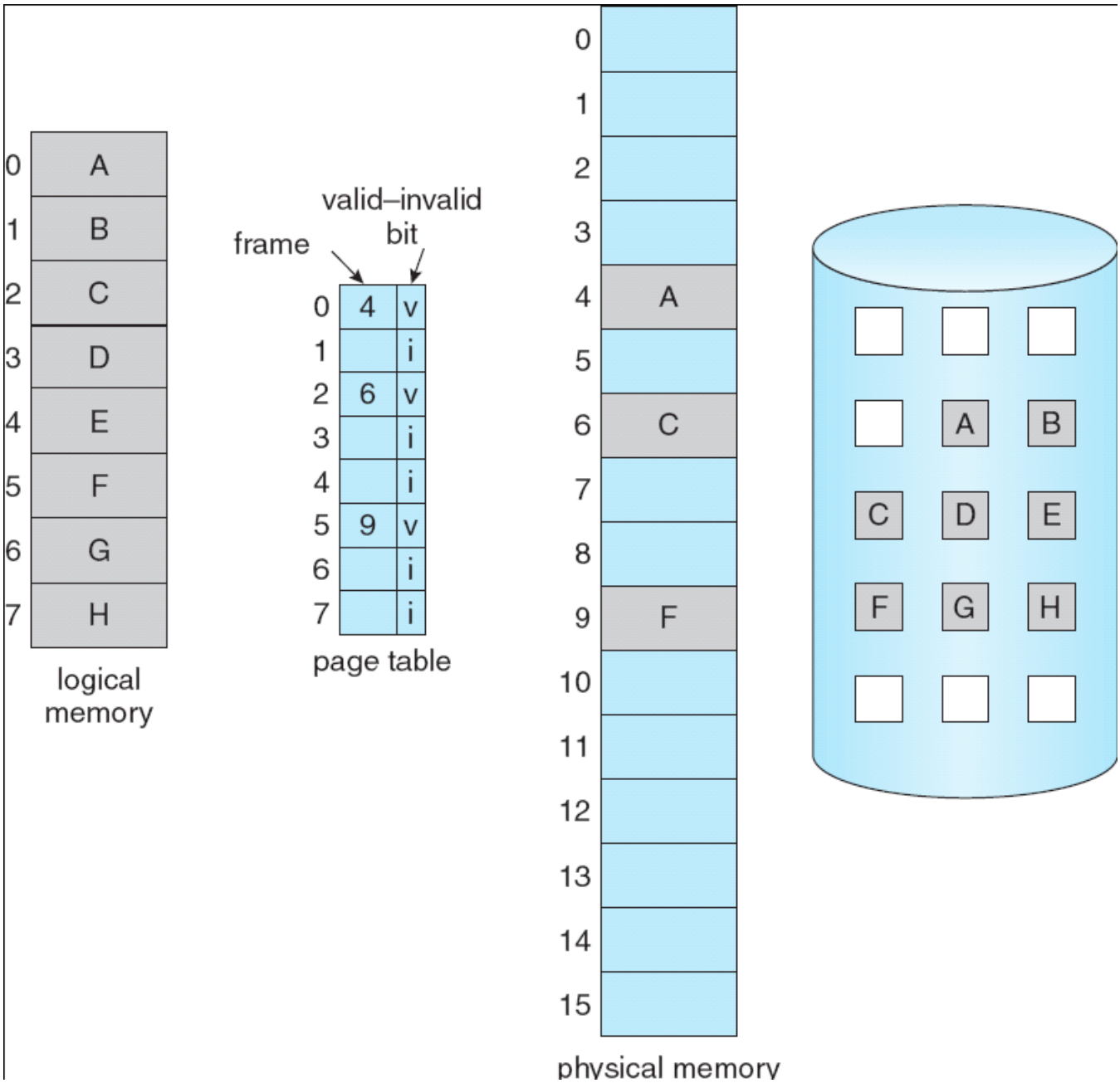
Ruolo del Bit di Validità nella Tabella delle Pagine:

- **Valido (`v`)**: Indica che la pagina è attualmente in memoria.
- **Invalido (`i`)**: Indica che la pagina non è in memoria. Questo può anche significare che la pagina di riferimento non è valida per il processo.

Note Aggiuntive:

- **Interruzione di Pagina**: Si verifica quando si fa riferimento a una pagina che non è presente in memoria centrale, attivando un'eccezione per la pagina mancante.

Questo bit di validità nella tabella delle pagine aiuta a gestire la memoria in modo efficiente, tenendo traccia di quali pagine sono in uso e quali no.



Domanda, ma perchè si chiama bit di validità?
Il termine "bit di validità" viene utilizzato perché questo bit specifico serve a indicare la "validità" di una pagina di memoria, ossia se la pagina è effettivamente presente in memoria centrale. Ecco un po' più di dettaglio:

- **Valido**: Quando il bit è impostato a "1" (o "v"), indica che la pagina corrispondente è attualmente in memoria. Quindi, il processore può accedere direttamente alla pagina senza problemi.
- **Invalido**: Quando il bit è impostato a "0" (o "i"), significa che la pagina non è in memoria. In questo caso, se il processore tenta di accedere a quella pagina, si verifica un "page fault" e il sistema operativo deve intervenire per caricare la pagina da disco a memoria.

PAGE FAULT TRAP

- **Definizione**: Un "page fault" è un'eccezione sollevata dal sistema operativo quando un programma tenta di accedere a una pagina di memoria che non è attualmente presente nella memoria centrale (RAM).
- **Motivo dell'eccezione**: Questo può accadere per diverse ragioni:
  - La pagina è stata spostata su disco per liberare spazio in memoria (paginazione).
  - Il programma sta tentando di accedere a una pagina che non ha ancora caricato.

Situazione di Blocco del Processo:

- **Contesto**: Il processo era in esecuzione (run) e ha incontrato l'istruzione `a = somma`, che lo ha portato nello stato di **blocco** (blocked).
- **Domanda**: Il processo è andato in stato di blocco da solo?
  - **Risposta**: No, non è accaduto autonomamente.

Dettaglio Tecnico dell'Evento:

60. **Generazione dell'Indirizzo**: La CPU ha generato un indirizzo di memoria, specificamente pagina 1, riga 5.
61. **Verifica del Sistema Operativo**: Durante questa operazione, il sistema operativo ha rilevato che il bit di validità dell'indirizzo era impostato su invalido.
62. **Conseguenza del Bit Invalido**:
    - Questo indica che la pagina richiesta non è presente in memoria centrale.
    - Il processo non può continuare l'esecuzione perché non sa dove trovare i dati necessari.

Generazione della TRAP:

- **Cos'è una TRAP?**: Una TRAP è un'eccezione generata internamente dal processo stesso quando incontra un'istruzione come `a = somma`.
- **Differenza tra TRAP e Interruzione (Interrupt)**:
  - Una TRAP è generata dal processo in esecuzione nel momento in cui si verifica un errore.
  - Un Interrupt, invece, è causato da un evento esterno, tipicamente da un dispositivo hardware che interrompe l'esecuzione del processo.

Caso dell'Eccezione di Pagina Mancante:

- Questo evento specifico, una **TRAP** dovuta a una pagina non presente in memoria centrale, è un esempio di gestione della memoria e di come il sistema operativo intervenga per risolvere il problema.

GESTIONE DEL PAG FAULT

63. **Rilevamento del Page Fault**: La CPU rileva che la pagina richiesta non è in memoria e genera una TRAP.
64. **Intervento del Sistema Operativo**: Il sistema operativo interviene e controlla la causa del page fault.
    - Se la pagina è semplicemente su disco, la carica in memoria.
    - Se il tentativo di accesso è illegittimo, genera un errore di accesso o termina il processo.
65. **Ripristino del Processo**: Una volta caricata la pagina in memoria, il sistema operativo riprende l'esecuzione del processo dal punto in cui è stato interrotto.

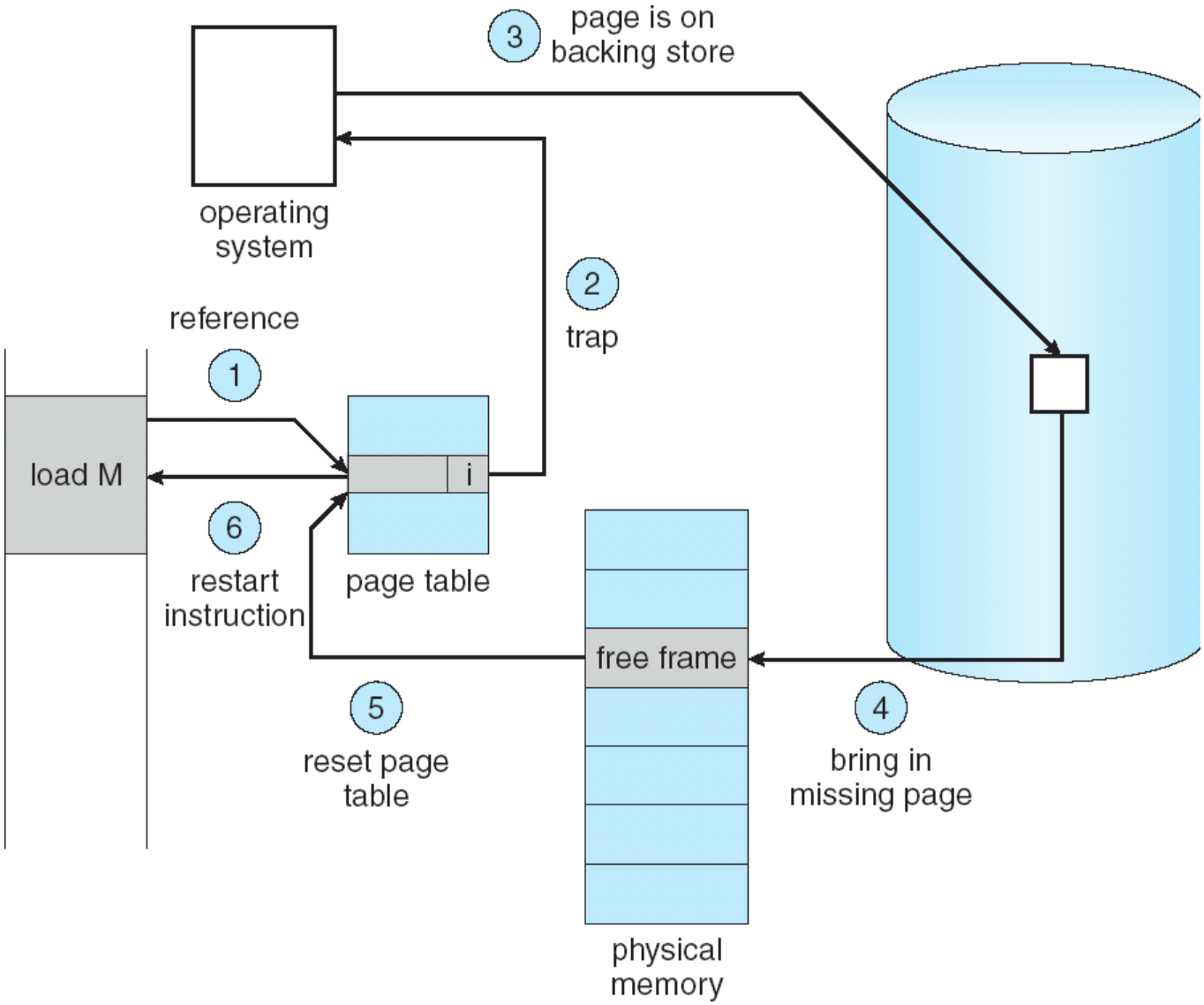


Diagramma dei Passaggi:

66. **Riferimento**: Il sistema operativo fa riferimento a una pagina.
67. **Trap**: Si verifica un'interruzione perché la pagina non è in memoria.
68. **Pagina su backing store**: Il sistema operativo controlla se la pagina è sul backing store.
69. **Portare la pagina mancante**: La pagina mancante viene portata in un frame libero nella memoria fisica.
70. **Reimpostare la tabella delle pagine**: La tabella delle pagine viene aggiornata per riflettere la nuova posizione della pagina.
71. **Riavviare l'istruzione**: L'istruzione che ha causato il page fault viene riavviata.

SPIEGAZIONE TESTUALE

- **Il sistema operativo (attraverso l'uso di una tabella esterna indicata nel PCB) determina se**:
  - Riferimento non valido → abortisce

    - Pagina non in memoria
- **Individuazione di un blocco libero**
- **Trasferimento della pagina nel blocco**
- **Aggiornamento della tabella delle pagine (include, bit di validazione = v)**
- **Riavviare l'istruzione che ha causato il page fault... questa volta la pagina referenziata sarà in memoria...**

In pratica, quando il sistema operativo cerca di accedere a una pagina che non si trova in memoria, si verifica un page fault. Il sistema quindi controlla se la pagina è presente nel backing store (un'area di memoria secondaria). Se la pagina è presente, viene caricata in un blocco libero della memoria fisica, la tabella delle pagine viene aggiornata e l'istruzione viene riavviata.

domanda è può accadere che mente il processo è in attesa di un evento sia spostato in blocked souspended? ce un processo che aveva bisogno di un pezzo che sta in mmeoria di mette in coda

PAGINAZIONE SU RICHIESTA

PAGINA SU RICHIESTA PURA:

72. **Richiesta di una Pagina**:
    - Una pagina viene trasferita in memoria solo quando viene richiesta.
    - È possibile avviare l'esecuzione di un processo senza pagine in memoria.
73. **Generazione di un Page Fault**:
    - Si genera un page fault al primo riferimento a una pagina non in memoria.
    - Dopo che la pagina viene caricata, l'esecuzione del processo continua fino al prossimo page fault.
74. **Meccanismi Ausiliari per il SO**:
    - **Tabella delle pagine** con bit di validità per ogni pagina.
    - **Memoria ausiliaria**:
        - La parte della memoria ausiliaria dedicata al processo di swapping è chiamata area di swap.

Spiegazione

La paginazione su richiesta è un metodo per gestire la memoria in cui le pagine non sono caricate in memoria fisica finché non sono necessarie. Quando un processo cerca di accedere a una pagina che non è presente in memoria, si verifica un page fault. Il sistema operativo intercetta il page fault, controlla la tabella delle pagine e carica la pagina richiesta dalla

memoria ausiliaria nell'area di swap alla memoria fisica. Una volta caricata, il processo può continuare fino a quando non si verifica un altro page fault.

PERFORMANCE DELLA PAGINAZIONE SU RICHIESTA

Fino a quando non si verificano page fault, il tempo di accesso effettivo è uguale al tempo di accesso alla memoria (nella ipotesi di assenza della cache $)

Questa pagina parla della performance dei sistemi di paginazione su richiesta e introduce il concetto di **tempo di accesso effettivo (EAT Effective Access Time)** in presenza di page faults.

Tempo di Accesso Effettivo (EAT)

- **Definizione**:
  - L'EAT rappresenta il tempo medio necessario per accedere ai dati in memoria, considerando sia gli accessi riusciti che i page fault.
- **Formula**:
  - EAT=(1−p)×Tempo di Accesso alla Memoria+p×Tempo di Gestione del Page Fault • Dove:
    - p = probabilità di page fault.
    - Tempo di Accesso alla Memoria = tempo per accedere ai dati quando non c'è page fault.
    - Tempo di Gestione del Page Fault = tempo necessario per gestire un page fault.

Gestione del Page Fault

75. **Generazione della Trap**:
    - Quando si verifica un page fault, il processore genera una trap, un segnale che indica un errore di memoria.
76. **Salvataggio del Contesto del Processo**:
    - Il sistema salva lo stato attuale del processo, permettendo di riprendere l'esecuzione successivamente.
77. **Determinazione della Natura della Trap**:
    - Il sistema operativo verifica il tipo di errore e decide le azioni correttive necessarie.
78. **Controllo del Riferimento**:
    - Verifica se l'indirizzo richiesto è valido e se appartiene al processo corrente.
79. **Lettura dal Disco & Trasferimento Dati**:

- Il sistema legge la pagina mancante dal disco e la carica in memoria, aggiornando le tabelle delle pagine.

80. **Generazione di un Interrupt (I/O Completato)**
    - Quando un'operazione di I/O è completata, il dispositivo di I/O genera un interrupt per notificare al processore il completamento dell'operazione. Questo segnala al sistema operativo che è necessario intervenire per gestire il page fault.
81. **Salvataggio del Contesto del Processo nello Stato di Run**
    - Quando l'interrupt viene gestito, il sistema operativo salva lo stato corrente del processo in esecuzione. Questo stato include i registri della CPU, il contatore del programma e altre informazioni necessarie per ripristinare il processo esattamente da dove era stato interrotto.
82. **Gestione dell'Interruzione**
    - Il sistema operativo gestisce l'interrupt determinando quale tipo di interrupt è stato generato e quale azione è necessaria per risolverlo. Nel caso di un page fault, il sistema deve recuperare la pagina mancante dalla memoria secondaria e caricarla nella memoria principale.
83. **Aggiornamento della Tabella delle Pagine per il Processo che Aveva Generato il Page Fault**
    - Durante la gestione del page fault, la tabella delle pagine del processo viene aggiornata. Questo comporta:
        - Caricamento della pagina mancante dalla memoria secondaria alla memoria principale.
        - Modifica della tabella delle pagine per riflettere che la pagina è ora presente in memoria.
        - Cambiamento dello stato del processo da **blocked** a **ready**.
84. **Attesa in Ready**
    - Il processo aggiornato viene inserito nella coda dei processi pronti (ready). Qui, attenderà il suo turno per essere schedulato ed eseguito di nuovo dalla CPU.
85. **Ripristino del Contesto**
    - Quando il processo viene nuovamente schedulato per l'esecuzione, il sistema operativo ripristina il contesto salvato. Questo include il ripristino dei registri della CPU, il contatore del programma e altre informazioni critiche, permettendo al processo di riprendere esattamente dal punto in cui era stato interrotto.

ESEMPIO

Esempio:

- Memory access time = 100 nsec
- Average page-fault service time = 25 msec

Calcolo dell'EAT (Effective Access Time):

Formula:

EAT=(1−p)×Tempo di Accesso alla Memoria+p×Tempo di Gestione del Page Fault

Sostituzione dei Valori:

EAT=(1−p)×100 ns+p×25 ms
Dato che 1 millisecondo (ms) equivale a 1,000,000 nanosecondi (ns):
25 ms=25,000,000 ns

Quindi:
EAT=(1−p)×100 ns+p×25,000,000 ns
EAT=100 ns+p×24,999,900 ns

Esempio Specifico:

Se un accesso ogni 1000 genera un page fault, la probabilità di page fault p è:
p= 1 / 1000=0.001

- Se un accesso ogni 1000 genera un page fault: EAT = 25,000 nsec

RALLENTAMENTO DEL 250%!!!
Per avere un rallentamento del 10% il sistema deve garantire meno di una assenza ogni 2,500,000 di riferimenti

Calcolo dell'EAT con p=0.001:

EAT=100 ns+0.001×24,999,900 ns
EAT=100 ns+24,999.9 ns
EAT=25,099.9 ns

Quindi, in questo caso, l'EAT è di circa 25,100 nanosecondi, il che rappresenta un rallentamento significativo rispetto al tempo di accesso senza page fault.

Rallentamento del Sistema:

Se il tempo di accesso senza page fault è di 100 nanosecondi e con page fault è di 25,100 nanosecondi, il rallentamento è:

25,100 /100 ns = 251

Il che rappresenta un rallentamento del 250%!

Per un Rallentamento del 10%:

Per ottenere un rallentamento del 10%, l'EAT Effective Access Time dovrebbe essere:
$EAT=1.1\times100\ ns=110\ ns$
Calcoliamo p per cui:
$110=100+p\times24{,}999{,}900\ ns$
$10=p\times24{,}999{,}900$
$p=10/24{,}999{,}900\approx4\times10^{-7}$

Quindi, il sistema deve garantire meno di una assenza ogni 2,500,000 riferimenti per ottenere un rallentamento inferiore al 10%.

Come Selezionare le Pagine da Portare in Memoria

- **Concetto di Località:** I programmi tendono ad accedere frequentemente a un piccolo sottoinsieme del loro spazio degli indirizzi durante l'esecuzione. Questo sottoinsieme è chiamato località.
- Una parte significativa degli accessi avviene su questo insieme ridotto di pagine virtuali, che tende a cambiare lentamente nel tempo.
- Questa proprietà viene utilizzata per progettare strategie per l'allocazione e la sostituzione delle pagine in memoria.

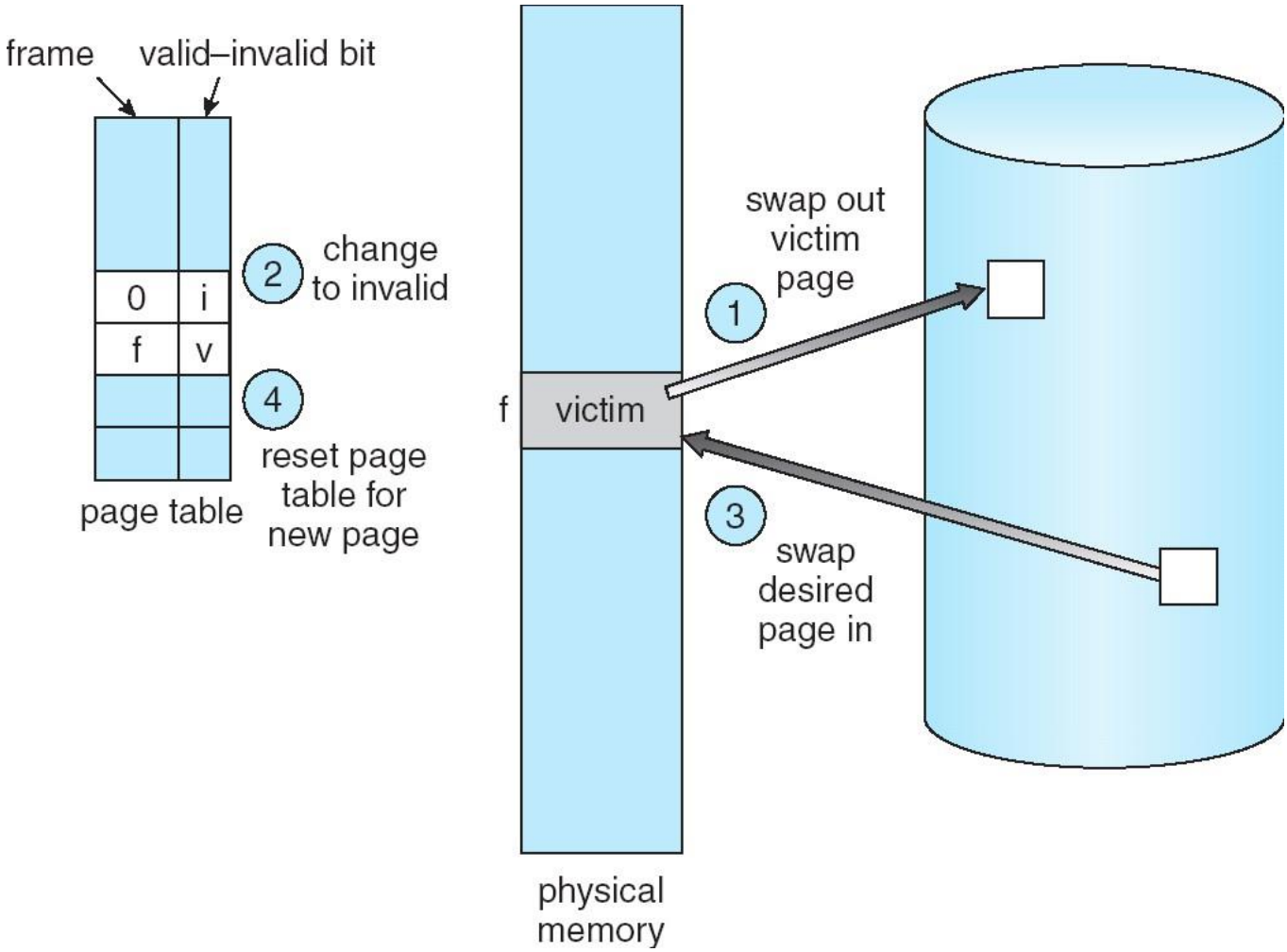
Come Gestire lo Spazio Insufficiente in Memoria

86. **Terminare un Processo Utente:**
    - Questa opzione è considerata estrema e viene utilizzata solo in situazioni critiche. La terminazione di un processo libera immediatamente spazio in memoria, ma può causare perdita di dati o interrompere operazioni importanti.
87. **Scaricare l'Intero Processo dalla Memoria:**
    - Questa tecnica, nota come "swapping", consiste nel trasferire l'intero processo dalla memoria principale al disco. Sebbene liberi spazio in memoria, può ridurre l'efficienza del sistema a causa del tempo necessario per caricare e scaricare i processi.
88. **Sostituzione delle Pagine:**
    - Questa è la strategia più comune e prevede l'uso di algoritmi per determinare quali pagine rimuovere dalla memoria.
        - **Algoritmo LRU (Least Recently Used):** Sostituisce la pagina che non è stata utilizzata per il periodo di tempo più lungo.

- **Algoritmo FIFO (First In, First Out):** Sostituisce la pagina che è stata caricata per prima in memoria.
- **Algoritmo di Conteggio (Counting-Based):** Utilizza contatori per tracciare la frequenza di utilizzo delle pagine e sostituire quelle meno utilizzate.

89. **Utilizzare il Bit di Modifica (Dirty Bit):**
    - Ogni pagina in memoria ha un bit associato che indica se è stata modificata. Solo le pagine modificate (con il bit impostato) devono essere scritte nuovamente sul disco durante la sostituzione, riducendo l'overhead.

PAGE REPLACEMENT



Processo di Sostituzione delle Pagine

90. **Swapping Out della Pagina Vittima:** La pagina scelta come vittima viene rimossa dalla memoria fisica.
91. **Cambio di Stato a Invalid:** Lo stato della pagina nella tabella delle pagine viene cambiato a "invalido", indicando che non è più presente in memoria.

92. **Swapping In della Pagina Desiderata:** La pagina desiderata viene caricata in memoria fisica.
93. **Reset della Tabella delle Pagine:** La tabella delle pagine viene aggiornata per riflettere che la nuova pagina è ora in memoria.

Algoritmo di Sostituzione

- **Minor Page-Fault Possibile:** Si cerca di minimizzare i page-fault scegliendo quale pagina sostituire in modo ottimale.
- **Valutare l'Algoritmo su Sequenze di Accesso:** L'efficacia dell'algoritmo viene valutata su specifiche sequenze di accesso alla memoria.

Esempio di Sequenza di Accesso

Consideriamo la sequenza di accesso:
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

Passaggi Dettagliati

1, 2, 3, 4:
- Supponiamo che abbiamo quattro frame di memoria disponibili. Le prime quattro richieste (1, 2, 3, 4) saranno caricate nei frame uno per uno, causando quattro page fault (una per ogni richiesta).
1, 2:
- Le richieste successive per le pagine 1 e 2 non causano page fault perché sono già presenti nei frame.
5:
- Quando viene richiesta la pagina 5, tutti i frame sono già occupati (1, 2, 3,4). Bisogna scegliere una pagina da sostituire.
- Con l'algoritmo FIFO (First In, First Out), sostituiamo la pagina 1 con la pagina 5.
1:
- La richiesta per la pagina 1 ora causa un page fault perché è stata rimossa. Supponiamo che la pagina 2 sia ora la più vecchia (secondo FIFO) e quindi viene sostituita con la pagina 1.
2:
- La richiesta per la pagina 2 causa un altro page fault. La pagina successiva in FIFO (pagina 3) viene sostituita con la pagina 2.
3:
- Simile al passo precedente, la pagina 3 causa un page fault e sostituisce la pagina 4.
4:
- La pagina 4 causa un page fault e sostituisce la pagina 5.

5:
- La pagina 5 causa un page fault e sostituisce la pagina 1.

ALGORITMI DI REPLACEMENT: FIFO

L'algoritmo FIFO sostituisce le pagine che risiedono in memoria da più tempo. Il gestore della memoria tiene traccia dell'ordine di caricamento delle pagine, spesso utilizzando una coda (FIFO) dei numeri di pagina.

Pregi dell'Algoritmo FIFO

- **Semplice Realizzazione:** Facile da implementare.
- **Aggiornamento Semplice:** La coda viene aggiornata solo in caso di page fault.
- **Non Richiede Supporti Hardware Specifici:** Non necessita di particolari hardware per funzionare.

Difetti dell'Algoritmo FIFO

- **Swap Out di Pagine Frequentemente Indirizzate:** Può rimuovere pagine utilizzate frequentemente perché rimangono in memoria per lungo tempo.
- **Non Tiene Traccia dell'Ordine di Riferimento delle Pagine:** Non considera quando le pagine sono state effettivamente utilizzate per l'ultima volta, portando a sostituzioni non ottimali.

Esempio

stringa 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Con 3 Frame di Memoria:

94. **1, 2, 3, 4**:
    - I primi quattro accessi (1, 2, 3, 4) causano page fault, poiché i frame sono vuoti. Le pagine vengono caricate nei frame uno per uno.
95. **1, 2**:
    - Le richieste successive per le pagine 1 e 2 non causano page fault perché sono già presenti nei frame.
96. **5**:
    - Quando viene richiesta la pagina 5, tutti i frame sono già occupati (1, 2, 3, 4). Bisogna sostituire una pagina. Con FIFO, sostituiamo la pagina 1 (la più vecchia).
97. **1**:

- La richiesta per la pagina 1 ora causa un page fault perché è stata rimossa. La pagina 2 è ora la più vecchia e viene sostituita con la pagina 1.

98. **2**:
    - La richiesta per la pagina 2 causa un altro page fault. La pagina 3 viene sostituita con la pagina 2.
99. **3, 4**:
    - Le richieste per le pagine 3 e 4 causano page fault e sostituiscono le pagine rispettivamente.

Totale di 9 page fault.

Con 4 Frame di Memoria:

100. **1, 2, 3, 4**:
- I primi quattro accessi (1, 2, 3, 4) causano page fault, poiché i frame sono vuoti. Le pagine vengono caricate nei frame uno per uno.

101. **1, 2, 3, 4**:
- Non causano page fault perché le pagine sono già presenti nei frame.

102. **5**:
- La richiesta per la pagina 5 causa un page fault. La pagina 1 (la più vecchia) viene sostituita con la pagina 5.

103. **1**:
- La richiesta per la pagina 1 causa un altro page fault, e sostituisce la pagina 2

104. **2**:
- La pagina 2 causa un altro page fault e sostituisce la pagina 3.

105. **3**:
- La pagina 3 causa un altro page fault e sostituisce la pagina 4.

Totale di 10 page fault.

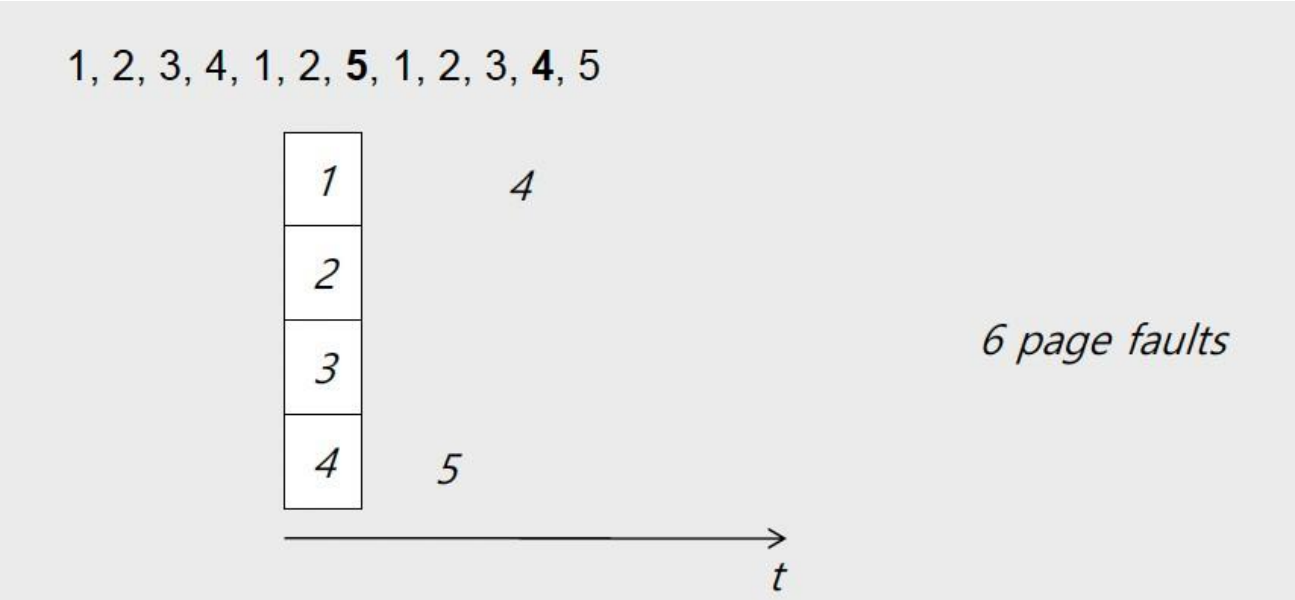
Anomalia di Belady

Questa anomalia si verifica in alcuni algoritmi di sostituzione delle pagine (come FIFO), dove l'aumento del numero di frame può portare a un aumento del numero di page fault.

Algoritmo Ottimale o di Belady

- **Principio di Funzionamento:** Questo algoritmo sostituisce le pagine che non saranno utilizzate per il periodo di tempo più lungo in futuro (pagine che saranno utilizzate dopo molto tempo). L'obiettivo è minimizzare il numero di page faults.

- **Esempio di Sequenza di Accesso:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.



- **Diagramma di Sostituzione:**
  - Inizia caricando le pagine 1, 2, 3 e 4 nei frame.
  - Quando viene richiesta la pagina 5, sostituisce la pagina 1 (non sarà utilizzata a breve).
  - Prosegue sostituendo le pagine in base alla sequenza di accesso futura.

Sequenza di Accesso con 4 Frame

106. **1** - Page fault (frame contiene: 1)
107. **2** - Page fault (frame contiene: 1, 2)
108. **3** - Page fault (frame contiene: 1, 2, 3)
109. **4** - Page fault (frame contiene: 1, 2, 3, 4)
110. **1** - Nessun page fault (già presente)
111. **2** - Nessun page fault (già presente)
112. **5** - Page fault (sostituisce 1) (frame contiene: 2, 3, 4, 5)
113. **1** - Page fault (sostituisce 2) (frame contiene: 3, 4, 5, 1)
114. **2** - Page fault (sostituisce 3) (frame contiene: 4, 5, 1, 2)
115. **3** - Page fault (sostituisce 4) (frame contiene: 5, 1, 2, 3)
116. **4** - Page fault (sostituisce 5) (frame contiene: 1, 2, 3, 4)
117. **5** - Page fault (sostituisce 1) (frame contiene: 2, 3, 4, 5)

Caratteristiche dell'Algoritmo

- **Minimizzazione dei Page Faults:** Questo algoritmo ottimizza il numero di page faults, rendendolo il più efficiente possibile in teoria.

- **Non Realizzabile Praticamente:** Richiede la conoscenza futura degli accessi alla memoria, che non è possibile ottenere nella pratica.
- **Strumento di Confronto:** Viene utilizzato come benchmark per confrontare le prestazioni degli altri algoritmi di sostituzione delle pagine.
- **Non Soggetto all'Anomalia di Belady:** Questo algoritmo non soffre dell'anomalia di Belady, dove l'aumento del numero di frame può paradossalmente aumentare il numero di page faults.

Algoritmo LRU (Least Recently Used)

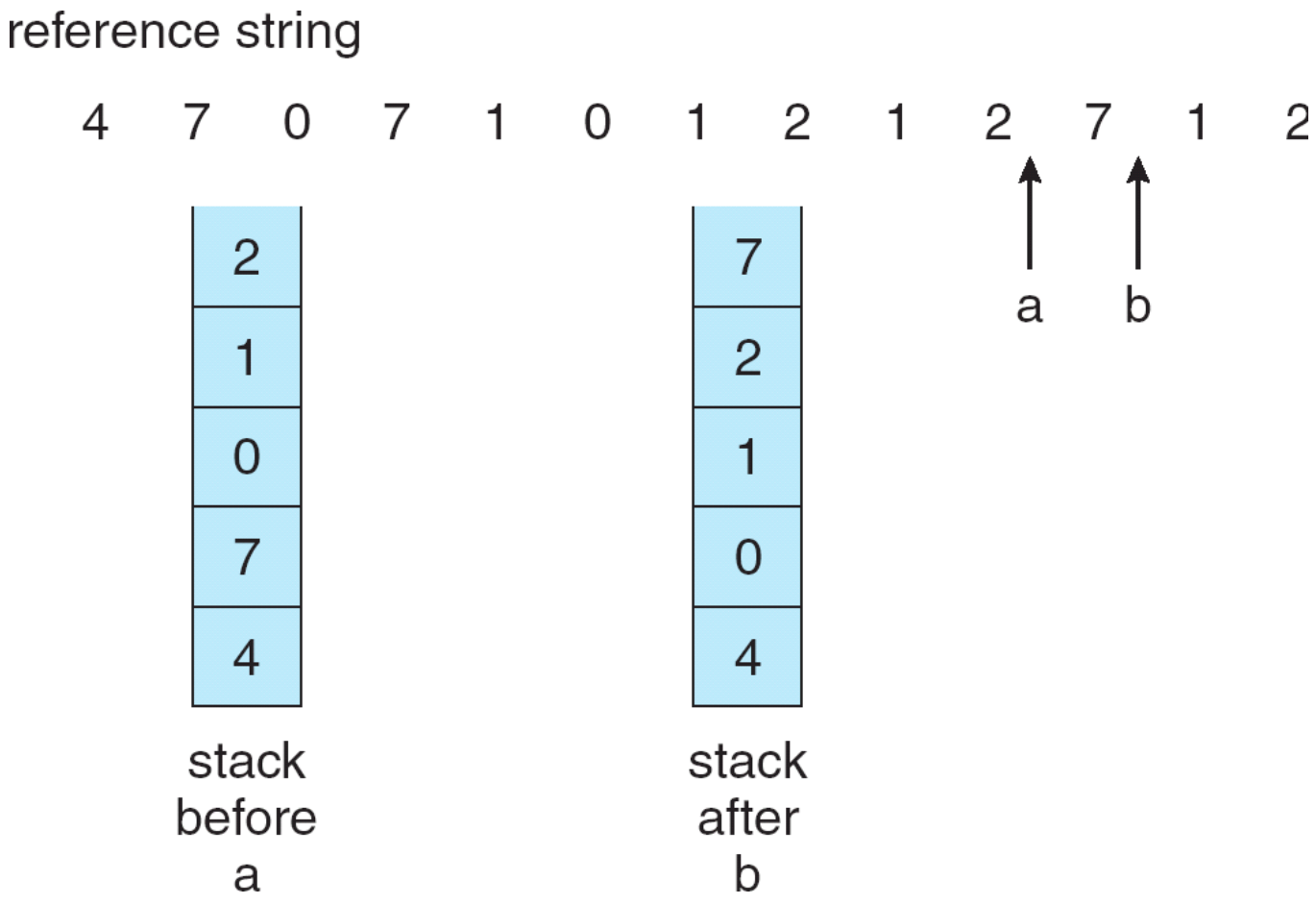
L'algoritmo LRU sostituisce la pagina meno recentemente utilizzata, ipotizzando che la pagina meno usata di recente ha meno probabilità di essere referenziata in futuro.

Caratteristiche dell'Algoritmo LRU

- **Efficienza:** Tende a essere più efficiente di FIFO in molti scenari.
- **Implementazione:** Può essere implementato con un timestamp per ogni pagina, registrando l'ultimo accesso, oppure con una struttura a pila, dove ogni pagina referenziata viene spostata in cima alla pila.
- **No Anomalia di Belady:** Non soffre dell'anomalia di Belady, dove un aumento del numero di frame può aumentare i page faults.

L'algoritmo LRU sostituisce la pagina che non è stata utilizzata per il periodo di tempo più lungo. Utilizza una pila (stack) per tracciare l'ordine di utilizzo delle pagine.

Funzionamento dello Stack



- **Prima di A:** Lo stack contiene le pagine [2, 1, 0, 7, 4].
- **Dopo B:** Lo stack viene aggiornato e contiene le pagine [7, 2, 1, 0, 4]. Ogni volta che una pagina viene utilizzata, viene spostata in cima alla pila.

Vantaggi

- Maggiore accuratezza nella previsione delle pagine da sostituire.
- Migliori prestazioni in scenari pratici.
- In media si comporta meglio rispetto al FIFO.

Svantaggi

- L'organizzazione dell'LRU è tale da richiedere un supporto hardware sofisticato e dedicato alle operazioni relative all'aggiornamento dello stack.
- L'aggiornamento dello stack va effettuato ad ogni riferimento alla pagina (praticamente ad ogni accesso in memoria) l'organizzazione dell'LRU

Algoritmi di Sostituzione delle Pagine: Approssimazioni a LRU

Utilizzo del Contatore

- **Numero di Accessi:** Utilizza un contatore per numerare il numero di accessi alle pagine. Ogni volta che una pagina viene referenziata, il suo contatore viene incrementato.

Algoritmo LFU (Least Frequently Used)

- **Descrizione:** Sostituisce le pagine con il valore di contatore più basso.
- **Filosofia:** Le pagine utilizzate frequentemente avranno valori di contatore elevati.
- **Problema:** Alcune pagine potrebbero essere frequentemente referenziate all'avvio di un processo e poi non più necessarie.
- **Soluzione:** Azzerare periodicamente i contatori per evitare che pagine non più necessarie continuino ad avere valori alti.

Algoritmo MFU (Most Frequently Used)

- **Descrizione:** Sostituisce le pagine con il valore di contatore più alto.
- **Filosofia:** La pagina con un valore basso di contatore è stata appena portata in memoria e sarà utilizzata in futuro.

Bit di Riferimento (REFERECE BIT)

- **Funzionamento:** Ogni pagina nella tabella delle pagine ha un bit associato, inizialmente impostato a 0. Quando una pagina viene referenziata (letta o scritta), il bit viene impostato a 1.
- **Sostituzione:** Quando è necessario sostituire una pagina, si scelgono quelle con il bit impostato a 0.
- **Nota Bene:** Non tiene traccia dell'ordine di utilizzo e i bit possono essere azzerati a intervalli regolari.

Seconda Chance (basato su FIFO)

- **Funzionamento:** Utilizza il bit di riferimento insieme alla sostituzione delle pagine in ordine FIFO (First In, First Out).
- **Reference Bit:** Ogni pagina ha un bit di riferimento associato, inizialmente impostato a 0. Quando una pagina viene referenziata, il bit viene impostato a 1.
- Le pagine sono organizzate come un anello circolare, e un puntatore (noto come clock hand o "lancetta dell'orologio") si muove in senso orario attraverso le pagine.
  Procedura di Sostituzione:

118. **Posizione Iniziale:** La clock hand punta alla pagina corrente da esaminare.
119. **Controllo del Reference Bit:**- **Reference Bit = 0:** Se il bit di riferimento della pagina è 0, la pagina viene sostituita.

```
- **Reference Bit = 1:** Se il bit di riferimento della pagina è 1, la
pagina riceve una "seconda chance". Il bit viene impostato a 0 e la clock
hand passa alla pagina successiva.
```

120. **Avanzamento della Clock Hand:** La clock hand continua a muoversi in senso orario fino a trovare una pagina con il bit di riferimento impostato a 0, pronta per essere sostituita.

Seconda Chance Migliorato

Utilizzo di Reference e Dirty Bit

Questo algoritmo utilizza due bit per ogni pagina:
121. **Reference Bit (bit di riferimento)**: Indica se la pagina è stata referenziata (usata) recentemente.
122. **Dirty Bit (bit di modifica)**: Indica se la pagina è stata modificata (scritta).

Criteri di Sostituzione:
123. **(0,0) – Pagina non referenziata né modificata:**
- **Descrizione:** Questa pagina non è stata recentemente utilizzata né modificata.
- **Azione:** È la migliore candidata per la sostituzione poiché non richiede operazioni aggiuntive.

124. **(0,1) – Pagina non referenziata ma modificata:**

```
- **Descrizione:** La pagina non è stata recentemente utilizzata ma è stata
modificata.
- **Azione:** Prima di sostituire questa pagina, deve essere salvata
(scritta sul disco) per non perdere i dati modificati.
```

125. **(1,0) – Pagina utilizzata recentemente ma non modificata:**

```
- **Descrizione:** La pagina è stata recentemente utilizzata ma non è stata
modificata.
- **Azione:** Questa pagina potrebbe essere nuovamente referenziata a breve,
quindi è meno preferibile sostituirla rispetto alle pagine con bit di
riferimento 0.
```

126. **(1,1) – Pagina utilizzata recentemente e modificata:**

```
- **Descrizione:** La pagina è stata recentemente utilizzata e modificata.
- **Azione:** Anche questa pagina è meno preferibile da sostituire poiché è probabile che venga nuovamente referenziata e deve essere salvata prima della sostituzione.
```

Implementazione: MACH OS

- **Sistema Operativo:** Il sistema operativo MACH OS utilizza questo algoritmo di sostituzione delle pagine per ottimizzare la gestione della memoria.
- **Benefici:** Utilizzando sia il bit di riferimento che il dirty bit, MACH OS può fare decisioni di sostituzione delle pagine più efficienti, migliorando le prestazioni complessive del sistema. **Vantaggi**:
- **Efficienza:** Minimizza il numero di scritture su disco e riduce i page fault.
- **Semplicità:** Facile da implementare rispetto ad altre varianti più complesse.

FILE SYSTEM

Necessità dell'utilizzo del file system:

- **Memorizzazione e accesso**: Le applicazioni su un computer hanno bisogno di memorizzare e rintracciare informazioni.
- **Limitazioni dello spazio degli indirizzi**: Le informazioni memorizzate nello spazio degli indirizzi di un processo vengono perse al termine dell'esecuzione del processo.
- **Memoria permanente**: Può essere necessario che le informazioni siano memorizzate per lungo tempo.
- **Accesso condiviso**: Più processi possono aver bisogno delle stesse informazioni contemporaneamente, quindi queste informazioni devono essere indipendenti dal processo che utilizza.

Soluzione: Registrazione delle in formazioni su dischi.

- **Grandi quantità di informazioni**: Per memorizzare grandi quantità di informazioni e renderle permanenti, è necessario utilizzare dischi o altri supporti.
- **File**: Le informazioni vengono registrate in unità chiamate file, che possono essere accessibili da più processi.

I FILE

File system: questo modulo del sistema operativo non gestisce solo il contenuto del file, ma anche informazioni complementari, come:

- **strutture**: la disposizione del file.
- **tipo di accesso**: la modalità con cui è possibile accedere al file (lettura, scrittura, esecuzione).
- **nome**: il nome del file, che di solito è suddiviso in due parti.
- **protezione**: le autorizzazioni di accesso e sicurezza applicate al file.

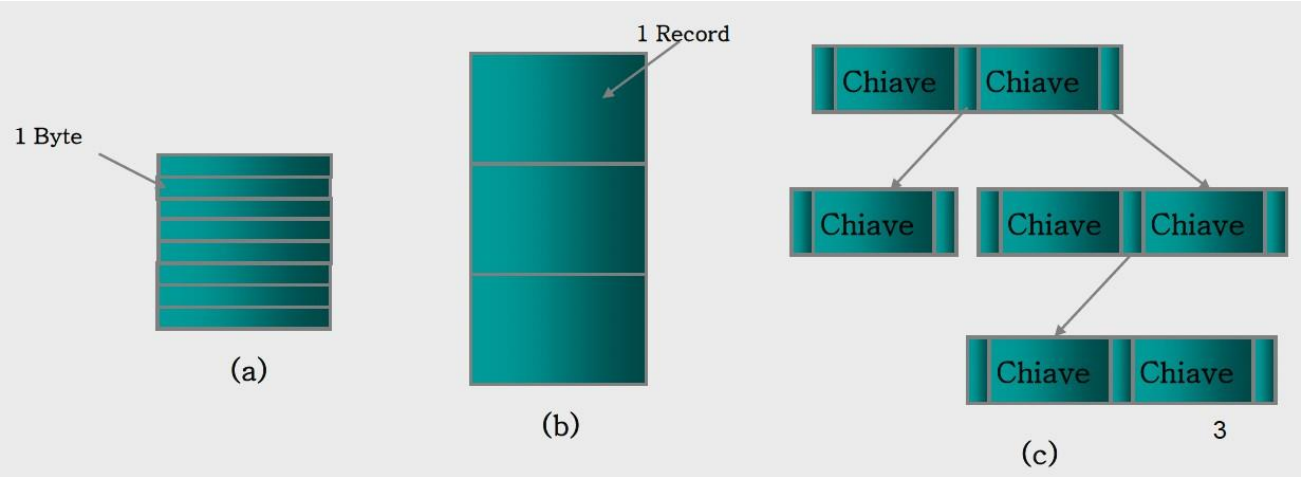
NOME DEL FILE

- **suddivisione**: il nome del file è generalmente suddiviso in due parti.
  - **nome**: parte principale del nome del file.
  - **estensione**: un'ulteriore parte del nome che specifica il tipo di file (es. .txt .jpg .c .exe .dmg ecc).
- **variabilità**: entrambe le parti possono avere un numero variabile di caratteri e possono utilizzare maiuscole o minuscole.

STRUTTURA DEI FILE

I file contengono l informazioni in tre tipi di strutture:

- **Sequenza non strutturata di byte**: consente la massima flessibilità al proprio utente. Ogni file è visto come una semplice sequenza di byte. Unix utilizza questa struttura, consente l'accesso diretto ai singoli byte. (immagine a).
- **Sequenza di record a lunghezza fissa**: In questa struttura, un file è organizzata come una sequenza di record a lunghezza fissa. Questa struttura ha perso di popolarità nel tempo. (immagine b).
- **Albero di record con campo chiave**: i file sono organizzati come un albero di record con campi chiave, dove ogni nodo rappresenta un record. Questa struttura è ancora in uso nei mainframe per l'elaborazione dei dati commerciali. (immagine c).



TIPI E FORMATI DEI FILE

TIPI DI FILE:

- **file regolari**: Contengono informazioni dell'utente, programmi eseguibili, ecc. Utilizzati per memorizzare dati di vario tipo come documenti, immagini, audio, video e programmi.
- **directory**: preservano la struttura del file system e contengono informazioni su file regolari. Utilizzate per organizzare e gestire i file sul disco, permettendo la creazione di una struttura gerarchica di file e cartelle.
- **file speciali e caratteri**: utilizzati per modellare unità di I/O seriali come video, stampanti ecc. Consentono la comunicazione con dispositivi che trasmettono dati un byte alla volta.
- **file speciali a blocchi**: utilizzati per modellare unità I/O a blocchi come dischi. Consentono l'accesso a dispositivi di memorizzazione che leggono e scrivono dati in blocchi.

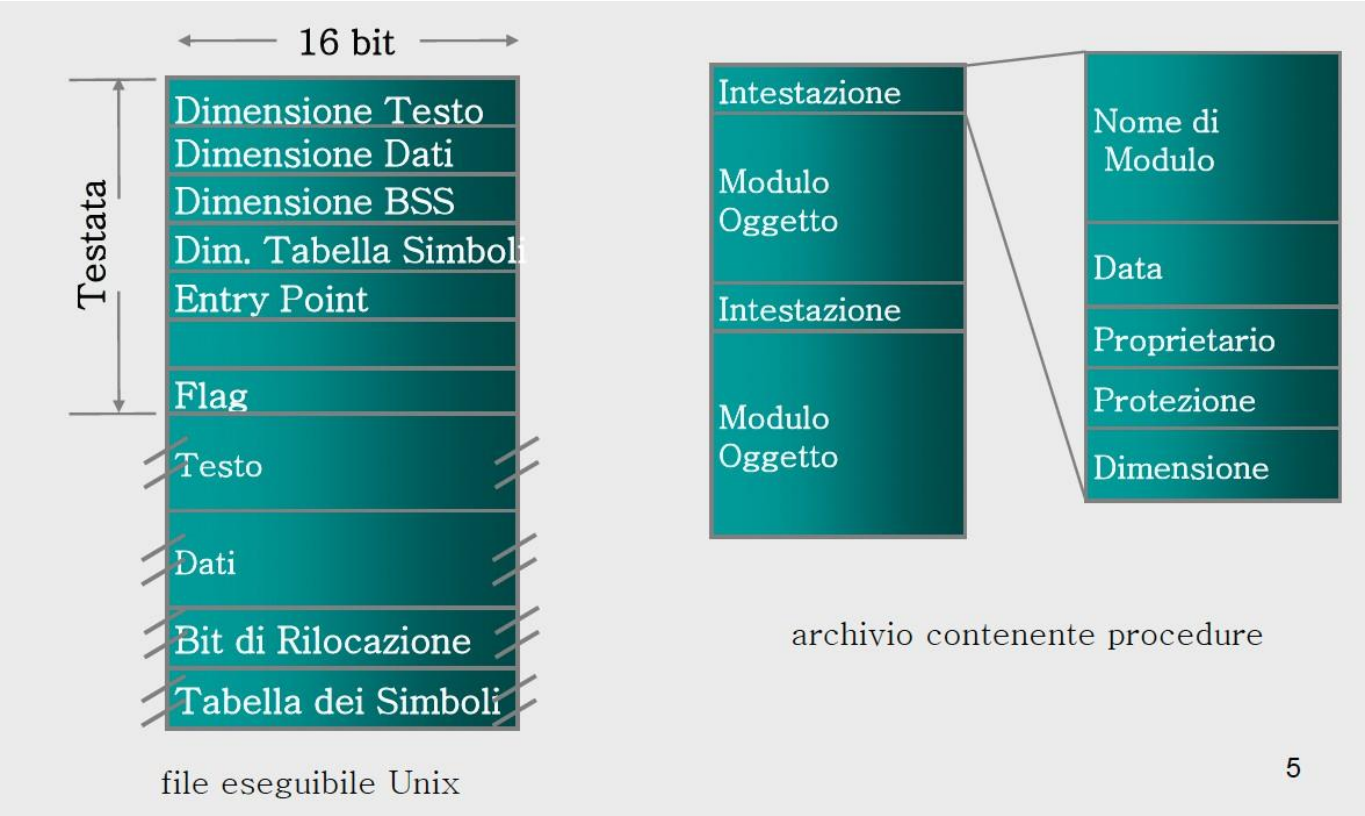
FORMATI DELLE INFORMAZIONI NEI FILE

CODICE ASCII E UNICODE

- **Codice ASCII**: utilizza 8 bit, con un bit di controllo: 7bit=128 configurazioni. I file di programmi sorgente e documenti sono generalmente registrati in questo formato.
- **UNICODE**: codice a 16 bit (codifica i caratteri usati in tutte le lingue vive e alcune lingue morte, nonchè simboli matematici e chimici, simboli cartografi, alfabeto Braile ideogrammi, ecc.).

BINARIO

- Le sequenze di codici sono "leggibili". I file memorizzati in questo formato di solito hanno una struttura interna. Utilizzati per memorizzare dati binari che non devono essere letti direttamente dagli utenti, come eseguibili e file multimediali.



I FILE ACCESSO

Tipi di accesso ai file:

- **Accesso sequenziale**: in questo metodo, i byte o i record che costruiscono i file sono letti uno dopo l'altro, dall'inizio alla fine.
- **Accesso casuale (casuale)**: in questo metodo, l'accesso al blocco di informazioni avviene in maniera diretta. Si può accedere a qualsiasi parte del file senza dover leggere tutto ciò che lo precede. È reso possibile dall'avvento dei dischi, che consentono l'accesso rapido e casuale ai dati. Spesso utilizzate nelle basi di dati e nei sistemi che richiedono un accesso rapido non sequenziale ai dati.

ATTRIBUTI DEI FILE

I sistemi operativi associano ai file altre informazioni come:

- data e ora della creazione
- il proprietario

- dimensione ecc.

Campo	Significato
Protezione	Chi può accedere al file e in che modo
Password	Parola d'ordine necessaria per accedere al file
Flag di sola lettura	0 per lettura/scrittura, 1 per sola lettura
Flag di sistema	0 per file normale, 1 per file di sistema
Flag ASCII/binario	0 per file ASCII, 1 per file binario
Lunghezza record	Numero di byte in un record
Tempo di creazione	Data e ora del momento in cui è stato creato il file
Dimensione attuale	Numero di byte nel file
Dimensione massim	Dimensione massima che il file può raggiungere

OPERAZIONI DEI FILE

Il sistema operativo mette a disposizione all'utente **comandi di alto livello** e **chiamate di sistema** per la gestione dei file. A>lcune chiamate di sistema più comunI sono:

- **create**: creazione di un file.
- **delete**: cancellazione del file.
- **open**: apertura del file con varie modalità (lettura, scrittura ecc).
- **close**: chiusura del file.
- **read**: lettura di blocchi di dati.
- **write**: scrittura di blocchi di dati.
- **append**: aggiunta di nuovi dati alla fine di un file.
- **seek**: posizionamento di un puntatore all'interno di un file. Permette di spostare il puntatore a una posizione specifica nel file per leggere o scrivere in quella posizione.

FILE MAPPATI IN MEMORIA

Mappatura: un file viene caricato, totalmente o parzialmente, nella memoria centrale (RAM). Questo file viene assegnato a indirizzi virtuali, permettendo al processo di accedere al file direttamente in memoria come se fosse un array in RAM.
Vantaggi:

- **Accesso trasparente**: i processi possono accedere ai file come se fossero tutti memorizzati, senza dover gestire manualmente l'I/O del file sul disco.

- **Velocità**: Le operazioni sui file sono più veloci, specialmente quando si ha a che fare con parti limitate del file, dato che l'accesso in memoria è più rapido rispetto all'accesso su disco.

PROBLEMI

- Se più processi aprono lo stesso file simultaneamente, potrebbero verificarsi inconsistenze nei dati.
- Quando un processo che usa un file mappato in memoria termina, il file viene riscritto su disco. Se il processo si arresta in modo anomalo, potrebbero verificarsi perdite di dati.
- In caso di fallimento del sistema, alcune informazioni non ancora salvate potrebbero andare perse.

LA DIRECTORY

introduzione: Una **directory** è una struttura che contiene e organizza file e altre directory (subdirectory). È simile a una cartella nel sistema operativo, permettendo di mantenere i dati ordinati e facilmente accessibili.
Nel passato i vecchi sistemi operativi utilizzavano un'unica directory in cui confluivano tutti i file. Nei moderni sistemi adottiamo una struttura gerarchica di directory, dove le directory possono contenere sia file regolari che altre directory, creando un albero di directory e subdirectory.

CONTENUTO DI UNA DIRECTORY

Un elemento tipico di una directory può contenere:

- **nome di un file**
- **attributi del file**: informazioni aggiuntive come dimensioni, tipo, permessi di accesso, data di creazione/modifica, ecc.
- **puntatore**: un riferimento a un'altra struttura deovo sono memorizzati gli attributi e gli indirizzi del disco per il file.

STRUTTURA GERARCHICA

- **root directory(directory radice)**: la directory principale da cui si dirama l'intero albero di directory e subdirectory.
- **file regolari**: file che contengono dati utente o programmi.
- **altre directory**: sottodirectory che possono contenere ulteriori file e directory.

METODO DI ORGANIZZAZIONE DELLE DIRECTORY

STRUTTURA AD ALBERO

L'organizzazione dell'albero delle directory varia in base alle decisioni dell'amministratore di un sistema, per esempio:

- **root dirctory**: è la directory principale da cui parte tutto il file system. Contiene varie directory di sistema essenziali come:
  - **etc**: contiene file di configurazione del sistema.
  - **bin**: contiene file eseguibili binari essenziali.
  - **lib**: contiene librerie di sistema condivise.
  - **tmp**: contiene file temporanei.
- **directory usr**: contiene una subdirectory per ogni utente del sistema. Ogni utente può organizzare il proprio sottoalbero nel modo più efficiente e comodo per le proprie attività. Ad esempio, un utente potrebbe avere sottodirectory come "documenti", "immagini", e "musica".

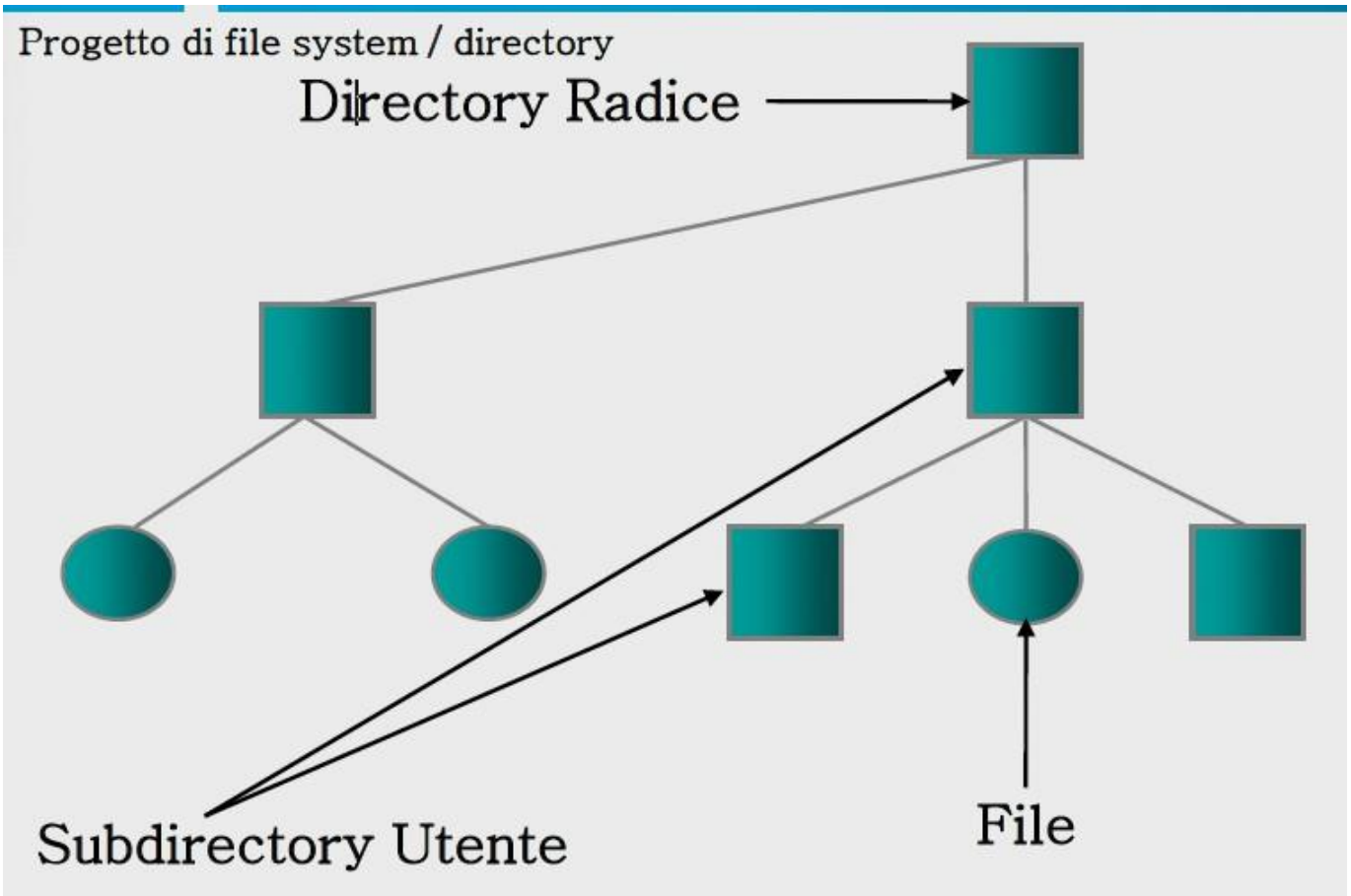
PERCORSI (PATH)

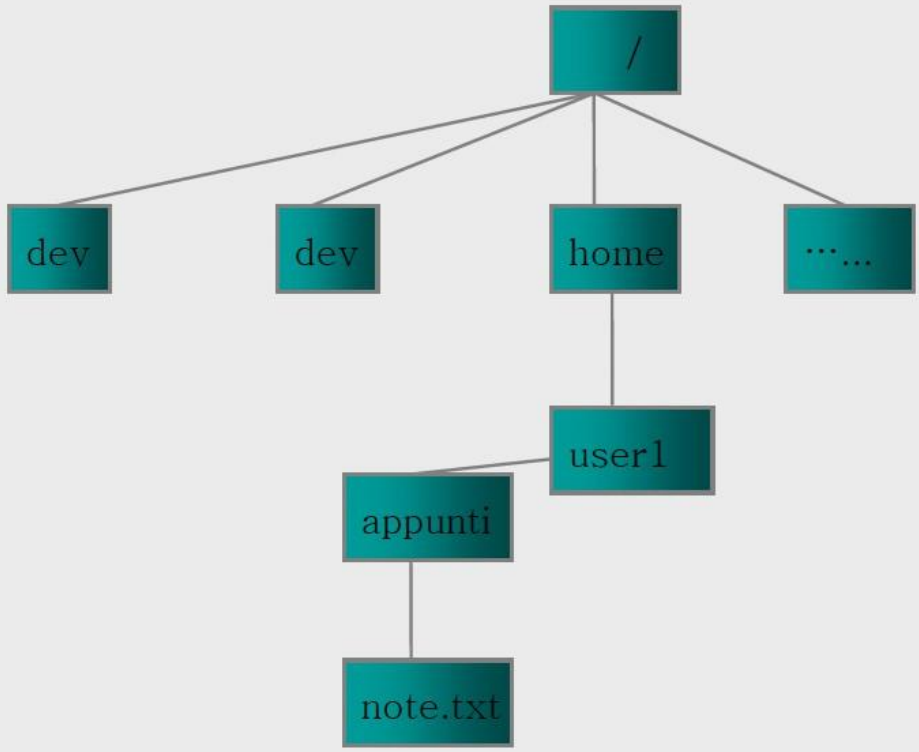
Path name assoluto: è il percorso completo della directory radice al file desiderato. In Unix, i componenti del precorso sono separati dal carattere `/`

- **esempio**: `/home/utente/documenti/file.txt` è un path name assoluto che parte dalla radice `/` e segue l'intera gerarchia fino al file specifico.

Path name relativo: è il percorso relativo alla directory di lavoro corrente dell'utente. Ogni utente può impostare una directory come "directory di lavoro corrente". Tutti i path name che non iniziano con la directory radice sono considerati relativi alla directory di lavoro corrente.

- **esempio**: se la directory di lavoro corrente è `/home/utente`, un percorso relativo per il file poitrebbe essere `documenti/file.txt`, il che significa che il file si trova all'interno della directory "documenti" sotto la directory di lavoro corrente.





- **path name assoluto:** /home/user1/appunti/note.txt
- **path name relativo:** appunti/note.txt

OPERAZIONI DELLE DIRECTORY

Alcune operazioni comuni per la gestione della directory in un sistema operativo di tipo Unix sono:
create: crea una directory vuota.

- comando: mkdir nome, serve a creare una nuova directory vuota con un nome specifico.
  delete: cancella una directory.
- comando: rmdir nome, elimina una directory vuota con nome specificato.
- comando: rm -rf, elimina un a directory non vuota e tutto il suo contenuto in modo ricorsivo.
  spostarsi tra directory
- comando: cd newdir, Cambia la directory corrente a newdir.
- comando: cd, passa alla directory home dell'utente.
- comando: cd ~bill, passa alla home directory dell'utente bill.
  rename
- comando: mv source dest, rinomina la directory o sposta una directory da una posizione a un'altra.
  elencazione dei file(list file)
- comando: ls -l, fornisce un elenco dettagliato dei file, inclusi i permessi, proprietari, dimensioni e date di modifica.

METODI DI ALLOCAZIONE DEI BLOCCHI

Un fattore chiave della memorizzazione dei file è tener traccia di quali blocchi del disco associare a ciascun file.

Le modalità di allocazione sono:

- contigua.
- a lista concatenata.
- a lista concatenata con indice.
- mediate uso di tabelle i-node.

ALLOCAZIONE CONTIGUA

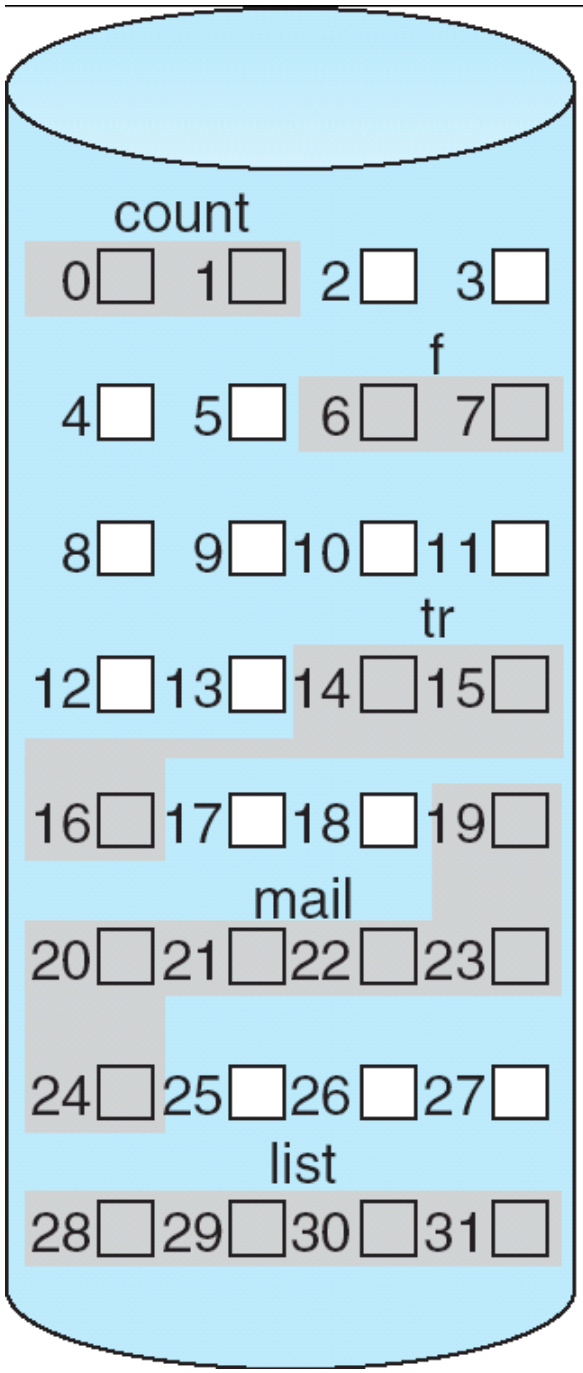
In questo metodo, i file sono memorizzati in blocchi di disco consecutivi. Ogni file occupa un segmento continuo di blocchi. Per ogni file, è necessario tenere traccia dell'indirizzo di inizio del file e della sua lunghezza.

Pregi:

- **Semplice da implementare**: occorre solo conoscere l'indirizzo di un file e la sua lunghezza per gestirlo.
- **Efficienza**: permette di leggere l'intero file con una sola operazione di I/O poichè i blocchi sono contigui.
  Difetti:
- **Dimensione massima del file**: occorre conoscere la dimensione massima del file al momento della sua creazione, il che può essere problematico per file che crescono dinamicamente.
- **Frammentazione esterna**: Il disco può diventare frammentato, con molti piccoli spazi vuoti tra i file, rendendo difficile trovare spazio sufficiente per nuovi file.
- **Difficoltà di reperimento dello spazio ibero**: può essere complesso trovare spazi contigui sufficientemente grandi per nuovi file, specialmente su dischi molto utilizzati.

Metodi di allocazione dello spazio:

- **Best fit**: Cerco lo spazio libero che meglio si adatta alla dimensione del file.
- **First fit**: Usa il primo spazio libero sufficiente trovato.
- **Worst fit**: Cerca lo spazio libero più grande disponibile.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Spiegazione dei Dati

- **File** `count`**:**
  - Inizia al blocco 0 e occupa 2 blocchi.
  - Blocchi occupati: 0 e 1.
- **File** `tr`**:**
  - Inizia al blocco 14 e occupa 3 blocchi.
  - Blocchi occupati: 14, 15, e 16.
- **File** `mail`**:**
  - Inizia al blocco 19 e occupa 6 blocchi.

    - Blocchi occupati: 19, 20, 21, 22, 23, e 24.
- **File `list`:**
    - Inizia al blocco 28 e occupa 4 blocchi.
    - Blocchi occupati: 28, 29, 30, e 31.
- **File `f`:**
    - Inizia al blocco 6 e occupa 2 blocchi.
    - Blocchi occupati: 6 e 7.

ALLOCAZIOE A LISTA CONCATENATA

Nell'allocazione a lista concatenata, ogni blocco di dati di un file contiene due parti.

- **Dati veri e propri**: La perte principale del blocco dove sono memorizzati i dati del file.
- **Numero del blocco successivo**: Un puntatore che indica il blocco di dati successivo nel file.
  Questo crea una catena di blocchi collegati tra loro, formando una lista concatenata.

PREGI

- **Nessuno spreco di spazio**: I blocchi vengono allocati dinamicamente, quindi non c'è bisogno di riservare in anticipo spazio contiguo fra i file.
- **Nessuna frammentazione esterna**: Poichè i blocchi non devono essere contigui, non si verifica frammentazione esterna.
- **Indirizzo del primo e ultimo blocco**: Nella directory, viene memorizzato l'indirizzo del primo e dell'ultimo blocco del file, facilitando l'accesso al file.
  DIFETTII
- **Accesso diretto lento**: L'accesso diretto ai blocchi è estremamente lento perchè è necessario seguire i puntatori attraverso la lista concatenata per raggiungere un blocco specifico.
- **Overhead dei puntatori**: Ogni blocco deve contenere un puntatore al blocco successivo. Questa area del blocco deve essere non accessibile ai programmi utenti per motivi di protezione, riducendo lo spazio disponibile per i dati veri e propri.

ALLOCAZIONE A LISTA CONCATENATA CON INDICE

Descrizione

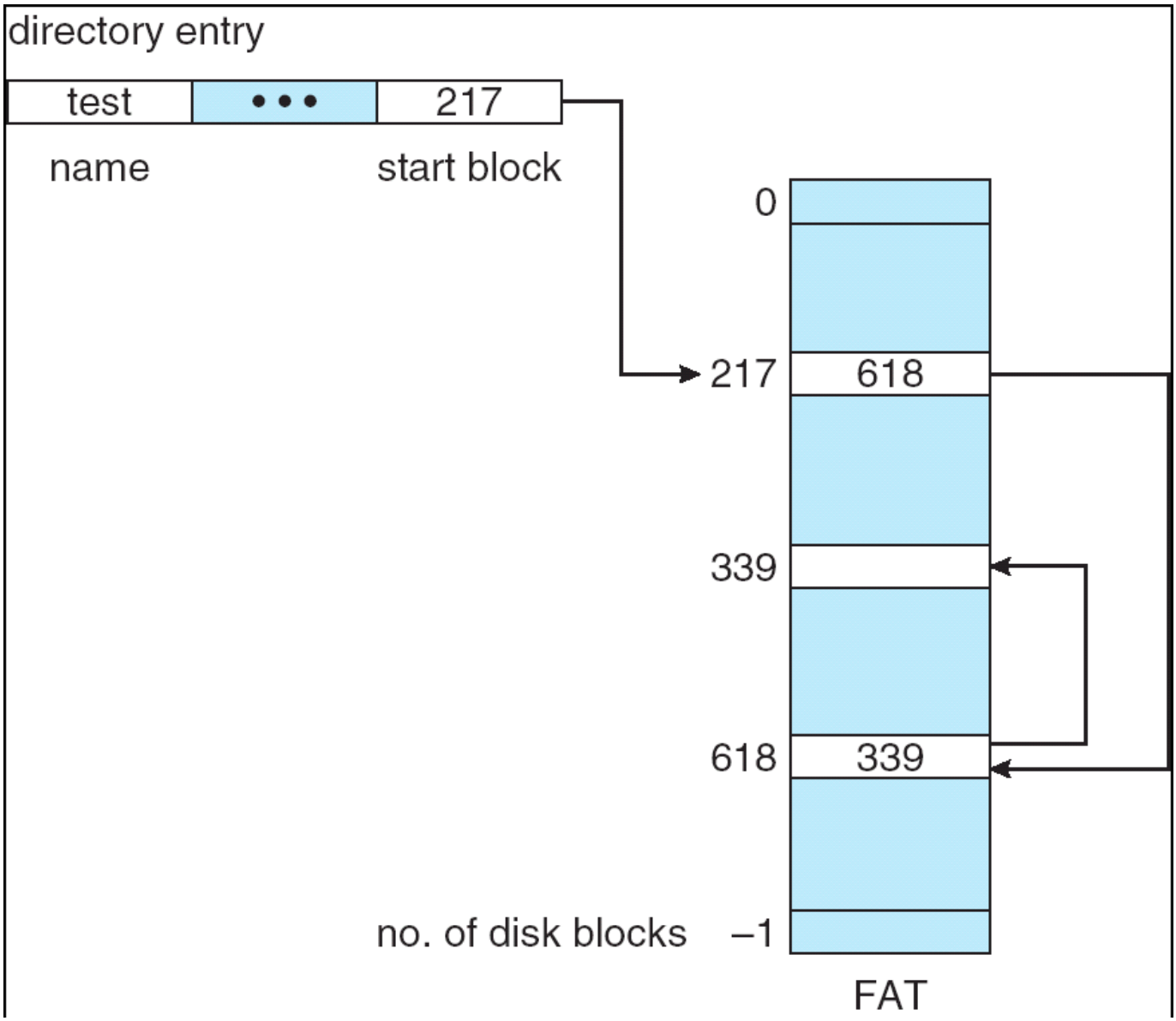
- **Non contigui**: I blocchi di disco assegnati a un file non sono necessariamente contigui.
- **Lista di blocchi**: La lista dei blocchi assegnati a un file è mantenuta attraverso una tabella (sezione sul disco all'inizio di ogni partizione).

- **Indice della tabella**: Ogni elemento della tabella indicizzato dal numero di blocco contiene il numero del blocco successivo del file. L'ultimo blocco punta a NULL.
- **Blocchi vuoti**: I blocchi vuoti sono indicati da NULL.
- **Directory**: L'elemento della directory contiene il numero del primo blocco del file.

Vantaggi:

- **Spazio per dati**: L'intero blocco è disponibile per i dati, senza overhead aggiuntivi per i puntatori.
- **Accesso diretto**: L'accesso diretto a un blocco è rapido grazie alla tabella di allocazione. **Svantaggi**:
- **Dimensione della tabella**: Un numero elevato di blocchi di disco corrisponde a una tabella che occupa molto spazio nella memoria principale.
- **Movimenti della testina**: La lettura della FAT(File Allocation Table) può causare numerosi movimenti della tastiera del disco, rallentando l'accesso ai dati.

Descrizione della tabella.



Directory Entry

- **Nome del File:** `test`
- **Blocco Iniziale:** `217`

File Allocation Table (FAT)

La FAT è una tabella che mappa la sequenza di blocchi di disco utilizzati dal file. Ogni elemento della tabella rappresenta un blocco sul disco e contiene un puntatore al blocco successivo del file. Se il valore è `-1`, indica la fine del file. Vediamo l'allocazione di questo file:

- **Blocco 217:** Punta al blocco `618`.
- **Blocco 618:** Punta al blocco `339`.
- **Blocco 339:** Punta di nuovo al blocco `618`, creando un loop.

- **Fine del File:** Indicata dal valore `-1`, che segnala la fine della catena di blocchi.

Problematiche

- **Riferimenti Ciclici:** La presenza di un ciclo (loop) nella FAT, dove il blocco `339` punta di nuovo al blocco `618`, può causare un ciclo infinito durante la lettura del file. Questo problema può portare a errori nel file system e rendere il file inaccessibile o danneggiato.

ALLOCAZIONE MEDIANTE USO DI TABELLE I-NODE

L'allocazione indicizzata prevede che tutti i puntatori ai blocchi di un file siano raggruppati in un singola posizione, chiamata **blocco indice**. Ogni file ha il proprio blocco indice, e l'elemento di directory contiene l'indirizzo di questo blocco indice. Questo metodo di allocazione è utilizzato nei sistemi Unix.

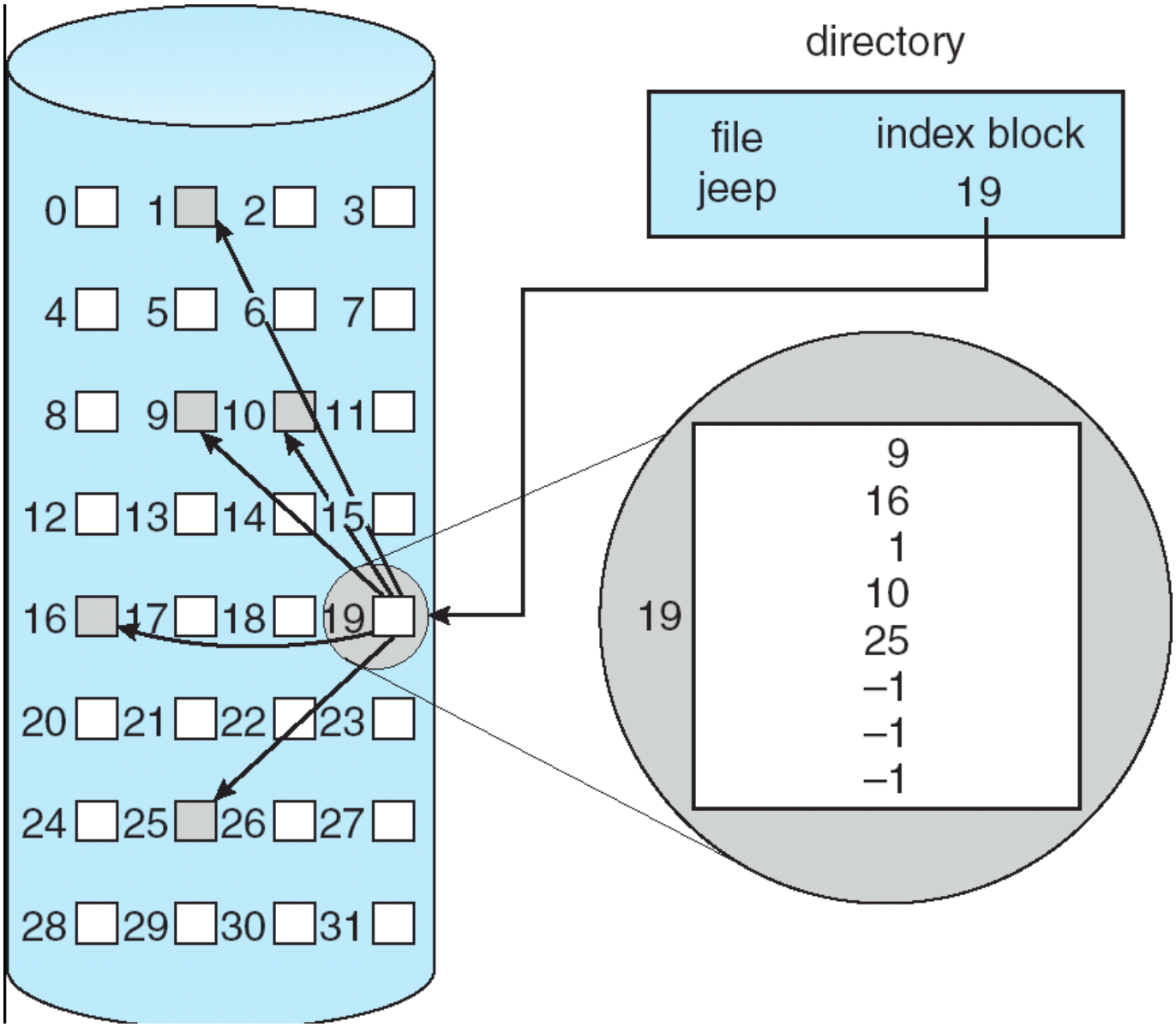
Gli **I-NODE** sono piccole tabelle che contengono:

- **Attributi del file**: Informazioni come permessi di accesso, proprietà, dimensione, e date di modifica e creazione.
- **Indirizzi dei blocchi**: Gli i-node contengono indirizzi dei primi quindici blocchi assegnati al file.

DETTAGLI DEGLI INDIRIZZI NEI BLOCCHI:

- **Blocco indiretto singolo**: Il puntatore indirizza un blocco che contiene ulteriori indirizzi di blocchi di dati.
- **Blocco indiretto doppio**: Il puntatore indirizza un blocco che contiene ulteriori indirizzi di blocchi di dati.
- **Blocco indiretto triplo**: Il puntatore indirizza un blocco che contiene indirizzi di blocchi che, a loro volta, contengono indirizzi di blocchi che contengono indirizzi di blocchi di dati.

ESEMPIO:

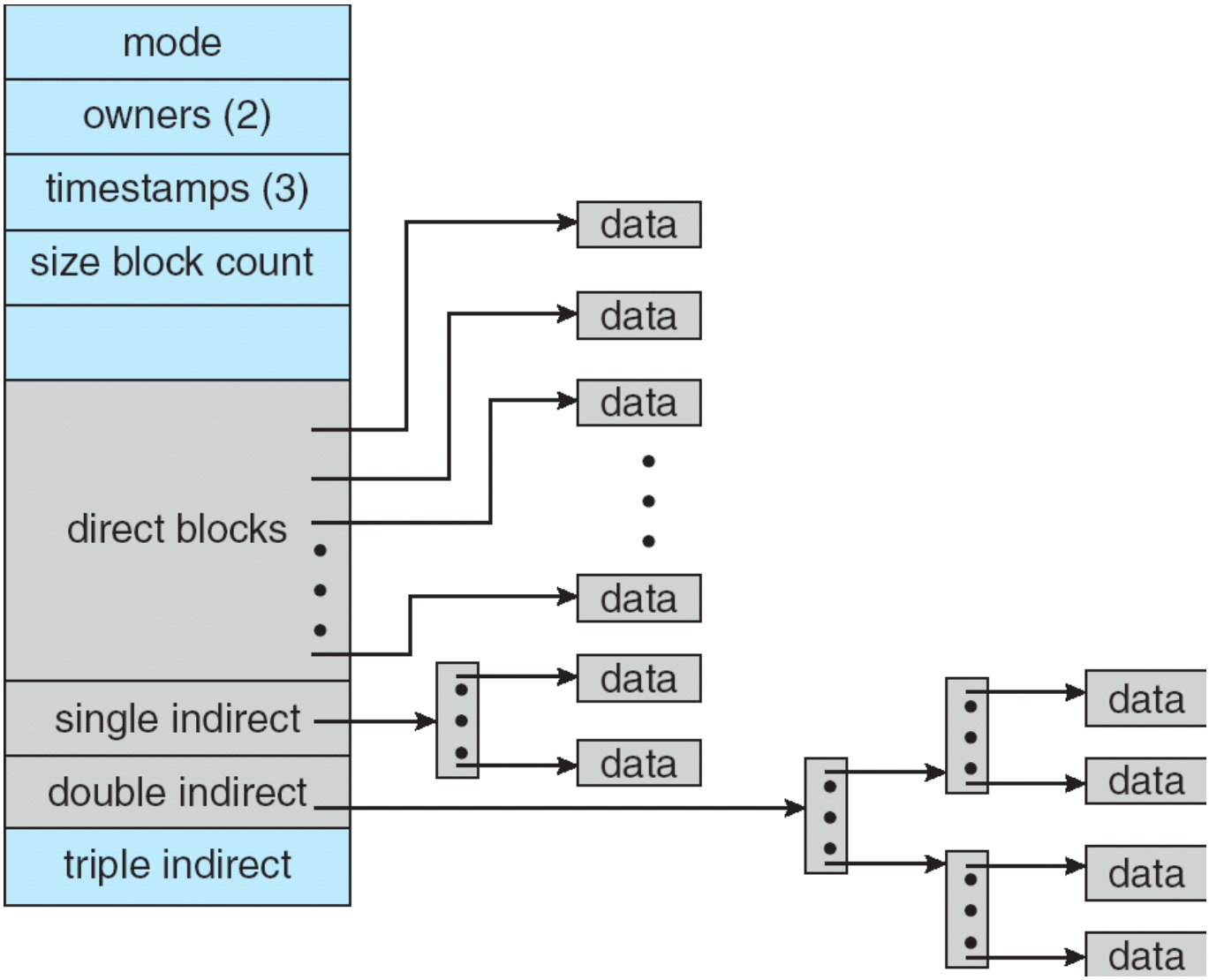


L'immagine mostra la struttura del blocco indice e i puntatori agli altri blocchi:
Rappresenta un disco con blocchi numerati da 0 a 31. Alcuni blocchi sono evidenziati e collegati a un blocco indice, che è il blocco numero 19. Il blocco indice contiene puntatori ai blocchi di dati effettivi del file chiamato "jeep". L'elemento della directory per il file"jeep" punta al blocco indice 19.

In questo sistema di allocazione, tutti i puntatori ai blocchi di un file sono raggruppati in un'unica posizione, chiamata **blocco indice**. L'elemento della directory per un file contiene l'indirizzo del blocco indice. Il blocco indice, a sua volta, contiene i puntatori ai blocchi di dati effettivi del file.

Vantaggi mediate uso di tabelle i-node:

- **Efficienza dello spazio**: L'elemento di directory contiene solo il nome del file e il numero dell'i-node, risparmiando spazio.
- **Gestione dinamica**: Permette di mantenere gli i-node dei file aperti in memoria, utilizzando poco spazio.
- **Flessibilità**: Consente di indirizzare un numero enorme di blocchi di disco, gestendo efficacemente file di grandi dimensioni.
- **Accesso rapido**: Per file di piccole dimensioni, consente di trovare rapidamente l'indirizzo dei primi blocchi di disco.



Struttura dell'i-node

Un **i-node** (index node) è una struttura dati che contiene tutte le informazioni su un file, ad eccezione del suo nome e del contenuto reale. Gli i-node includono i seguenti campi:

- **Mode:** Specifica il tipo di file e i permessi di accesso (lettura, scrittura, esecuzione).
- **Owners:** Informazioni sui proprietari del file.
- **Timestamps:** Date e orari di creazione, modifica e accesso del file.
- **Size:** Dimensione del file.

- **Block Count:** Numero di blocchi di disco utilizzati dal file.
- **Direct Blocks:** Puntatori ai blocchi di dati diretti.
- **Single Indirect:** Puntatore a un blocco che contiene ulteriori puntatori ai blocchi di dati.
- **Double Indirect:** Puntatore a un blocco che contiene puntatori a blocchi di puntatori ai blocchi di dati.
- **Triple Indirect:** Puntatore a un blocco che contiene puntatori a blocchi di puntatori a blocchi di puntatori ai blocchi di dati.

GESTIONE DELLO SPAZIO SU DISCO: DIMENSIONE DEI BLOCCHI

BLOCCO MOLTO GRANDE:
Vantaggi:

- **Velocità di lettura**: Un blocco più grande riduce il numero di operazioni di lettura (seek, rotazione, trasferimento), rendendo la lettura dei dati più veloce.
  Svantaggi
- **Spreco di spazio (frammentazione interna)**: I blocchi più grandi possono lasciare spazi inutilizzati all'interno del blocco, causando spreco di spazio.

BLOCCO MOLTO PICCOLO:
Vantaggi

- **Gestione efficiente dello spazio**: Blocchi più piccoli riducono lo spreco di spazio, permettendo di utilizzare lo spazio su disco in modo più efficiente.
  Svantaggi:
- **Numerosi accessi ai blocchi**: La lettura di dati distribuiti su molti blocchi piccoli può causare numerosi accessi ai blocchi, rallentando le prestazioni.

CONFLITTO INTRINSECO
L'efficienza nel tempo (velocità di lettura)e l'efficienza nello spazio (uso ottimale dello spazio su disco) sono in conflitto. Un compromesso è necessario per bilanciare questi due aspetti.

Compromesso:

- **La dimensione del blocco di allocazione** può variare, generalmente da 512 byte a 8 KB. La dimensione del blocco di allocazione deve essere scelta in modo tale da bilanciare la velocità di trasferimento dei dati e l'efficienza dello spazio su disco.
  Settore del disco
- **Settore**:La minima unità fisica leggibile dal disco. La dimensione del settore sul disco è generalmente di 512 byte. La lettura di un blocco di allocazione 1 KB causerà sempre la

lettura di due settori di disco.

GESTIONE DELLO SPAZIO LIBERO SU DISCO

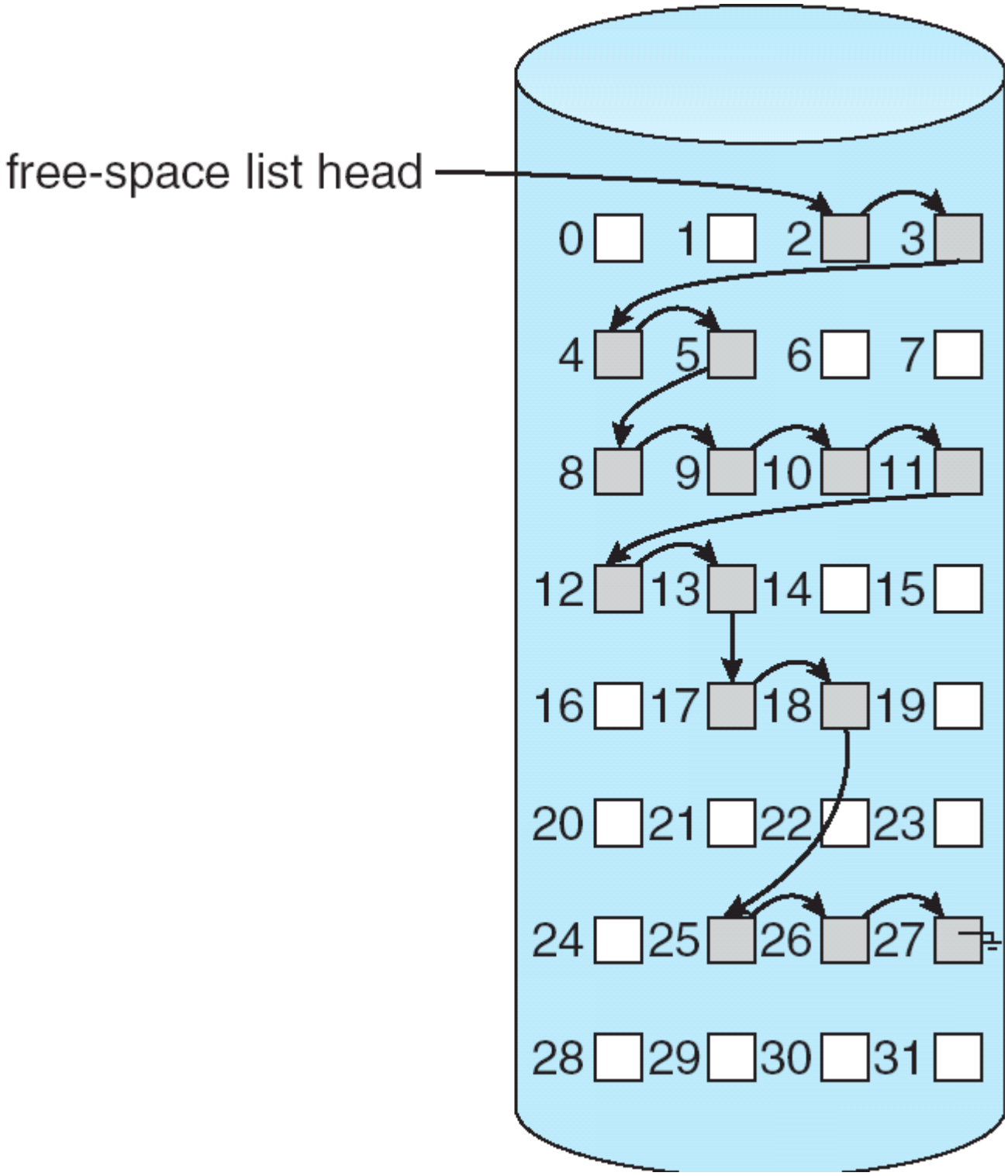
I metodi usati comunemente sono due:
Lista concatenata:

- Tutti i blocchi sono collegati tra loro tramite puntatori: formando una lista concatenata. C'è un puntatore che indica il primo blocco libro della lista.
  - **Protezione del puntatore**: È importante proteggere il puntatore al primo blocco libro per evitare perdite o corruzione dei dati.
- **Efficienza**: Questo metodo è più efficiente quando il disco è quasi pieno, poichè il numero di blocchi liberi è limitato. In tali condizioni, la bitmap potrebbe risultare piena e richiedere accessi continui ad altre parti della mappa per allocazione di blocchi.

Bit Map:

- La bitmap utilizza una mappa di n bit per un disco con n blocchi. I blocchi liberi sono rappresentati da 1 nella mappa, mentre allocati sono rappresentati da 0 (o viceversa).
- Esempio di bitmap per 32 blocchi:
  00111100111111000110000001110000
- **Preferibilità**: Se c'è abbastanza memoria principale per contenere la bitmap, questo metodo è generalmente preferibile. La bitmap è più semplice da gestire e permette una rapida verifica dello stato dei blocchi.

Metodo della lista concatenata con una serie di blocchi liberi.



Funzionamento della Lista Concatenata

127. **Puntatore al Primo Blocco:** Un puntatore (head) indica il primo blocco libero della lista. Nel diagramma, questo è il blocco `2`.
128. **Collegamento dei Blocchi:** Ogni blocco libero contiene un puntatore al blocco libero successivo, creando una lista concatenata.

129. **Fine della Lista:** L'ultimo blocco libero della lista non punta a nessun altro blocco, indicando la fine della catena.

Importanza della Gestione dello Spazio Libero

La gestione efficiente dello spazio libero è cruciale per:

- **Allocazione:** Permettere al file system di trovare rapidamente blocchi liberi per allocare nuovi dati.
- **Deallocazione:** Restituire blocchi non più utilizzati alla lista di spazio libero, mantenendo l'integrità e la disponibilità dello spazio di memorizzazione.

AFFIDABILITÀ DEL FILE SYSTEM

Importanza del file system:

- **Perdita dei dati**: Il file system può contenere dati critici che non devono essere persi perchè recuperarli può essere costoso.
- **Priorità all'affidabilità**: Spesso, l'affidabilità del file system viene perseguita anche a scapito dell'efficienza o dei costi. Questo significa che si preferisce avere un sistema più sicuro e stabile piuttosto che uno veloce ma meno affidabile.

Problemi nei dispositivi di memorizzazione:

- **Blocchi inutilizzati**: I dispositivi di memorizzazione, come i dischi rigidi, possono presentare blocchi di memoria che non possono essere utilizzati a causa di vari motivi:
  - **Difetti di fabbricazione**: Alcuni blocchi possono essere difettosi fin dall'inizio.
  - **Usura**: L'uso prolungato può causare il degrado di alcuni blocchi.
  - **Urti violenti**: Impatti fisici possono danneggiare i blocchi del disco.

GESTIONE DEI BLOCCHI INUTILIZZATI:

- **Non assegnazione**: I blocchi inutilizzati non devono essere assegnati a nessun file. Assegnare tali blocchi può portare a corruzione dei dati e problemi di stabilità del file system.
- **Gestione appropriata**: È necessario gestire correttamente questi blocchi per mantenere l'integrità del file system. Spesso, questo implica la mappatura dei blocchi inutilizzati e la loro esclusione dai processi di allocazione del file system.

GESTIONE DEI BLOCCHI DANNEGGIATI:

- **Soluzioni hardware**:

    - **Blocchi di riserva**: Una parte del disco è riservata per tenere traccia dei blocchi danneggiati e contiene blocchi di riserva.
    - **Sostituzione dei blocchi**: Durante l'inizializzazione, il controller del dispositivo legge i blocchi danneggiati e li sostituisce con i blocchi di riserva, registrando la sostituzione.
    - **Trasparenza**: Questa soluzione è trasparente per il file system, ovvero il file system non ha bisogno di sapere quali blocchi sono stati sostituiti.
- **Soluzioni software**:
    - **Monitoraggio dei blocchi danneggiati**: Il file system tiene traccia dei blocchi danneggiati assegnandoli a un file speciale che non viene utilizzato.
    - **Esclusione dei blocchi danneggiati**: In questo modo, i blocchi danneggiati non vengono mai allocati ai file normali, prevenendo la corruzione dei dati.

BACKUP

Nella pianificazione delle operazioni di backup vanno valutati principalmente:

- La frequenza con cui effettuare le copie di backup.
- I file che presentano la maggior necessità di essere copiati.
- La tecnica migliore per effettuare il backup.

FREQUENZA

La frequenza dei backup è essenziale per minimizzare la perdita di dati e deve essere pianificata in base alla quantità di lavoro svolto nel tempo e al costo del ripristino dei dati.
Ecco le frequenze tipiche:

- **Giornaliero**: Backup eseguiti quotidianamente.
- **Settimanale**: Backup eseguiti una volta alla settimana.
- **Mensile**: Backup eseguiti una volta al mese.
- **Annuale**: Backup eseguiti una volta l'anno, solitamente per archivi storici o dati che devono essere conservati a lungo termine.

SELEZIONE DEI FILE DA COPIARE

La scelta dei file da includere ne backup deve essere pianificata attentamente, tenendo conto del contenuto dei file e del costo del ripristino dei dati.

- **File critici**: I file cirtici sono quelli essenziali per operazioni aziendali e devono essere protetti con maggiore frequenza.
  I file critici potrebbero essere raggruppati in directory dedicate che vengono copiate più frequentemente rispetto agli altri file.

Questo garantisce che i dati più importanti siano sempre aggiornati e disponibili in caso di necessità.

- **Duplicazione dei file critici**: La duplicazione di alcuni file critici può essere appropriata se il costo dello spazio di memoria sprecato è giustificabile.
  La duplicazione aumenta la sicurezza dei dati, riducendo il rischio di perdita in caso di guasto di uno dei supporti di memorizzazione.
  Anche se può comportare un maggiore utilizzo di spazio di memoria, il beneficio di avere una copia di sicurezza aggiuntiva può essere significativo per dati particolarmente sensibili.

TECNICA DI BACKUP

La scelta va pianificata in base al tempo disponibile ed al costo sopportabile per i supporti. Le tecniche di backup possono essere:

- **Backup completo su nastri dell'intero file system**: Consiste nel fare una copia completa di tutti i file system su nastri magnetici.
  Vantaggi: Garantisce un backup completo, permettendo il ripristino di tutto il file system in caso di perdita totale dei dati.
  Svantaggi:Richiede molto tempo e spazio di archiviazione, rendendolo meno pratico per backup frequenti.
- **Backup incrementale**: Questo tipo di backup copia solo i file nuovi o modificati dall'ultimo backup, sia esso completo o incrementale.
  Vantaggi: Risparmia tempo e spazio di archiviazione, poichè vengono copiati solo i dati modificati.
  Svantaggi: Il processo di ripristino può essere più complesso, in quanto potrebbe richiedere l'accesso a più set di backup incrementali.
- **Duplicazione incrociata dei dati su due dispositivi**: Ogni disco contiene metà dei propri dati e l'altra metà dei dati dell'altro disco. Questo metodo prevede la duplicazione incrociata dei dati su due dispositivi diversi.
  Vantaggi: Aumenta la ridondanza e la sicurezza dei dati, garantendo che i dati possano essere recuperati anche se uno dei dischi fallisce.
  Svantaggi: Richiede una maggiore quantità di spazio di archiviazione e una gestione attenta per mantenere i dati sincronizzati tra i due dispositivi.

PRESTAZIONI

Le tecniche per migliorare le prestazioni del file system consistono nel ridurre il numero di accessi al disco, poichè l'accesso ai dischi è un'operazione relativamente lenta. I metodi sono:

- **Caching**: È una tecnica che memorizza temporaneamente i dai frequentemente acceduti in una memoria più veloce rispetto al disco, come RAM. Quando i dati vengono richiesti, il

sistema controlla prima la chache $ e, se i dati sono presenti, li recupera rapidamente senza dover accedere al disco.
Vantaggi: Riduce il tempo di accesso ai dati. Diminuisce il numero di operazioni di lettura/scrittura sul disco, prolunga la vita del disco.
Svantaggi: in caso di crash del sistema, i dati presenti nella cache potrebbero andare persi se sono stati scritti sul disco. La gestione della cache può aggiungere complessità e overhead al sistema.

- **Riduzione dei movimenti del braccio del disco (Soluzione soft)**: Questa tecnica prevede l'ottimizzazione dell'ordine delle operazioni di lettura/scrittura per minimizzare i movimenti del braccio del disco. Un esempio comune è l'algoritmo dell'ascensore, che organizza le operazioni in modo tale che il braccio del disco si muova in modo sequenziale lungo le tracce, riducendo i tempi di seek.
  Vantaggi: Migliora l'efficienza e riduce il tempo di accesso ai dati. Ottimizza l'utilizzo del disco, riducendo l'usura meccanica.
  Svantaggi: Gli algoritmi di ottimizzazione possono essere complessi da implementare e mantenere. Potrebbe non essere sempre possibile ottimizzare tutti i movimenti a causa di richieste di accesso non prevedibili.
- **Riduzione dei movimenti del braccio disco (Soluzione hard)**: Questa tecnica comporta modifiche hardware che riducono fisicamente i tempi di movimenti del braccio del disco. Può includere l'uso di dischi con bracci più veloci o tecnologie come dischi a stato solido (SSD) che non hanno parti mobili.
  Vantaggi: Drastica riduzione dei tempi di accesso ai dati. Aumento significativo della velocità di lettura/scrittura. Maggiore affidabilità e minore usura rispetto ai dischi tradizionali.
  Svantaggi: L'adozione di hardware più avanzato o SSD può essere costosa. Potrebbe richiedere aggiornamenti significativi al sistema per supportare le nuove tecnologie.

TECNICHE MA NEL DETTAGLIO

- **Block cache**: Un certo numero di blocchi di memoria viene riservato per contenere blocchi di dati dal disco.
  - Ogni volta che viene richiesto un blocco, il sistema prima controlla nella tabella dei blocchi di memoria (cache) per vedere se il blocco è già presente.
  - Se il blocco non è presente in cache, viene caricato dal disco, copiato nella cache, e poi reso disponibile per le richieste successive.
  - La lista dei blocchi in cache può essere gestita con algoritmi di rimpiazzamento delle pagine come FIFO (First in First Out) e LRU (Least Recently Used).
  - **Comportamento nei sistemi operativi**:
    - **UNIX**: Riscrive tutti i blocchi modificati sul disco ogni 30 secondi o su richiesta dell'utente.
    - **WINDOWS**: Riscrive ogni blocco modificato immediatamente.

- **Riduzione dei movimenti del braccio del disco(soluzione soft)**:
  - **Raggruppare i Blocchi in Uso per Accessi Sequenziali**:
    Questa tecnica prevede di raggruppare i bocchi di dati in modo che possano essere letti in modo sequenziale. Questo riduce la necessità di movimenti del braccio del disco per accedere ai dati non contigui.
    Vantaggi: Diminuisce il tempo necessario per accedere ai dati e migliora le prestazioni generali del sistema riducendo i tempi di seek (movimento del braccio sul disco).
  - **Posizionamento dell'indice dei blocchi**:
    Quando si legge un file, il sistema deve accedere almeno due volte al disco: una per leggere l'i-node (o indice dei blocchi) e una per leggere il blocco di dati. Posizionare gli i-node o la tabella di allocazione dei file al centro del disco riduce la distanza tra questi e i blocchi di dati a cui fanno riferimento.
    Vantaggi: Riduce il tempo di accesso ai dati poichè la distanza tra i-node e i blocchi di dati è minimizzata, migliorando l'efficienza del sistema.
  - **Partizioni Aperte**:
    L'obbiettivo è avvicinare gli i-node il più possibile hai blocchi di dati a cui fanno riferimento. Questo viene fatto dividendo il disco in gruppi di cilindri, ciascuno con i propri i-node e una lista di blocchi interi.
    Quando una partizione esaurisce i blocchi: Se una partizione esaurisce i blocchi, un blocco può essere allocato in una partizione diversa.
    Vantaggio: Migliora la localizzazione spaziale dei dati, riducendo i movimenti del braccio e migliora le prestazioni di accesso ai dati.
- **Interleave**:
  Un certo processo richieda un certo tempo Tb per richiedere e recuperare un blocco dal disco. In situazioni in cui vengono fatte richieste consecutive di blocchi sequenziali, possono verificarsi i seguenti problemi:
- **Il blocco richiesto è già passato**: Se il blocco è già passato sotto la testina di lettura/scrittura, il sistema deve attendere un'intera rotazione del disco per accedere al blocco.
- **Attesa di una rotazione completa**: Questo ritardo può rallentare significativamente il processo di accesso ai dati.

TECNICA DI Interlive

La tecnica di interleave alloca i blocchi fisici del disco in modo non strettamente sequenziale, ma in modo sequenziale con salti (fattore di interleaving). Questo significa che i blocchi che verranno letti consecutivamente non sono posizionati adiacenti l'uno all'altro fisicamente sul disco.

Vantaggi: Allocando i blocchi con interleave, il tempo di attesa per la rotazione del disco viene ridotto. La testina di lettura/scrittura ha una maggiore probabilità di trovare il blocco successivo

senza dover attendere un'intera rotazione del disco. Questa tecnica migliora l'efficienza dell'acceso ai dati, riducendo i ritardi e aumentando la velocità complessiva del file system.

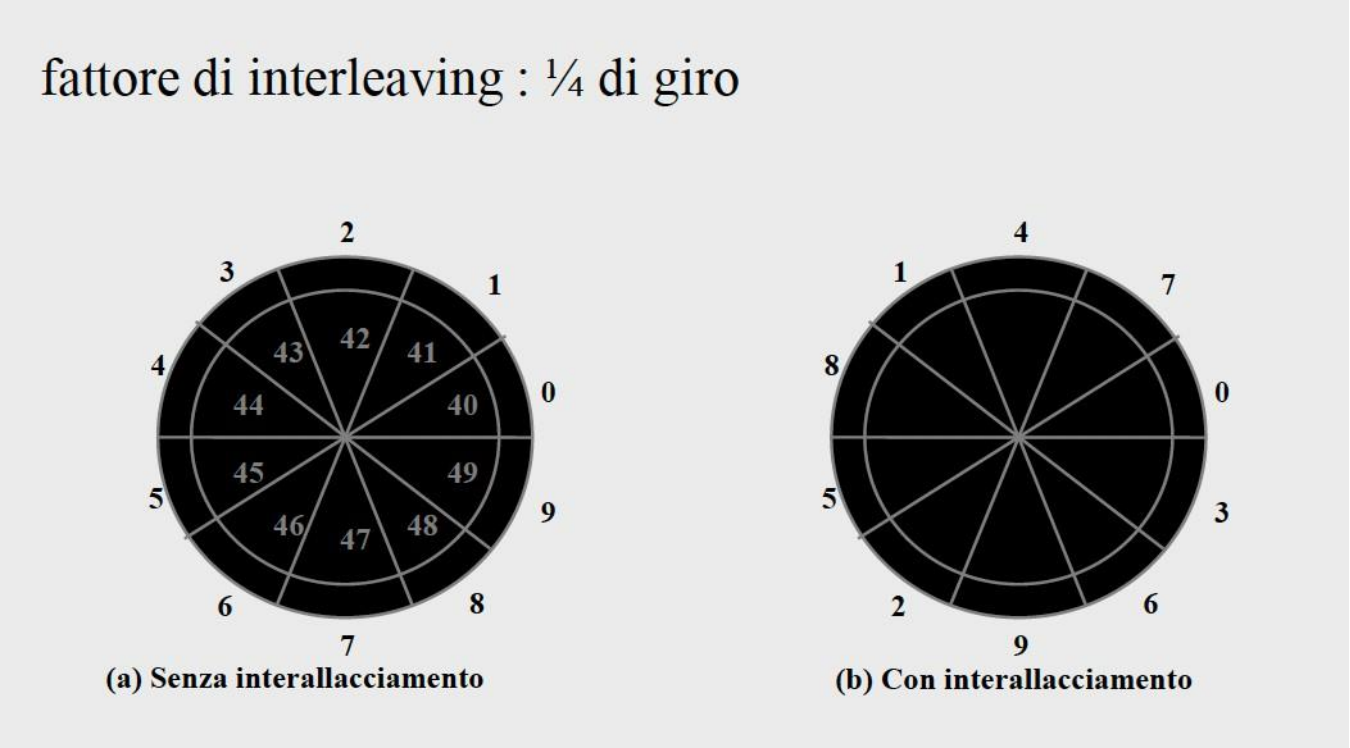


Diagramma a Sinistra: Senza Interleave

- **Descrizione:** I numeri da 0 a 49 sono disposti in sequenza attorno al cerchio, rappresentando i blocchi di dati sul disco.
- **Effetti:** Senza interleave, quando il sistema richiede blocchi consecutivi, deve spesso attendere un'intera rotazione del disco per accedere al blocco successivo.

Diagramma a Destra: Con Interleave

- **Descrizione:** I numeri da 0 a 9 sono disposti con un fattore di interleave di ¼ di giro, rendendo i blocchi non strettamente sequenziali.
- **Effetti:** Con interleave, il sistema può accedere ai blocchi successivi con meno ritardi, migliorando le prestazioni complessive.
- **Schedulazione del braccio del disco (soluzione hard)**:
  Quando un sistema deve leggere un blocco dal disco, sono coinvolti diversi tempi che influenzano la velocità complessiva dell'operazione. Ecco i tre tempi principali:
  - **Seek Time (Tempo di Posizionamento):** È il tempo necessario affinché il braccio del disco si posizioni sul cilindro corretto dove si trova il blocco desiderato.
    Il seek time può variare significativamente a seconda della distanza tra la posizione attuale del braccio e quella del cilindro desiderato. Ridurre i movimenti del braccio attraverso tecniche di ottimizzazione può migliorare le prestazioni.

- **Latency Time (Tempo di Rotazione):** È il tempo di attesa necessario affinché il settore desiderato passi sotto la testina di lettura/scrittura dopo che il braccio del disco si è posizionato sul cilindro corretto.
  - **Impatto:** La latenza dipende dalla velocità di rotazione del disco. Dischi con velocità di rotazione più elevate hanno tempi di latenza inferiori.
- **Transfer Time (Tempo di Trasferimento):** È il tempo necessario per trasferire i dati dal disco al sistema dopo che la testina di lettura/scrittura ha raggiunto il settore desiderato.
  - **Impatto:** Il transfer time dipende dalla velocità di trasferimento dei dati del disco e dalla quantità di dati da trasferire.

Le tecniche di schedulazione del braccio del disco (soluzione hard) mirano a ottimizzare l'ordine delle operazioni di lettura/scrittura per minimizzare il seek time e la latenza. Gli algoritmi utilizzati sono:

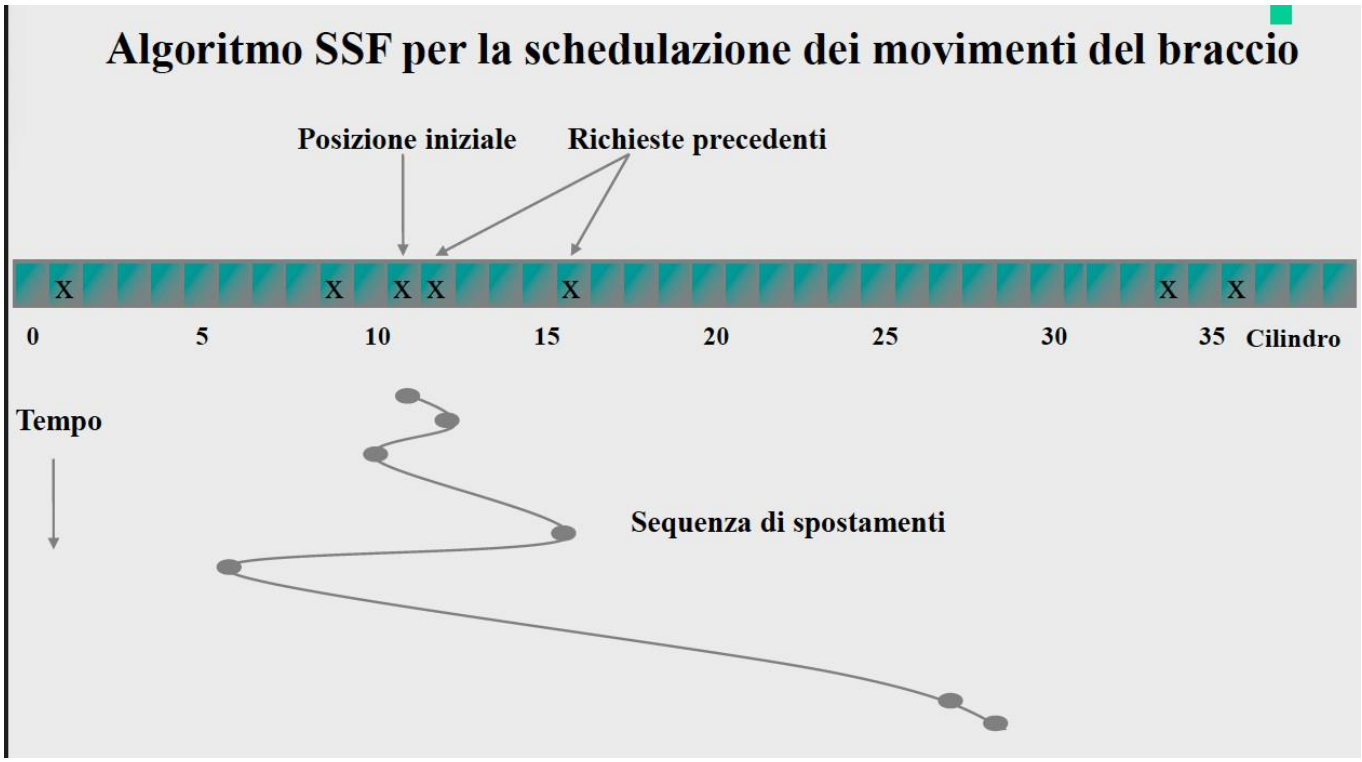
Algoritmo First Come First Served (FCFS)

- **Ordine di Arrivo:** Le richieste di accesso ai dati sul disco vengono soddisfatte nell'ordine in cui arrivano, senza alcuna priorità particolare.
- **Supporto Hardware/Software:** Non richiede supporto hardware o software specifico, rendendolo semplice da implementare.
- **Correttezza:** È considerato l'algoritmo più corretto dal punto di vista delle richieste, poiché tratta tutte le richieste con la stessa priorità.
- **Non Ottimale:** Sebbene sia corretto, non è l'algoritmo più efficiente in termini di prestazioni, poiché può causare lunghi tempi di attesa se le richieste sono molto distanti tra loro sul disco.

Shortest Seek First (SSF)

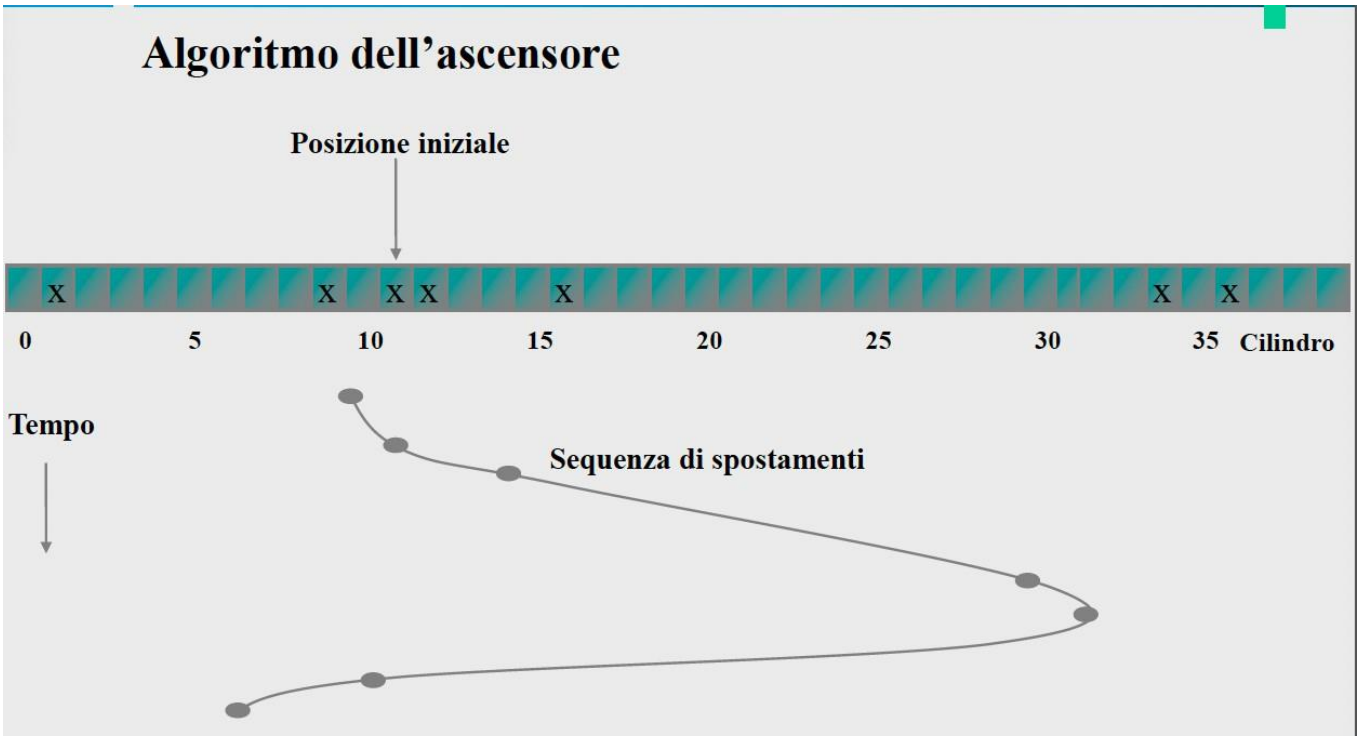
- **Tabella delle Richieste Pendenti:**
  - L'algoritmo SSF mantiene tutte le richieste pendenti in una tabella.
  - Quando deve scegliere la prossima richiesta da soddisfare, l'algoritmo seleziona quella più vicina alla posizione attuale del braccio del disco.
- **Riduzione degli Spostamenti del Braccio del Disco:**
  - Il metodo SSF riduce significativamente il numero di movimenti del braccio del disco, circa la metà rispetto al metodo FCFS (First Come First Served).
  - Questo perché il braccio si sposta solo per soddisfare la richiesta più vicina, minimizzando il tempo di seek.
- **Problema della Localizzazione delle Richieste:**
  - Un potenziale problema del SSF è che può causare una situazione nota come "starvation" in cui le richieste lontane dalla posizione attuale del braccio del disco

possono subire ritardi molto lunghi o addirittura essere ignorate se ci sono sempre richieste più vicine da soddisfare.



Algoritmo dell'Ascensore (Elevator Algorithm)

- **Tabella delle Richieste Pendenti:**
  Mantiene tutte le richieste pendenti in una tabella.
- **Direzione del Braccio del Disco**:
  Un bit up/down indica la direzione attuale del braccio del disco (verso l'alto o verso il basso).
- **Servizio delle Richieste:**
  Serve le richieste più vicine procedendo sempre nella stessa direzione fino a raggiungere l'estremità del disco.
  Questo significa che, se il braccio si sta muovendo verso l'alto, soddisferà tutte le richieste in quella direzione fino a raggiungere l'ultima traccia. Successivamente, il braccio inverte la direzione e soddisferà le richieste nella direzione opposta (verso il basso).
- **Cambio di Direzione:**
  Il braccio del disco cambia direzione quando raggiunge l'estremità del disco o quando non ci sono più richieste nella direzione attuale.



Identificazione degli Utenti

Contesto

In un sistema interattivo multi-utente, è cruciale identificare correttamente gli utenti che si connettono per garantire la sicurezza degli archivi.

Processo di Identificazione

130. **Progenitore dei Processi:**

```
- Nei sistemi multi-utente, il primo processo di ogni utente, noto
come **progenitore**, viene creato quando l'utente si connette al sistema
attraverso un terminale.
- Questo processo progenitore è responsabile di generare tutti gli altri
processi per quell'utente.
```

131. **Responsabilità del Sistema Operativo:**

```
- Per ogni processo diverso dal progenitore, il sistema operativo è
responsabile di definire correttamente il proprietario del processo.
- Questa definizione è fondamentale per garantire che ogni operazione sui
```

```
dati sia autorizzata e sicura.
```

132. **Sicurezza degli Archivi:**

```
- La sicurezza degli archivi dipende dalla corretta definizione del
proprietario del processo progenitore.
- La definizione corretta, a sua volta, dipende dalla sicura identificazione
dell'utente che si connette al sistema.
```

Tecniche di Identificazione

- **Password:**
  - La tecnica comunemente utilizzata per l'identificazione sicura degli utenti è la password.
  - Le password permettono al sistema di verificare l'identità dell'utente in modo sicuro, prevenendo accessi non autorizzati.